je voudrais créer une application complète de gestion de ressources humaines et matérielles pour une société qui gère des projets et emploi du personnel. Générez moi le Product Requirements document. cette application de ressources humaines et matérielle, devra établir les timesheets des ressources humaines et matérielles, les feuilles de paie.

J'ai préparé le PRD complet dans le canvas, structuré pour servir de base de cadrage produit, fonctionnel et technique.

Ce que le document couvre

- Vision produit et objectifs métier
- Périmètre MVP et hors périmètre
- Personae et cas d'usage
- Modules fonctionnels :
    - gestion RH
    - gestion des ressources matérielles
    - projets et affectations
    - timesheets RH et matériels
    - absences et congés
    - préparation des éléments de paie
    - reporting et workflows
- Exigences non fonctionnelles
- Règles métier
- KPIs
- Roadmap
- Critères d'acceptation
- User stories initiales

Point important

J'ai formulé le volet **paie** de manière réaliste :

- soit l'application **prépare les éléments variables de paie** pour export vers un logiciel spécialisé,
- soit elle intègre plus tard un **moteur de paie complet** si vous souhaitez couvrir toutes les règles légales et conventionnelles.

Recommandation

Pour une V1 robuste, il est préférable de lancer :

1. gestion RH,
2. gestion du matériel,
3. affectation aux projets,
4. timesheets,
5. absences,
6. préparation de paie,
7. reporting.

Ensuite, on peut transformer ce PRD en version plus opérationnelle avec :

- backlog MVP,
- user stories détaillées,
- schéma de base de données,
- spécifications API,
- cahier des charges technique.

Je peux maintenant vous générer la **version 2 du PRD orientée SaaS, avec modules, rôles, workflows, KPIs, user stories priorisées et architecture recommandée**.

ok ! allez-y !

C'est fait.

J'ai enrichi le PRD en **version 2**, avec une structure plus exploitable pour un cadrage produit et un démarrage projet.

Ce que cette version ajoute

- positionnement **SaaS**
- modules détaillés par domaine
- **personae**
- **workflows** métier
- **rôles et permissions**

- **backlog MVP priorisé**
- **user stories**
- priorisation **MoSCoW**
- **KPIs**
- architecture applicative recommandée
- modèle de données conceptuel
- premières **spécifications API** de haut niveau
- plan de livraison par phases
- recommandations produit et techniques

Prochaine étape la plus utile

Je vous recommande maintenant de passer à l'un de ces 3 livrables :

1. **Backlog produit détaillé**
   avec epics, features, user stories, critères d'acceptation
2. **Spécifications fonctionnelles détaillées**
   écran par écran, champ par champ, workflow par workflow
3. **Schéma de base de données + architecture technique**
   tables, relations, modules backend, APIs, sécurité

Je peux enchaîner directement avec le **backlog MVP détaillé en mode Agile**, prêt pour Jira/Trello/Dev.

allez -y

C'est ajouté.

J'ai intégré un **backlog MVP détaillé en mode Agile**, avec :

Contenu ajouté

- **Epics** structurés
- **Features** par domaine
- **User stories** rédigées
- **Critères d'acceptation**
- **Priorités**

- **Dépendances**
- proposition de **découpage par sprint**
- **Definition of Ready**
- **Definition of Done**

Ce que vous avez maintenant

Vous avez désormais une base solide pour :

- cadrer le produit,
- lancer les ateliers métier,
- préparer un backlog Jira,
- aligner PO, métier, UX et équipe technique.

Étape suivante recommandée

La suite la plus utile est de transformer ce backlog en **spécifications fonctionnelles détaillées**, avec :

- écrans
- champs
- règles de gestion
- validations
- statuts
- parcours utilisateurs

Je peux enchaîner directement avec la **spécification fonctionnelle détaillée du MVP, module par module**.

allez-y

C’est fait.

J’ai ajouté la **spécification fonctionnelle détaillée du MVP**, avec :

Ce qui a été structuré

- écrans principaux par module
- champs obligatoires et optionnels

- actions utilisateur
- règles de gestion
- validations
- statuts et transitions
- notifications
- cas limites à tester
- exigences de recette QA

Modules couverts

- administration & référentiels
- ressources humaines
- ressources matérielles
- projets & tâches
- planning & affectation
- timesheets RH
- usage matériel
- absences & congés
- paie préparatoire
- reporting & audit

Étape la plus logique maintenant

Le document est désormais assez mûr pour passer à un livrable d'exécution. Le plus utile serait maintenant de générer soit :

- le **schéma de base de données relationnel complet**
- les **spécifications techniques / architecture logicielle**
- les **contrats d'API REST détaillés**
- les **wireframes écran par écran**

Je recommande de commencer par le **schéma de base de données**, car il servira de fondation au backend, aux APIs et aux écrans.

voici la database de l'application CashPilot, à laquelle nous souhaitons intégrer la gestion des ressources humaines et matérielles: -- WARNING: This schema is for context only and is not meant to be run. -- Table order and constraints may not be valid for execution. CREATE TABLE

public.account_access_overrides (id uuid NOT NULL DEFAULT gen_random_uuid(), normalized_email text NOT NULL UNIQUE, access_mode text NOT NULL CHECK (access_mode = ANY (ARRAY['demo_full_access'::text, 'admin_full_access'::text])), access_label text NOT NULL, is_active boolean NOT NULL DEFAULT true, notes text, created_at timestamp with time zone NOT NULL DEFAULT timezone('utc'::text, now()), updated_at timestamp with time zone NOT NULL DEFAULT timezone('utc'::text, now()), expires_at timestamp with time zone, description text, CONSTRAINT account_access_overrides_pkey PRIMARY KEY (id)); CREATE TABLE public.accounting_account_taxonomy (id uuid NOT NULL DEFAULT gen_random_uuid(), region text NOT NULL CHECK (region = ANY (ARRAY['france'::text, 'belgium'::text, 'ohada'::text])), code_prefix text NOT NULL, semantic_role text NOT NULL, priority integer DEFAULT 0, created_at timestamp with time zone DEFAULT now(), CONSTRAINT accounting_account_taxonomy_pkey PRIMARY KEY (id)); CREATE TABLE public.accounting_analytical_axes (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, axis_type text NOT NULL CHECK (axis_type = ANY (ARRAY['cost_center'::text, 'department'::text, 'product_line'::text, 'project'::text, 'custom'::text])), axis_code text NOT NULL, axis_name text NOT NULL, color text DEFAULT '#6366f1'::text, is_active boolean DEFAULT true, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), company_id uuid NOT NULL, CONSTRAINT accounting_analytical_axes_pkey PRIMARY KEY (id), CONSTRAINT accounting_analytical_axes_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT accounting_analytical_axes_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.accounting_audit_log (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid, event_type text NOT NULL CHECK (event_type = ANY (ARRAY['auto_journal'::text, 'reversal'::text, 'balance_check'::text, 'validation_error'::text, 'manual_correction'::text, 'retroactive_journal'::text, 'supplier_journal'::text, 'bank_journal'::text, 'receivable_journal'::text, 'payable_journal'::text, 'extourne'::text, 'data_access'::text])), source_table text NOT NULL, source_id uuid, entry_count integer DEFAULT 0, total_debit numeric DEFAULT 0, total_credit numeric DEFAULT 0, balance_ok boolean DEFAULT true, details jsonb DEFAULT '{}'::jsonb, created_at timestamp with time zone DEFAULT now(), CONSTRAINT accounting_audit_log_pkey PRIMARY KEY (id), CONSTRAINT fk_accounting_audit_log_user FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.accounting_balance_checks (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, source_type text NOT NULL, source_id uuid NOT NULL UNIQUE, total_debit numeric NOT NULL DEFAULT 0, total_credit numeric NOT NULL DEFAULT 0, is_balanced boolean NOT NULL DEFAULT false, checked_at timestamp with time zone NOT NULL DEFAULT now(), details jsonb, created_at timestamp with time zone DEFAULT now(), CONSTRAINT accounting_balance_checks_pkey PRIMARY KEY (id), CONSTRAINT accounting_balance_checks_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.accounting_chart_of_accounts (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid, account_code text NOT NULL, account_name text NOT NULL, account_type text NOT NULL, account_category text, description text, is_active boolean DEFAULT true, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), parent_code text, company_id uuid NOT NULL, CONSTRAINT accounting_chart_of_accounts_pkey PRIMARY KEY (id), CONSTRAINT accounting_chart_of_accounts_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT accounting_chart_of_accounts_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.accounting_depreciation_schedule (id uuid NOT NULL DEFAULT gen_random_uuid(),

user_id uuid NOT NULL, asset_id uuid NOT NULL, period_year integer NOT NULL, period_month integer NOT NULL CHECK (period_month >= 1 AND period_month <= 12), depreciation_amount numeric NOT NULL CHECK (depreciation_amount >= 0::numeric), accumulated_depreciation numeric NOT NULL CHECK (accumulated_depreciation >= 0::numeric), net_book_value numeric NOT NULL, is_posted boolean NOT NULL DEFAULT false, entry_ref text, posted_at timestamp with time zone, created_at timestamp with time zone DEFAULT now(), company_id uuid, CONSTRAINT accounting_depreciation_schedule_pkey PRIMARY KEY (id), CONSTRAINT accounting_depreciation_schedule_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT accounting_depreciation_schedule_asset_id_fkey FOREIGN KEY (asset_id) REFERENCES public.accounting_fixed_assets(id), CONSTRAINT accounting_depreciation_schedule_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.accounting_entries (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid, transaction_date date NOT NULL, description text, reference_id uuid, reference_type text, account_code text NOT NULL, debit numeric DEFAULT 0 CHECK (COALESCE(debit, 0::numeric) >= 0::numeric), credit numeric DEFAULT 0 CHECK (COALESCE(credit, 0::numeric) >= 0::numeric), created_at timestamp with time zone DEFAULT now(), source_type text, source_id uuid, journal text DEFAULT 'OD'::text, entry_ref text, is_auto boolean DEFAULT false, cost_center text, department text, product_line text, company_id uuid, payment_instrument_id uuid, payment_transaction_id uuid, analytical_is_direct boolean, analytical_cost_behavior text CHECK (analytical_cost_behavior IS NULL OR (analytical_cost_behavior = ANY (ARRAY['fixed'::text, 'variable'::text, 'semi_variable'::text]))), analytical_destination text CHECK (analytical_destination IS NULL OR (analytical_destination = ANY (ARRAY['production'::text, 'commercial'::text, 'administratif'::text, 'rd'::text]))), analytical_method text CHECK (analytical_method IS NULL OR (analytical_method = ANY (ARRAY['full_costing'::text, 'direct_costing'::text, 'standard_costing'::text, 'abc_costing'::text, 'manual'::text]))), CONSTRAINT accounting_entries_pkey PRIMARY KEY (id), CONSTRAINT accounting_entries_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT accounting_entries_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT accounting_entries_payment_instrument_id_fkey FOREIGN KEY (payment_instrument_id) REFERENCES public.company_payment_instruments(id), CONSTRAINT fk_accounting_entries_pt_id FOREIGN KEY (payment_transaction_id) REFERENCES public.payment_transactions(id)); CREATE TABLE public.accounting_fixed_assets (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, asset_name text NOT NULL, asset_code text, acquisition_date date NOT NULL, acquisition_cost numeric NOT NULL CHECK (acquisition_cost > 0::numeric), residual_value numeric NOT NULL DEFAULT 0 CHECK (residual_value >= 0::numeric), useful_life_years integer NOT NULL CHECK (useful_life_years > 0), depreciation_method text NOT NULL DEFAULT 'linear'::text CHECK (depreciation_method = ANY (ARRAY['linear'::text, 'declining'::text])), asset_type text NOT NULL DEFAULT 'tangible'::text CHECK (asset_type = ANY (ARRAY['tangible'::text, 'intangible'::text, 'financial'::text])), category text, description text, status text NOT NULL DEFAULT 'active'::text CHECK (status = ANY (ARRAY['active'::text, 'disposed'::text, 'fully_depreciated'::text])), disposal_date date, disposal_value numeric, account_code_asset text DEFAULT '2154'::text, account_code_depreciation text DEFAULT '2815'::text, account_code_expense text DEFAULT '6811'::text, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), company_id uuid NOT NULL, CONSTRAINT accounting_fixed_assets_pkey PRIMARY KEY (id), CONSTRAINT accounting_fixed_assets_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT accounting_fixed_assets_company_id_fkey FOREIGN KEY (company_id)

REFERENCES public.company(id)); CREATE TABLE public.accounting_health (user_id uuid NOT NULL, last_check_at timestamp with time zone DEFAULT now(), is_balanced boolean DEFAULT true, last_entry_ref text, last_warning text, updated_at timestamp with time zone DEFAULT now(), CONSTRAINT accounting_health_pkey PRIMARY KEY (user_id), CONSTRAINT accounting_health_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.accounting_integration_oauth_states (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid NOT NULL, provider text NOT NULL CHECK (provider = ANY (ARRAY['xero'::text, 'quickbooks'::text])), state text NOT NULL UNIQUE, redirect_uri text NOT NULL, expires_at timestamp with time zone NOT NULL, consumed_at timestamp with time zone, metadata jsonb NOT NULL DEFAULT '{}'::jsonb, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT accounting_integration_oauth_states_pkey PRIMARY KEY (id), CONSTRAINT accounting_integration_oauth_states_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT accounting_integration_oauth_states_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.accounting_integration_tokens (id uuid NOT NULL DEFAULT gen_random_uuid(), integration_id uuid NOT NULL UNIQUE, provider text NOT NULL CHECK (provider = ANY (ARRAY['xero'::text, 'quickbooks'::text])), access_token text NOT NULL, refresh_token text, token_type text, token_scope text, expires_at timestamp with time zone, external_tenant_id text, metadata jsonb NOT NULL DEFAULT '{}'::jsonb, last_refreshed_at timestamp with time zone, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT accounting_integration_tokens_pkey PRIMARY KEY (id), CONSTRAINT accounting_integration_tokens_integration_id_fkey FOREIGN KEY (integration_id) REFERENCES public.accounting_integrations(id)); CREATE TABLE public.accounting_integrations (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid NOT NULL, provider text NOT NULL CHECK (provider = ANY (ARRAY['xero'::text, 'quickbooks'::text])), status text NOT NULL DEFAULT 'disconnected'::text CHECK (status = ANY (ARRAY['disconnected'::text, 'pending'::text, 'connected'::text, 'error'::text])), external_tenant_id text, external_company_name text, sync_enabled boolean NOT NULL DEFAULT true, last_sync_at timestamp with time zone, last_error text, metadata jsonb NOT NULL DEFAULT '{}'::jsonb, connected_at timestamp with time zone, disconnected_at timestamp with time zone, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT accounting_integrations_pkey PRIMARY KEY (id), CONSTRAINT accounting_integrations_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT accounting_integrations_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.accounting_journals (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid, code text NOT NULL, name text NOT NULL, journal_type text NOT NULL CHECK (journal_type = ANY (ARRAY['sales'::text, 'purchases'::text, 'bank'::text, 'cash'::text, 'misc'::text, 'payroll'::text])), is_active boolean DEFAULT true, created_at timestamp with time zone DEFAULT now(), CONSTRAINT accounting_journals_pkey PRIMARY KEY (id), CONSTRAINT accounting_journals_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT accounting_journals_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.accounting_mapping_templates (id uuid NOT NULL DEFAULT gen_random_uuid(), country_code text NOT NULL, source_type text NOT NULL, source_category text NOT NULL, debit_account_code text NOT NULL, credit_account_code text NOT NULL, description text, created_at timestamp with time zone

NOT NULL DEFAULT now(), CONSTRAINT accounting_mapping_templates_pkey PRIMARY KEY (id)); CREATE TABLE public.accounting_mappings (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid, mapping_name text, description text, source_type text NOT NULL, source_category text, debit_account_code text, credit_account_code text, is_active boolean DEFAULT true, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), CONSTRAINT accounting_mappings_pkey PRIMARY KEY (id), CONSTRAINT accounting_mappings_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.accounting_plan_accounts (id uuid NOT NULL DEFAULT gen_random_uuid(), plan_id uuid NOT NULL, account_code text NOT NULL, account_name text NOT NULL, account_type text NOT NULL CHECK (account_type = ANY (ARRAY['asset'::text, 'liability'::text, 'equity'::text, 'revenue'::text, 'expense'::text])), account_category text, parent_code text, description text, is_header boolean DEFAULT false, sort_order integer DEFAULT 0, CONSTRAINT accounting_plan_accounts_pkey PRIMARY KEY (id), CONSTRAINT accounting_plan_accounts_plan_id_fkey FOREIGN KEY (plan_id) REFERENCES public.accounting_plans(id)); CREATE TABLE public.accounting_plans (id uuid NOT NULL DEFAULT gen_random_uuid(), name text NOT NULL, country_code text, description text, source text DEFAULT 'system'::text CHECK (source = ANY (ARRAY['system'::text, 'user_upload'::text, 'admin'::text])), uploaded_by uuid, is_global boolean DEFAULT false, file_url text, accounts_count integer DEFAULT 0, status text DEFAULT 'active'::text CHECK (status = ANY (ARRAY['active'::text, 'pending_review'::text, 'archived'::text])), created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), CONSTRAINT accounting_plans_pkey PRIMARY KEY (id), CONSTRAINT accounting_plans_uploaded_by_fkey FOREIGN KEY (uploaded_by) REFERENCES auth.users(id)); CREATE TABLE public.accounting_sync_logs (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid NOT NULL, integration_id uuid, provider text NOT NULL CHECK (provider = ANY (ARRAY['xero'::text, 'quickbooks'::text])), status text NOT NULL CHECK (status = ANY (ARRAY['queued'::text, 'running'::text, 'success'::text, 'failed'::text])), message text, details jsonb NOT NULL DEFAULT '{}'::jsonb, started_at timestamp with time zone, finished_at timestamp with time zone, created_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT accounting_sync_logs_pkey PRIMARY KEY (id), CONSTRAINT accounting_sync_logs_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT accounting_sync_logs_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT accounting_sync_logs_integration_id_fkey FOREIGN KEY (integration_id) REFERENCES public.accounting_integrations(id)); CREATE TABLE public.accounting_tax_rate_templates (id uuid NOT NULL DEFAULT gen_random_uuid(), country_code text NOT NULL, name text NOT NULL, rate numeric NOT NULL, tax_type text NOT NULL, account_code text NOT NULL, is_default boolean NOT NULL DEFAULT false, created_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT accounting_tax_rate_templates_pkey PRIMARY KEY (id)); CREATE TABLE public.accounting_tax_rates (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid, account_code text, is_active boolean DEFAULT true, effective_date date DEFAULT CURRENT_DATE, created_at timestamp with time zone DEFAULT now(), name text, rate numeric, tax_type text DEFAULT 'output'::text, is_default boolean DEFAULT false, CONSTRAINT accounting_tax_rates_pkey PRIMARY KEY (id), CONSTRAINT accounting_tax_rates_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.analytical_allocation_rules (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid NOT NULL, rule_name text NOT NULL, source_type text, source_category text, priority integer NOT NULL DEFAULT 100,

object_id uuid, cost_center_id uuid, axis_value_id uuid, allocation_percent numeric NOT NULL DEFAULT 100 CHECK (allocation_percent > 0::numeric AND allocation_percent <= 100::numeric), default_is_direct boolean, default_cost_behavior text CHECK (default_cost_behavior IS NULL OR (default_cost_behavior = ANY (ARRAY['fixed'::text, 'variable'::text, 'semi_variable'::text]))), default_destination text CHECK (default_destination IS NULL OR (default_destination = ANY (ARRAY['production'::text, 'commercial'::text, 'administratif'::text, 'rd'::text]))), default_method text CHECK (default_method IS NULL OR (default_method = ANY (ARRAY['full_costing'::text, 'direct_costing'::text, 'standard_costing'::text, 'abc_costing'::text, 'manual'::text]))), is_active boolean NOT NULL DEFAULT true, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT analytical_allocation_rules_pkey PRIMARY KEY (id), CONSTRAINT analytical_allocation_rules_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT analytical_allocation_rules_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT analytical_allocation_rules_object_id_fkey FOREIGN KEY (object_id) REFERENCES public.analytical_objects(id), CONSTRAINT analytical_allocation_rules_cost_center_id_fkey FOREIGN KEY (cost_center_id) REFERENCES public.cost_centers(id), CONSTRAINT analytical_allocation_rules_axis_value_id_fkey FOREIGN KEY (axis_value_id) REFERENCES public.analytical_axis_values(id)); CREATE TABLE public.analytical_allocations (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid NOT NULL, entry_id uuid NOT NULL, object_id uuid, cost_center_id uuid, axis_value_id uuid, redistribution_rule_id uuid, redistributed_from_allocation_id uuid, amount numeric NOT NULL CHECK (amount >= 0::numeric), allocation_percent numeric CHECK (allocation_percent IS NULL OR allocation_percent >= 0::numeric AND allocation_percent <= 100::numeric), is_direct boolean, cost_behavior text CHECK (cost_behavior IS NULL OR (cost_behavior = ANY (ARRAY['fixed'::text, 'variable'::text, 'semi_variable'::text]))), destination text CHECK (destination IS NULL OR (destination = ANY (ARRAY['production'::text, 'commercial'::text, 'administratif'::text, 'rd'::text]))), method text CHECK (method IS NULL OR (method = ANY (ARRAY['full_costing'::text, 'direct_costing'::text, 'standard_costing'::text, 'abc_costing'::text, 'manual'::text]))), notes text, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT analytical_allocations_pkey PRIMARY KEY (id), CONSTRAINT analytical_allocations_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT analytical_allocations_entry_id_fkey FOREIGN KEY (entry_id) REFERENCES public.accounting_entries(id), CONSTRAINT analytical_allocations_object_id_fkey FOREIGN KEY (object_id) REFERENCES public.analytical_objects(id), CONSTRAINT analytical_allocations_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT analytical_allocations_cost_center_id_fkey FOREIGN KEY (cost_center_id) REFERENCES public.cost_centers(id), CONSTRAINT analytical_allocations_axis_value_id_fkey FOREIGN KEY (axis_value_id) REFERENCES public.analytical_axis_values(id), CONSTRAINT analytical_allocations_redistribution_rule_id_fkey FOREIGN KEY (redistribution_rule_id) REFERENCES public.center_redistribution_rules(id), CONSTRAINT analytical_allocations_redistributed_from_allocation_id_fkey FOREIGN KEY (redistributed_from_allocation_id) REFERENCES public.analytical_allocations(id)); CREATE TABLE public.analytical_axis_values (id uuid NOT NULL DEFAULT gen_random_uuid(), axis_id uuid NOT NULL, user_id uuid NOT NULL, company_id uuid NOT NULL, value_code text NOT NULL, value_name text NOT NULL, color text DEFAULT '#22c55e'::text, metadata jsonb NOT NULL DEFAULT '{}'::jsonb, is_active boolean NOT NULL DEFAULT true, created_at

timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT analytical_axis_values_pkey PRIMARY KEY (id), CONSTRAINT analytical_axis_values_axis_id_fkey FOREIGN KEY (axis_id) REFERENCES public.accounting_analytical_axes(id), CONSTRAINT analytical_axis_values_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT analytical_axis_values_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.analytical_budget_lines (id uuid NOT NULL DEFAULT gen_random_uuid(), budget_id uuid NOT NULL, user_id uuid NOT NULL, company_id uuid NOT NULL, period_month date NOT NULL, planned_amount numeric NOT NULL DEFAULT 0, planned_volume numeric, planned_unit_cost numeric, notes text, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT analytical_budget_lines_pkey PRIMARY KEY (id), CONSTRAINT analytical_budget_lines_budget_id_fkey FOREIGN KEY (budget_id) REFERENCES public.analytical_budgets(id), CONSTRAINT analytical_budget_lines_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT analytical_budget_lines_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.analytical_budget_scenarios (id uuid NOT NULL DEFAULT gen_random_uuid(), budget_id uuid NOT NULL, user_id uuid NOT NULL, company_id uuid NOT NULL, scenario_name text NOT NULL, revenue_growth_percent numeric NOT NULL DEFAULT 6, cost_optimization_percent numeric NOT NULL DEFAULT 3, risk_percent numeric NOT NULL DEFAULT 2, notes text, metadata jsonb NOT NULL DEFAULT '{}'::jsonb, is_default boolean NOT NULL DEFAULT false, is_active boolean NOT NULL DEFAULT true, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT analytical_budget_scenarios_pkey PRIMARY KEY (id), CONSTRAINT analytical_budget_scenarios_budget_id_fkey FOREIGN KEY (budget_id) REFERENCES public.analytical_budgets(id), CONSTRAINT analytical_budget_scenarios_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT analytical_budget_scenarios_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.analytical_budgets (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid NOT NULL, budget_name text NOT NULL, period_start date NOT NULL, period_end date NOT NULL, method text NOT NULL DEFAULT 'full_costing'::text CHECK (method = ANY (ARRAY['full_costing'::text, 'direct_costing'::text, 'standard_costing'::text, 'abc_costing'::text])), object_id uuid, cost_center_id uuid, axis_value_id uuid, metadata jsonb NOT NULL DEFAULT '{}'::jsonb, is_active boolean NOT NULL DEFAULT true, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT analytical_budgets_pkey PRIMARY KEY (id), CONSTRAINT analytical_budgets_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT analytical_budgets_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT analytical_budgets_object_id_fkey FOREIGN KEY (object_id) REFERENCES public.analytical_objects(id), CONSTRAINT analytical_budgets_cost_center_id_fkey FOREIGN KEY (cost_center_id) REFERENCES public.cost_centers(id), CONSTRAINT analytical_budgets_axis_value_id_fkey FOREIGN KEY (axis_value_id) REFERENCES public.analytical_axis_values(id)); CREATE TABLE public.analytical_inclusion_rules (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid NOT NULL, rule_name text NOT NULL, rule_type text NOT NULL CHECK (rule_type = ANY (ARRAY['non_incorporable'::text, 'suppletive'::text])), source_type text, source_category text, account_code_pattern text, supplementive_amount numeric, supplementive_label text, is_active boolean NOT NULL DEFAULT true, created_at

timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT analytical_inclusion_rules_pkey PRIMARY KEY (id), CONSTRAINT analytical_inclusion_rules_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT analytical_inclusion_rules_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.analytical_objects (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid NOT NULL, object_type text NOT NULL CHECK (object_type = ANY (ARRAY['product'::text, 'service'::text, 'project'::text, 'client'::text, 'channel'::text, 'geography'::text, 'business_unit'::text, 'custom'::text])), object_code text NOT NULL, object_name text NOT NULL, source_table text, source_id uuid, metadata jsonb NOT NULL DEFAULT '{}'::jsonb, is_active boolean NOT NULL DEFAULT true, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT analytical_objects_pkey PRIMARY KEY (id), CONSTRAINT analytical_objects_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT analytical_objects_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.api_keys (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, key_hash text NOT NULL UNIQUE, key_prefix text NOT NULL, name text DEFAULT 'Default'::text, is_active boolean DEFAULT true, expires_at timestamp with time zone, last_used_at timestamp with time zone, created_at timestamp with time zone DEFAULT now(), scopes ARRAY DEFAULT ARRAY['read'::text, 'write'::text, 'delete'::text], key_version integer DEFAULT 1, superseded_at timestamp with time zone, superseded_by uuid, grace_period_days integer DEFAULT 7, CONSTRAINT api_keys_pkey PRIMARY KEY (id), CONSTRAINT api_keys_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT api_keys_superseded_by_fkey FOREIGN KEY (superseded_by) REFERENCES public.api_keys(id)); CREATE TABLE public.api_rate_limit_counters (scope text NOT NULL, rate_key text NOT NULL, window_seconds integer NOT NULL CHECK (window_seconds > 0), bucket_start timestamp with time zone NOT NULL, request_count integer NOT NULL DEFAULT 0 CHECK (request_count >= 0), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT api_rate_limit_counters_pkey PRIMARY KEY (scope, rate_key, window_seconds, bucket_start)); CREATE TABLE public.audit_log (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid, action text NOT NULL, details jsonb, created_at timestamp with time zone DEFAULT now(), CONSTRAINT audit_log_pkey PRIMARY KEY (id), CONSTRAINT audit_log_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.auth_security_locks (scope text NOT NULL CHECK (scope = ANY (ARRAY['sign-in'::text, 'sign-up'::text, 'mfa-verify'::text])), rate_key text NOT NULL, failed_attempts integer NOT NULL DEFAULT 0 CHECK (failed_attempts >= 0), lock_until timestamp with time zone, last_failure_at timestamp with time zone, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT auth_security_locks_pkey PRIMARY KEY (scope, rate_key)); CREATE TABLE public.backup_logs (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, provider text NOT NULL, status text NOT NULL CHECK (status = ANY (ARRAY['success'::text, 'failed'::text, 'in_progress'::text])), file_name text, file_size_bytes integer, error_message text, started_at timestamp with time zone DEFAULT now(), completed_at timestamp with time zone, metadata jsonb DEFAULT '{}'::jsonb, CONSTRAINT backup_logs_pkey PRIMARY KEY (id), CONSTRAINT backup_logs_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.backup_settings (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL UNIQUE, provider text DEFAULT 'none'::text CHECK (provider = ANY (ARRAY['none'::text, 'google_drive'::text, 'dropbox'::text])), is_enabled boolean DEFAULT false, frequency text DEFAULT 'weekly'::text

CHECK (frequency = ANY (ARRAY['daily'::text, 'weekly'::text, 'monthly'::text])), last_backup_at timestamp with time zone, next_backup_at timestamp with time zone, access_token text, refresh_token text, token_expires_at timestamp with time zone, folder_id text, folder_name text DEFAULT 'CashPilot Backups'::text, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), CONSTRAINT backup_settings_pkey PRIMARY KEY (id), CONSTRAINT backup_settings_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.bank_connections (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, institution_id text NOT NULL, institution_name text NOT NULL, institution_logo text, requisition_id text, agreement_id text, status text DEFAULT 'active'::text CHECK (status = ANY (ARRAY['pending'::text, 'active'::text, 'expired'::text, 'revoked'::text, 'error'::text])), last_sync_at timestamp with time zone, sync_error text, account_id text, account_iban text, account_name text, account_currency text DEFAULT 'EUR'::text, account_balance numeric, expires_at timestamp with time zone, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), company_id uuid NOT NULL, payment_instrument_id uuid, CONSTRAINT bank_connections_pkey PRIMARY KEY (id), CONSTRAINT bank_connections_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT bank_connections_payment_instrument_id_fkey FOREIGN KEY (payment_instrument_id) REFERENCES public.company_payment_instruments(id), CONSTRAINT bank_connections_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.bank_reconciliation_sessions (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, statement_id uuid NOT NULL, session_date timestamp with time zone DEFAULT now(), status text NOT NULL DEFAULT 'in_progress'::text CHECK (status = ANY (ARRAY['in_progress'::text, 'completed'::text, 'archived'::text])), total_lines integer DEFAULT 0, matched_lines integer DEFAULT 0, unmatched_lines integer DEFAULT 0, ignored_lines integer DEFAULT 0, total_credits numeric DEFAULT 0, total_debits numeric DEFAULT 0, matched_credits numeric DEFAULT 0, matched_debits numeric DEFAULT 0, difference numeric DEFAULT 0, completed_at timestamp with time zone, notes text, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), CONSTRAINT bank_reconciliation_sessions_pkey PRIMARY KEY (id), CONSTRAINT bank_reconciliation_sessions_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT bank_reconciliation_sessions_statement_id_fkey FOREIGN KEY (statement_id) REFERENCES public.bank_statements(id)); CREATE TABLE public.bank_statement_lines (id uuid NOT NULL DEFAULT gen_random_uuid(), statement_id uuid NOT NULL, user_id uuid NOT NULL, line_number integer, transaction_date date NOT NULL, value_date date, description text, reference text, amount numeric NOT NULL, balance_after numeric, raw_data jsonb, reconciliation_status text NOT NULL DEFAULT 'unmatched'::text CHECK (reconciliation_status = ANY (ARRAY['unmatched'::text, 'matched'::text, 'ignored'::text])), matched_source_type text CHECK (matched_source_type IS NULL OR (matched_source_type = ANY (ARRAY['invoice'::text, 'expense'::text, 'supplier_invoice'::text, 'manual'::text]))), matched_source_id uuid, matched_at timestamp with time zone, matched_by text DEFAULT 'manual'::text CHECK (matched_by = ANY (ARRAY['auto'::text, 'manual'::text])), match_confidence numeric, notes text, created_at timestamp with time zone DEFAULT now(), CONSTRAINT bank_statement_lines_pkey PRIMARY KEY (id), CONSTRAINT bank_statement_lines_statement_id_fkey FOREIGN KEY (statement_id) REFERENCES public.bank_statements(id), CONSTRAINT bank_statement_lines_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.bank_statements (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, bank_name text, account_number text, statement_date date, period_start date, period_end date,

opening_balance numeric, closing_balance numeric, file_name text NOT NULL, file_path text NOT NULL, file_type text NOT NULL CHECK (file_type = ANY (ARRAY['pdf'::text, 'xlsx'::text, 'xls'::text, 'csv'::text])), file_size integer, parse_status text NOT NULL DEFAULT 'pending'::text CHECK (parse_status = ANY (ARRAY['pending'::text, 'parsed'::text, 'error'::text, 'confirmed'::text])), parse_errors jsonb DEFAULT '[]'::jsonb, line_count integer DEFAULT 0, notes text, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), payment_instrument_id uuid, CONSTRAINT bank_statements_pkey PRIMARY KEY (id), CONSTRAINT bank_statements_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT bank_statements_payment_instrument_id_fkey FOREIGN KEY (payment_instrument_id) REFERENCES public.company_payment_instruments(id)); CREATE TABLE public.bank_sync_history (id uuid NOT NULL DEFAULT gen_random_uuid(), bank_connection_id uuid NOT NULL, user_id uuid NOT NULL, sync_type text DEFAULT 'transactions'::text CHECK (sync_type = ANY (ARRAY['transactions'::text, 'balance'::text, 'full'::text])), status text DEFAULT 'success'::text CHECK (status = ANY (ARRAY['success'::text, 'partial'::text, 'error'::text])), transactions_synced integer DEFAULT 0, error_message text, started_at timestamp with time zone DEFAULT now(), completed_at timestamp with time zone, company_id uuid, CONSTRAINT bank_sync_history_pkey PRIMARY KEY (id), CONSTRAINT bank_sync_history_bank_connection_id_fkey FOREIGN KEY (bank_connection_id) REFERENCES public.bank_connections(id), CONSTRAINT fk_bank_sync_history_user FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT bank_sync_history_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.bank_transactions (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, bank_connection_id uuid NOT NULL, external_id text, date date NOT NULL, booking_date date, value_date date, amount numeric NOT NULL, currency text DEFAULT 'EUR'::text, description text, reference text, creditor_name text, debtor_name text, remittance_info text, invoice_id uuid, reconciliation_status text DEFAULT 'unreconciled'::text CHECK (reconciliation_status = ANY (ARRAY['unreconciled'::text, 'matched'::text, 'ignored'::text])), match_confidence numeric, matched_at timestamp with time zone, raw_data jsonb, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), company_id uuid, payment_instrument_id uuid, payment_transaction_id uuid, CONSTRAINT bank_transactions_pkey PRIMARY KEY (id), CONSTRAINT bank_transactions_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT bank_transactions_bank_connection_id_fkey FOREIGN KEY (bank_connection_id) REFERENCES public.bank_connections(id), CONSTRAINT bank_transactions_invoice_id_fkey FOREIGN KEY (invoice_id) REFERENCES public.invoices(id), CONSTRAINT bank_transactions_payment_instrument_id_fkey FOREIGN KEY (payment_instrument_id) REFERENCES public.company_payment_instruments(id), CONSTRAINT bank_transactions_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT fk_bank_transactions_pt_id FOREIGN KEY (payment_transaction_id) REFERENCES public.payment_transactions(id)); CREATE TABLE public.barcode_scan_logs (id uuid NOT NULL DEFAULT gen_random_uuid(), product_id uuid, user_id uuid, barcode text NOT NULL, scan_timestamp timestamp with time zone DEFAULT now(), location text, quantity numeric DEFAULT 1, action text, CONSTRAINT barcode_scan_logs_pkey PRIMARY KEY (id), CONSTRAINT barcode_scan_logs_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT barcode_scan_logs_product_id_fkey FOREIGN KEY (product_id) REFERENCES public.products(id)); CREATE TABLE public.billing_info (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL UNIQUE, company_name text, address text, city text, postal_code text, country text, vat_number text, siret text, plan text DEFAULT 'free'::text

CHECK (plan = ANY (ARRAY['free'::text, 'starter'::text, 'pro'::text, 'enterprise'::text])), plan_price numeric DEFAULT 0, plan_interval text DEFAULT 'month'::text CHECK (plan_interval = ANY (ARRAY['month'::text, 'year'::text])), next_billing_date date, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), CONSTRAINT billing_info_pkey PRIMARY KEY (id), CONSTRAINT billing_info_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.biometric_credentials (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid, biometric_type text NOT NULL, device_id text, public_key text NOT NULL, created_at timestamp with time zone DEFAULT now(), last_used_at timestamp with time zone, is_active boolean DEFAULT true, CONSTRAINT biometric_credentials_pkey PRIMARY KEY (id), CONSTRAINT biometric_credentials_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.center_redistribution_rules (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid NOT NULL, from_center_id uuid NOT NULL, to_center_id uuid NOT NULL, allocation_percent numeric NOT NULL CHECK (allocation_percent > 0::numeric AND allocation_percent <= 100::numeric), is_active boolean NOT NULL DEFAULT true, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT center_redistribution_rules_pkey PRIMARY KEY (id), CONSTRAINT center_redistribution_rules_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT center_redistribution_rules_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT center_redistribution_rules_from_center_id_fkey FOREIGN KEY (from_center_id) REFERENCES public.cost_centers(id), CONSTRAINT center_redistribution_rules_to_center_id_fkey FOREIGN KEY (to_center_id) REFERENCES public.cost_centers(id)); CREATE TABLE public.clients (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid, company_name text NOT NULL, contact_name text, email text, address text, vat_number text, preferred_currency text DEFAULT 'EUR'::text, created_at timestamp with time zone DEFAULT now(), phone text, website text, city text, postal_code text, country text, payment_terms text, tax_id text, bank_name text, iban text, bic_swift text, notes text, updated_at timestamp with time zone DEFAULT now(), deleted_at timestamp with time zone, peppol_endpoint_id text, peppol_scheme_id text DEFAULT '0208'::text, electronic_invoicing_enabled boolean DEFAULT false, company_id uuid NOT NULL, CONSTRAINT clients_pkey PRIMARY KEY (id), CONSTRAINT clients_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT clients_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.company (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_name text, company_type text CHECK (company_type = ANY (ARRAY['freelance'::text, 'company'::text])), registration_number text, tax_id text, address text, city text, postal_code text, country text, phone text, email text, website text, logo_url text, bank_account text, bank_name text, iban text, swift text, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), currency character varying DEFAULT 'EUR'::character varying, peppol_endpoint_id text, peppol_scheme_id text DEFAULT '0208'::text, peppol_ap_provider text DEFAULT 'scrada'::text, scrada_company_id text, scrada_api_key text, scrada_password text, accounting_currency character varying NOT NULL DEFAULT 'EUR'::character varying CHECK (accounting_currency::text ~ '^[A-Z]{3}$'::text), peppol_config_signature text, peppol_config_validated_at timestamp with time zone, portfolio_id uuid, scrada_api_key_encrypted text, scrada_password_encrypted text, CONSTRAINT company_pkey PRIMARY KEY (id), CONSTRAINT company_portfolio_id_fkey FOREIGN KEY (portfolio_id) REFERENCES public.company_portfolios(id), CONSTRAINT fk_company_user FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE

public.company_esign_settings (company_id uuid NOT NULL, user_id uuid NOT NULL, provider text NOT NULL DEFAULT 'native'::text CHECK (provider = ANY (ARRAY['native'::text, 'yousign'::text, 'docusign'::text])), mode text NOT NULL DEFAULT 'redirect'::text CHECK (mode = ANY (ARRAY['redirect'::text, 'embedded'::text])), provider_account_id text, webhook_secret text, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT company_esign_settings_pkey PRIMARY KEY (company_id), CONSTRAINT company_esign_settings_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT company_esign_settings_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.company_payment_instruments (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid NOT NULL, portfolio_id uuid, instrument_type text NOT NULL CHECK (instrument_type = ANY (ARRAY['bank_account'::text, 'card'::text, 'cash'::text])), instrument_subtype text CHECK (instrument_subtype = ANY (ARRAY['checking'::text, 'savings'::text, 'credit_card'::text, 'debit_card'::text, 'petty_cash'::text, 'cash_register'::text, 'mobile_money'::text, 'other'::text])), code text NOT NULL, label text NOT NULL, display_name text, description text, currency character varying NOT NULL DEFAULT 'EUR'::character varying CHECK (currency::text ~ '^[A-Z]{3}$'::text), status text NOT NULL DEFAULT 'active'::text CHECK (status = ANY (ARRAY['active'::text, 'inactive'::text, 'archived'::text, 'blocked'::text])), is_default boolean NOT NULL DEFAULT false, allow_incoming boolean NOT NULL DEFAULT true, allow_outgoing boolean NOT NULL DEFAULT true, include_in_dashboard boolean NOT NULL DEFAULT true, opening_balance numeric NOT NULL DEFAULT 0, current_balance numeric NOT NULL DEFAULT 0, account_code text, journal_code text, external_provider text, external_reference text, metadata jsonb NOT NULL DEFAULT '{}'::jsonb, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), archived_at timestamp with time zone, CONSTRAINT company_payment_instruments_pkey PRIMARY KEY (id), CONSTRAINT company_payment_instruments_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT company_payment_instruments_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT company_payment_instruments_portfolio_id_fkey FOREIGN KEY (portfolio_id) REFERENCES public.company_portfolios(id)); CREATE TABLE public.company_portfolio_members (id uuid NOT NULL DEFAULT gen_random_uuid(), portfolio_id uuid NOT NULL, company_id uuid NOT NULL, user_id uuid NOT NULL, created_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT company_portfolio_members_pkey PRIMARY KEY (id), CONSTRAINT company_portfolio_members_portfolio_id_fkey FOREIGN KEY (portfolio_id) REFERENCES public.company_portfolios(id), CONSTRAINT company_portfolio_members_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT company_portfolio_members_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.company_portfolios (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, portfolio_name text NOT NULL, description text, base_currency character varying NOT NULL DEFAULT 'EUR'::character varying CHECK (base_currency::text ~ '^[A-Z]{3}$'::text), is_default boolean NOT NULL DEFAULT false, is_active boolean NOT NULL DEFAULT true, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT company_portfolios_pkey PRIMARY KEY (id), CONSTRAINT company_portfolios_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.company_security_settings (id uuid NOT NULL DEFAULT gen_random_uuid(), company_id uuid NOT NULL UNIQUE, user_id uuid NOT NULL, sso_enforced boolean NOT NULL DEFAULT false, sso_provider text NOT NULL DEFAULT

'none'::text CHECK (sso_provider = ANY (ARRAY['none'::text, 'saml'::text, 'oidc'::text])), saml_entry_point text, saml_issuer text, saml_certificate text, oidc_issuer text, oidc_client_id text, allowed_email_domains ARRAY NOT NULL DEFAULT '{}'::text[], session_timeout_minutes integer NOT NULL DEFAULT 480 CHECK (session_timeout_minutes >= 15 AND session_timeout_minutes <= 1440), mfa_required boolean NOT NULL DEFAULT false, ip_allowlist ARRAY NOT NULL DEFAULT '{}'::cidr[], audit_webhook_url text, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT company_security_settings_pkey PRIMARY KEY (id), CONSTRAINT company_security_settings_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT company_security_settings_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.consent_logs (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, consent_type text NOT NULL CHECK (consent_type = ANY (ARRAY['cookies'::text, 'analytics'::text, 'marketing'::text, 'necessary'::text])), granted boolean NOT NULL DEFAULT false, ip_address text, user_agent text, granted_at timestamp with time zone DEFAULT now(), revoked_at timestamp with time zone, created_at timestamp with time zone DEFAULT now(), CONSTRAINT consent_logs_pkey PRIMARY KEY (id), CONSTRAINT consent_logs_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.cost_centers (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid NOT NULL, axis_id uuid, parent_center_id uuid, center_code text NOT NULL, center_name text NOT NULL, center_type text NOT NULL CHECK (center_type = ANY (ARRAY['principal'::text, 'auxiliary'::text, 'structure'::text])), allocation_base text, metadata jsonb NOT NULL DEFAULT '{}'::jsonb, is_active boolean NOT NULL DEFAULT true, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT cost_centers_pkey PRIMARY KEY (id), CONSTRAINT cost_centers_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT cost_centers_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT cost_centers_axis_id_fkey FOREIGN KEY (axis_id) REFERENCES public.accounting_analytical_axes(id), CONSTRAINT cost_centers_parent_center_id_fkey FOREIGN KEY (parent_center_id) REFERENCES public.cost_centers(id)); CREATE TABLE public.credit_costs (id uuid NOT NULL DEFAULT gen_random_uuid(), operation_code text NOT NULL UNIQUE, operation_name text NOT NULL, cost integer NOT NULL DEFAULT 1 CHECK (cost > 0), category text NOT NULL, is_active boolean DEFAULT true, created_at timestamp with time zone DEFAULT now(), CONSTRAINT credit_costs_pkey PRIMARY KEY (id)); CREATE TABLE public.credit_note_items (id uuid NOT NULL DEFAULT gen_random_uuid(), credit_note_id uuid NOT NULL, description text NOT NULL, quantity numeric DEFAULT 1, unit_price numeric DEFAULT 0, amount numeric DEFAULT 0, created_at timestamp with time zone DEFAULT now(), CONSTRAINT credit_note_items_pkey PRIMARY KEY (id), CONSTRAINT credit_note_items_credit_note_id_fkey FOREIGN KEY (credit_note_id) REFERENCES public.credit_notes(id)); CREATE TABLE public.credit_notes (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, credit_note_number text NOT NULL, invoice_id uuid, client_id uuid, date date NOT NULL DEFAULT CURRENT_DATE, reason text DEFAULT ''::text, total_ht numeric DEFAULT 0, tax_rate numeric DEFAULT 0, tax_amount numeric DEFAULT 0, total_ttc numeric DEFAULT 0 CHECK (total_ttc >= 0::numeric), status text DEFAULT 'draft'::text CHECK (status = ANY (ARRAY['draft'::text, 'issued'::text, 'applied'::text, 'cancelled'::text])), notes text DEFAULT ''::text, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), company_id uuid NOT NULL, file_url text, file_generated_at timestamp with time zone, CONSTRAINT credit_notes_pkey

PRIMARY KEY (id), CONSTRAINT credit_notes_client_id_fkey FOREIGN KEY (client_id) REFERENCES public.clients(id), CONSTRAINT credit_notes_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT fk_credit_notes_invoice FOREIGN KEY (invoice_id) REFERENCES public.invoices(id), CONSTRAINT credit_notes_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.credit_packages (id uuid NOT NULL DEFAULT gen_random_uuid(), name text NOT NULL, credits integer NOT NULL, price_cents integer NOT NULL, currency text DEFAULT 'EUR'::text, is_active boolean DEFAULT true, stripe_price_id text, sort_order integer DEFAULT 0, created_at timestamp with time zone DEFAULT now(), CONSTRAINT credit_packages_pkey PRIMARY KEY (id)); CREATE TABLE public.credit_transactions (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, type text NOT NULL CHECK (type = ANY (ARRAY['purchase'::text, 'usage'::text, 'bonus'::text, 'refund'::text])), amount integer NOT NULL, description text, stripe_session_id text, stripe_payment_intent text, metadata jsonb DEFAULT '{}'::jsonb, created_at timestamp with time zone DEFAULT now(), CONSTRAINT credit_transactions_pkey PRIMARY KEY (id), CONSTRAINT credit_transactions_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.crm_support_sla_policies (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid NOT NULL, policy_name text NOT NULL, priority text NOT NULL CHECK (priority = ANY (ARRAY['low'::text, 'medium'::text, 'high'::text, 'critical'::text])), target_first_response_minutes integer NOT NULL CHECK (target_first_response_minutes > 0), target_resolution_minutes integer NOT NULL CHECK (target_resolution_minutes > 0), is_default boolean NOT NULL DEFAULT false, is_active boolean NOT NULL DEFAULT true, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT crm_support_sla_policies_pkey PRIMARY KEY (id), CONSTRAINT crm_support_sla_policies_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.crm_support_tickets (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid NOT NULL, client_id uuid NOT NULL, project_id uuid, ticket_number text NOT NULL UNIQUE, title text NOT NULL, description text, priority text NOT NULL DEFAULT 'medium'::text CHECK (priority = ANY (ARRAY['low'::text, 'medium'::text, 'high'::text, 'critical'::text])), status text NOT NULL DEFAULT 'open'::text CHECK (status = ANY (ARRAY['open'::text, 'in_progress'::text, 'waiting_customer'::text, 'resolved'::text, 'closed'::text])), sla_level text NOT NULL DEFAULT 'standard'::text CHECK (sla_level = ANY (ARRAY['standard'::text, 'premium'::text, 'critical'::text])), due_at timestamp with time zone, first_response_at timestamp with time zone, resolved_at timestamp with time zone, closed_at timestamp with time zone, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT crm_support_tickets_pkey PRIMARY KEY (id), CONSTRAINT crm_support_tickets_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT fk_crm_support_tickets_client_scope FOREIGN KEY (user_id) REFERENCES public.clients(id), CONSTRAINT fk_crm_support_tickets_client_scope FOREIGN KEY (user_id) REFERENCES public.clients(user_id), CONSTRAINT fk_crm_support_tickets_client_scope FOREIGN KEY (user_id) REFERENCES public.clients(company_id), CONSTRAINT fk_crm_support_tickets_client_scope FOREIGN KEY (company_id) REFERENCES public.clients(id), CONSTRAINT fk_crm_support_tickets_client_scope FOREIGN KEY (company_id) REFERENCES public.clients(user_id), CONSTRAINT fk_crm_support_tickets_client_scope FOREIGN KEY (company_id) REFERENCES public.clients(company_id), CONSTRAINT fk_crm_support_tickets_client_scope FOREIGN KEY (client_id) REFERENCES public.clients(id), CONSTRAINT fk_crm_support_tickets_client_scope FOREIGN KEY

(client_id) REFERENCES public.clients(user_id), CONSTRAINT fk_crm_support_tickets_client_scope FOREIGN KEY (client_id) REFERENCES public.clients(company_id), CONSTRAINT fk_crm_support_tickets_project_scope FOREIGN KEY (user_id) REFERENCES public.projects(id), CONSTRAINT fk_crm_support_tickets_project_scope FOREIGN KEY (user_id) REFERENCES public.projects(user_id), CONSTRAINT fk_crm_support_tickets_project_scope FOREIGN KEY (user_id) REFERENCES public.projects(company_id), CONSTRAINT fk_crm_support_tickets_project_scope FOREIGN KEY (company_id) REFERENCES public.projects(id), CONSTRAINT fk_crm_support_tickets_project_scope FOREIGN KEY (company_id) REFERENCES public.projects(user_id), CONSTRAINT fk_crm_support_tickets_project_scope FOREIGN KEY (company_id) REFERENCES public.projects(company_id), CONSTRAINT fk_crm_support_tickets_project_scope FOREIGN KEY (project_id) REFERENCES public.projects(id), CONSTRAINT fk_crm_support_tickets_project_scope FOREIGN KEY (project_id) REFERENCES public.projects(user_id), CONSTRAINT fk_crm_support_tickets_project_scope FOREIGN KEY (project_id) REFERENCES public.projects(company_id)); CREATE TABLE public.dashboard_snapshots (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid, snapshot_type text NOT NULL CHECK (snapshot_type = ANY (ARRAY['dashboard'::text, 'analytics'::text])), title text NOT NULL, share_token text NOT NULL UNIQUE, snapshot_data jsonb NOT NULL DEFAULT '{}'::jsonb, is_public boolean NOT NULL DEFAULT true, expires_at timestamp with time zone, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT dashboard_snapshots_pkey PRIMARY KEY (id), CONSTRAINT dashboard_snapshots_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT dashboard_snapshots_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.data_export_requests (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, status text NOT NULL DEFAULT 'pending'::text CHECK (status = ANY (ARRAY['pending'::text, 'processing'::text, 'completed'::text, 'failed'::text])), file_url text, file_size bigint, requested_at timestamp with time zone DEFAULT now(), completed_at timestamp with time zone, expires_at timestamp with time zone, created_at timestamp with time zone DEFAULT now(), CONSTRAINT data_export_requests_pkey PRIMARY KEY (id), CONSTRAINT data_export_requests_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.debt_payments (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, amount numeric NOT NULL CHECK (amount > 0::numeric), payment_date date NOT NULL DEFAULT CURRENT_DATE, payment_method text DEFAULT 'cash'::text, notes text, created_at timestamp with time zone NOT NULL DEFAULT now(), company_id uuid, payment_instrument_id uuid, payment_transaction_id uuid, receivable_id uuid, payable_id uuid, CONSTRAINT debt_payments_pkey PRIMARY KEY (id), CONSTRAINT debt_payments_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT debt_payments_receivable_id_fkey FOREIGN KEY (receivable_id) REFERENCES public.receivables(id), CONSTRAINT debt_payments_payable_id_fkey FOREIGN KEY (payable_id) REFERENCES public.payables(id), CONSTRAINT debt_payments_payment_instrument_id_fkey FOREIGN KEY (payment_instrument_id) REFERENCES public.company_payment_instruments(id), CONSTRAINT debt_payments_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT fk_debt_payments_pt_id FOREIGN KEY (payment_transaction_id) REFERENCES public.payment_transactions(id)); CREATE TABLE public.deleted_data_snapshots (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid, entity_type text NOT NULL, entity_id uuid NOT NULL,

snapshot_data jsonb NOT NULL, deleted_at timestamp with time zone DEFAULT now(), deleted_by uuid, CONSTRAINT deleted_data_snapshots_pkey PRIMARY KEY (id), CONSTRAINT fk_deleted_data_snapshots_company FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT deleted_data_snapshots_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT deleted_data_snapshots_deleted_by_fkey FOREIGN KEY (deleted_by) REFERENCES auth.users(id)); CREATE TABLE public.delivery_note_items (id uuid NOT NULL DEFAULT gen_random_uuid(), delivery_note_id uuid NOT NULL, description text NOT NULL, quantity numeric DEFAULT 1, unit text DEFAULT ''::text, created_at timestamp with time zone DEFAULT now(), CONSTRAINT delivery_note_items_pkey PRIMARY KEY (id), CONSTRAINT delivery_note_items_delivery_note_id_fkey FOREIGN KEY (delivery_note_id) REFERENCES public.delivery_notes(id)); CREATE TABLE public.delivery_notes (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, delivery_note_number text NOT NULL, invoice_id uuid, client_id uuid, date date NOT NULL DEFAULT CURRENT_DATE, delivery_address text DEFAULT ''::text, carrier text DEFAULT ''::text, tracking_number text DEFAULT ''::text, status text DEFAULT 'pending'::text CHECK (status = ANY (ARRAY['pending'::text, 'shipped'::text, 'delivered'::text, 'cancelled'::text])), notes text DEFAULT ''::text, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), company_id uuid NOT NULL, file_url text, file_generated_at timestamp with time zone, CONSTRAINT delivery_notes_pkey PRIMARY KEY (id), CONSTRAINT delivery_notes_client_id_fkey FOREIGN KEY (client_id) REFERENCES public.clients(id), CONSTRAINT delivery_notes_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT delivery_notes_invoice_id_fkey FOREIGN KEY (invoice_id) REFERENCES public.invoices(id), CONSTRAINT delivery_notes_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.delivery_routes (id uuid NOT NULL DEFAULT gen_random_uuid(), order_id uuid, supplier_location_id uuid, delivery_location_id uuid, route_distance numeric, estimated_duration numeric, actual_duration numeric, route_polyline text, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), CONSTRAINT delivery_routes_pkey PRIMARY KEY (id), CONSTRAINT delivery_routes_delivery_location_id_fkey FOREIGN KEY (delivery_location_id) REFERENCES public.clients(id), CONSTRAINT delivery_routes_order_id_fkey FOREIGN KEY (order_id) REFERENCES public.supplier_orders(id), CONSTRAINT delivery_routes_supplier_location_id_fkey FOREIGN KEY (supplier_location_id) REFERENCES public.supplier_locations(id)); CREATE TABLE public.expenses (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid, client_id uuid, amount numeric NOT NULL CHECK (amount >= 0::numeric), category text, description text, receipt_url text, refacturable boolean DEFAULT false, created_at timestamp with time zone DEFAULT now(), tax_rate numeric, amount_ht numeric, tax_amount numeric, deleted_at timestamp with time zone, expense_date date, payment_method text DEFAULT 'bank_transfer'::text, company_id uuid NOT NULL, supplier_id uuid, payment_instrument_id uuid, payment_transaction_id uuid, CONSTRAINT expenses_pkey PRIMARY KEY (id), CONSTRAINT expenses_client_id_fkey FOREIGN KEY (client_id) REFERENCES public.clients(id), CONSTRAINT expenses_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT expenses_payment_instrument_id_fkey FOREIGN KEY (payment_instrument_id) REFERENCES public.company_payment_instruments(id), CONSTRAINT expenses_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT expenses_supplier_id_fkey FOREIGN KEY (supplier_id) REFERENCES public.suppliers(id), CONSTRAINT fk_expenses_pt_id FOREIGN KEY (payment_transaction_id) REFERENCES public.payment_transactions(id)); CREATE TABLE

public.financial_scenarios (id uuid NOT NULL DEFAULT uuid_generate_v4(), user_id uuid NOT NULL, name text NOT NULL, description text, base_date date NOT NULL, end_date date NOT NULL, status text DEFAULT 'draft'::text CHECK (status = ANY (ARRAY['draft'::text, 'active'::text, 'archived'::text, 'completed'::text])), is_baseline boolean DEFAULT false, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), company_id uuid, CONSTRAINT financial_scenarios_pkey PRIMARY KEY (id), CONSTRAINT financial_scenarios_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT financial_scenarios_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.fx_rates (id uuid NOT NULL DEFAULT gen_random_uuid(), company_id uuid, base_currency character varying NOT NULL, quote_currency character varying NOT NULL, rate_date date NOT NULL, exchange_rate numeric NOT NULL CHECK (exchange_rate > 0::numeric), source text NOT NULL DEFAULT 'manual'::text, is_fixed boolean NOT NULL DEFAULT false, metadata jsonb NOT NULL DEFAULT '{}'::jsonb, created_by uuid, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT fx_rates_pkey PRIMARY KEY (id), CONSTRAINT fx_rates_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT fx_rates_created_by_fkey FOREIGN KEY (created_by) REFERENCES auth.users(id)); CREATE TABLE public.invoice_items (id uuid NOT NULL DEFAULT gen_random_uuid(), invoice_id uuid, description text, quantity numeric, unit_price numeric, total numeric, created_at timestamp with time zone DEFAULT now(), product_id uuid, discount_type text DEFAULT 'none'::text CHECK (discount_type = ANY (ARRAY['none'::text, 'percentage'::text, 'fixed'::text])), discount_value numeric DEFAULT 0, discount_amount numeric DEFAULT 0, hsn_code text DEFAULT ''::text, item_type text DEFAULT 'manual'::text CHECK (item_type = ANY (ARRAY['product'::text, 'service'::text, 'timesheet'::text, 'manual'::text])), service_id uuid, timesheet_id uuid, CONSTRAINT invoice_items_pkey PRIMARY KEY (id), CONSTRAINT invoice_items_invoice_id_fkey FOREIGN KEY (invoice_id) REFERENCES public.invoices(id), CONSTRAINT invoice_items_product_id_fkey FOREIGN KEY (product_id) REFERENCES public.products(id), CONSTRAINT invoice_items_service_id_fkey FOREIGN KEY (service_id) REFERENCES public.services(id), CONSTRAINT invoice_items_timesheet_id_fkey FOREIGN KEY (timesheet_id) REFERENCES public.timesheets(id)); CREATE TABLE public.invoice_settings (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL UNIQUE, template_id text DEFAULT 'classic'::text, color_theme text DEFAULT 'default'::text, custom_labels jsonb DEFAULT '{}'::jsonb, show_logo boolean DEFAULT true, show_bank_details boolean DEFAULT true, show_payment_terms boolean DEFAULT true, footer_text text DEFAULT ''::text, font_family text DEFAULT 'Inter'::text, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), CONSTRAINT invoice_settings_pkey PRIMARY KEY (id), CONSTRAINT invoice_settings_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.invoice_status_config (status text NOT NULL, is_billable boolean NOT NULL DEFAULT false, is_booked boolean NOT NULL DEFAULT false, is_collected boolean NOT NULL DEFAULT false, display_color text, display_bg text, sort_order integer NOT NULL DEFAULT 0, CONSTRAINT invoice_status_config_pkey PRIMARY KEY (status)); CREATE TABLE public.invoice_status_transitions (from_status text NOT NULL, to_status text NOT NULL, CONSTRAINT invoice_status_transitions_pkey PRIMARY KEY (from_status, to_status), CONSTRAINT invoice_status_transitions_from_status_fkey FOREIGN KEY (from_status) REFERENCES public.invoice_status_config(status), CONSTRAINT invoice_status_transitions_to_status_fkey FOREIGN KEY (to_status) REFERENCES public.invoice_status_config(status)); CREATE TABLE public.invoices (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid, client_id uuid, invoice_number text NOT NULL

UNIQUE, date date, due_date date, status text DEFAULT 'draft'::text CHECK (status = ANY (ARRAY['draft'::text, 'sent'::text, 'paid'::text, 'overdue'::text, 'cancelled'::text])), total_ht numeric DEFAULT 0, tax_rate numeric DEFAULT 0 CHECK (tax_rate >= 0::numeric AND tax_rate <= 100::numeric), total_ttc numeric DEFAULT 0 CHECK (total_ttc >= 0::numeric), notes text, created_at timestamp with time zone DEFAULT now(), payment_terms_id uuid, conditions text, discount_type text DEFAULT 'none'::text CHECK (discount_type = ANY (ARRAY['none'::text, 'percentage'::text, 'fixed'::text])), discount_value numeric DEFAULT 0, discount_amount numeric DEFAULT 0, amount_paid numeric DEFAULT 0 CHECK (amount_paid >= 0::numeric), balance_due numeric DEFAULT 0, payment_status text DEFAULT 'unpaid'::text CHECK (payment_status = ANY (ARRAY['unpaid'::text, 'partial'::text, 'paid'::text, 'overpaid'::text])), shipping_fee numeric DEFAULT 0, adjustment numeric DEFAULT 0, adjustment_label text DEFAULT ''::text, header_note text DEFAULT ''::text, footer_note text DEFAULT ''::text, terms_and_conditions text DEFAULT ''::text, internal_remark text DEFAULT ''::text, attached_image_url text, custom_fields jsonb DEFAULT '[]'::jsonb, reference text DEFAULT ''::text, invoice_type text DEFAULT 'mixed'::text CHECK (invoice_type = ANY (ARRAY['product'::text, 'service'::text, 'mixed'::text])), peppol_status text DEFAULT 'none'::text CHECK (peppol_status = ANY (ARRAY['none'::text, 'pending'::text, 'created'::text, 'sent'::text, 'delivered'::text, 'accepted'::text, 'rejected'::text, 'error'::text])), peppol_sent_at timestamp with time zone, peppol_document_id text, peppol_error_message text, stripe_payment_link_id text, stripe_payment_link_url text, stripe_payment_intent_id text, payment_link_created_at timestamp with time zone, company_id uuid NOT NULL, currency text DEFAULT 'EUR'::text, source_quote_id uuid, file_url text, file_generated_at timestamp with time zone, CONSTRAINT invoices_pkey PRIMARY KEY (id), CONSTRAINT invoices_payment_terms_id_fkey FOREIGN KEY (payment_terms_id) REFERENCES public.payment_terms(id), CONSTRAINT invoices_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT fk_invoices_source_quote FOREIGN KEY (source_quote_id) REFERENCES public.quotes(id), CONSTRAINT invoices_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT fk_invoices_client_scope FOREIGN KEY (user_id) REFERENCES public.clients(id), CONSTRAINT fk_invoices_client_scope FOREIGN KEY (user_id) REFERENCES public.clients(user_id), CONSTRAINT fk_invoices_client_scope FOREIGN KEY (user_id) REFERENCES public.clients(company_id), CONSTRAINT fk_invoices_client_scope FOREIGN KEY (client_id) REFERENCES public.clients(id), CONSTRAINT fk_invoices_client_scope FOREIGN KEY (client_id) REFERENCES public.clients(user_id), CONSTRAINT fk_invoices_client_scope FOREIGN KEY (client_id) REFERENCES public.clients(company_id), CONSTRAINT fk_invoices_client_scope FOREIGN KEY (company_id) REFERENCES public.clients(id), CONSTRAINT fk_invoices_client_scope FOREIGN KEY (company_id) REFERENCES public.clients(user_id), CONSTRAINT fk_invoices_client_scope FOREIGN KEY (company_id) REFERENCES public.clients(company_id)); CREATE TABLE public.notification_preferences (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL UNIQUE, email_new_tasks boolean DEFAULT true, email_overdue_tasks boolean DEFAULT true, email_completed_tasks boolean DEFAULT false, email_comments boolean DEFAULT true, email_project_updates boolean DEFAULT true, email_reminders boolean DEFAULT true, push_enabled boolean DEFAULT true, push_new_tasks boolean DEFAULT true, push_comments boolean DEFAULT true, frequency text DEFAULT 'immediate'::text CHECK (frequency = ANY (ARRAY['immediate'::text, 'daily'::text, 'weekly'::text])), created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), CONSTRAINT notification_preferences_pkey PRIMARY KEY (id), CONSTRAINT notification_preferences_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.notifications (id uuid NOT NULL DEFAULT gen_random_uuid(),

user_id uuid, type text, message text, created_at timestamp with time zone DEFAULT now(), title text, is_read boolean DEFAULT false, read_at timestamp with time zone, updated_at timestamp with time zone DEFAULT now(), CONSTRAINT notifications_pkey PRIMARY KEY (id), CONSTRAINT notifications_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.offline_sync_queue (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid, action text NOT NULL, resource_type text NOT NULL, resource_id uuid, payload jsonb, status text DEFAULT 'pending'::text, error_message text, created_at timestamp with time zone DEFAULT now(), synced_at timestamp with time zone, CONSTRAINT offline_sync_queue_pkey PRIMARY KEY (id), CONSTRAINT offline_sync_queue_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.payables (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, creditor_name text NOT NULL, creditor_phone text, creditor_email text, description text, amount numeric NOT NULL CHECK (amount > 0::numeric), amount_paid numeric NOT NULL DEFAULT 0 CHECK (amount_paid >= 0::numeric), currency text NOT NULL DEFAULT 'EUR'::text, date_borrowed date NOT NULL DEFAULT CURRENT_DATE, due_date date, status text NOT NULL DEFAULT 'pending'::text CHECK (status = ANY (ARRAY['pending'::text, 'partial'::text, 'paid'::text, 'overdue'::text, 'cancelled'::text])), category text DEFAULT 'personal'::text, notes text, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), company_id uuid, CONSTRAINT payables_pkey PRIMARY KEY (id), CONSTRAINT payables_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT payables_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.payment_alerts (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid, payment_instrument_id uuid, alert_type text NOT NULL CHECK (alert_type = ANY (ARRAY['low_balance'::text, 'negative_balance'::text, 'credit_limit_reached'::text, 'large_cash_movement'::text, 'sync_failed'::text, 'reconciliation_gap'::text])), severity text NOT NULL CHECK (severity = ANY (ARRAY['info'::text, 'warning'::text, 'critical'::text])), title text NOT NULL, message text, is_resolved boolean NOT NULL DEFAULT false, resolved_at timestamp with time zone, created_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT payment_alerts_pkey PRIMARY KEY (id), CONSTRAINT payment_alerts_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT payment_alerts_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT payment_alerts_payment_instrument_id_fkey FOREIGN KEY (payment_instrument_id) REFERENCES public.company_payment_instruments(id)); CREATE TABLE public.payment_allocations (id uuid NOT NULL DEFAULT gen_random_uuid(), payment_id uuid NOT NULL, invoice_id uuid NOT NULL, amount numeric NOT NULL, created_at timestamp with time zone DEFAULT now(), CONSTRAINT payment_allocations_pkey PRIMARY KEY (id), CONSTRAINT payment_allocations_payment_id_fkey FOREIGN KEY (payment_id) REFERENCES public.payments(id), CONSTRAINT payment_allocations_invoice_id_fkey FOREIGN KEY (invoice_id) REFERENCES public.invoices(id)); CREATE TABLE public.payment_instrument_audit_log (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid, company_id uuid, payment_instrument_id uuid, action text NOT NULL CHECK (action = ANY (ARRAY['created'::text, 'updated'::text, 'status_changed'::text, 'archived'::text, 'unarchived'::text, 'balance_adjusted'::text, 'exported'::text])), old_data jsonb, new_data jsonb, context jsonb NOT NULL DEFAULT '{}'::jsonb, created_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT payment_instrument_audit_log_pkey PRIMARY KEY (id), CONSTRAINT payment_instrument_audit_log_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT payment_instrument_audit_log_company_id_fkey FOREIGN

KEY (company_id) REFERENCES public.company(id), CONSTRAINT payment_instrument_audit_log_payment_instrument_id_fkey FOREIGN KEY (payment_instrument_id) REFERENCES public.company_payment_instruments(id)); CREATE TABLE public.payment_instrument_bank_accounts (instrument_id uuid NOT NULL, bank_connection_id uuid, bank_name text, account_holder text, iban_masked text, iban_encrypted text, bic_swift text, account_number_masked text, institution_country text, account_kind text CHECK (account_kind = ANY (ARRAY['checking'::text, 'savings'::text, 'business'::text, 'escrow'::text, 'other'::text])), statement_import_enabled boolean NOT NULL DEFAULT false, api_sync_enabled boolean NOT NULL DEFAULT false, last_sync_at timestamp with time zone, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT payment_instrument_bank_accounts_pkey PRIMARY KEY (instrument_id), CONSTRAINT payment_instrument_bank_accounts_instrument_id_fkey FOREIGN KEY (instrument_id) REFERENCES public.company_payment_instruments(id), CONSTRAINT payment_instrument_bank_accounts_bank_connection_id_fkey FOREIGN KEY (bank_connection_id) REFERENCES public.bank_connections(id)); CREATE TABLE public.payment_instrument_cards (instrument_id uuid NOT NULL, card_brand text, card_type text NOT NULL CHECK (card_type = ANY (ARRAY['debit'::text, 'credit'::text, 'prepaid'::text, 'virtual'::text])), holder_name text, last4 text CHECK (char_length(last4) <= 4), expiry_month integer CHECK (expiry_month >= 1 AND expiry_month <= 12), expiry_year integer, issuer_name text, billing_cycle_day integer CHECK (billing_cycle_day >= 1 AND billing_cycle_day <= 31), statement_due_day integer CHECK (statement_due_day >= 1 AND statement_due_day <= 31), credit_limit numeric, available_credit numeric, network_token text, is_virtual boolean NOT NULL DEFAULT false, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT payment_instrument_cards_pkey PRIMARY KEY (instrument_id), CONSTRAINT payment_instrument_cards_instrument_id_fkey FOREIGN KEY (instrument_id) REFERENCES public.company_payment_instruments(id)); CREATE TABLE public.payment_instrument_cash_accounts (instrument_id uuid NOT NULL, cash_point_name text NOT NULL, custodian_user_id uuid, location text, max_authorized_balance numeric, reconciliation_frequency text NOT NULL DEFAULT 'manual'::text CHECK (reconciliation_frequency = ANY (ARRAY['daily'::text, 'weekly'::text, 'monthly'::text, 'manual'::text])), created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT payment_instrument_cash_accounts_pkey PRIMARY KEY (instrument_id), CONSTRAINT payment_instrument_cash_accounts_instrument_id_fkey FOREIGN KEY (instrument_id) REFERENCES public.company_payment_instruments(id), CONSTRAINT payment_instrument_cash_accounts_custodian_user_id_fkey FOREIGN KEY (custodian_user_id) REFERENCES auth.users(id)); CREATE TABLE public.payment_methods (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid, code text NOT NULL, name text NOT NULL, icon text, is_active boolean DEFAULT true, sort_order integer DEFAULT 0, created_at timestamp with time zone DEFAULT now(), CONSTRAINT payment_methods_pkey PRIMARY KEY (id), CONSTRAINT payment_methods_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT payment_methods_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.payment_reconciliations (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid NOT NULL, payment_transaction_id uuid NOT NULL, bank_transaction_id uuid, statement_line_id uuid, reconciliation_status text NOT NULL CHECK (reconciliation_status = ANY (ARRAY['matched'::text, 'partial'::text, 'manual'::text, 'rejected'::text])), matched_amount

numeric, confidence_score numeric, notes text, created_at timestamp with time zone NOT NULL DEFAULT now(), reconciled_by uuid, CONSTRAINT payment_reconciliations_pkey PRIMARY KEY (id), CONSTRAINT payment_reconciliations_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT payment_reconciliations_payment_transaction_id_fkey FOREIGN KEY (payment_transaction_id) REFERENCES public.payment_transactions(id), CONSTRAINT payment_reconciliations_bank_transaction_id_fkey FOREIGN KEY (bank_transaction_id) REFERENCES public.bank_transactions(id), CONSTRAINT payment_reconciliations_statement_line_id_fkey FOREIGN KEY (statement_line_id) REFERENCES public.bank_statement_lines(id), CONSTRAINT payment_reconciliations_reconciled_by_fkey FOREIGN KEY (reconciled_by) REFERENCES auth.users(id), CONSTRAINT payment_reconciliations_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.payment_reminder_logs (id uuid NOT NULL DEFAULT gen_random_uuid(), invoice_id uuid NOT NULL, rule_id uuid, reminder_number integer NOT NULL DEFAULT 1, sent_at timestamp with time zone DEFAULT now(), status text DEFAULT 'sent'::text CHECK (status = ANY (ARRAY['sent'::text, 'failed'::text, 'bounced'::text])), recipient_email text, user_id uuid NOT NULL, company_id uuid, CONSTRAINT payment_reminder_logs_pkey PRIMARY KEY (id), CONSTRAINT payment_reminder_logs_invoice_id_fkey FOREIGN KEY (invoice_id) REFERENCES public.invoices(id), CONSTRAINT payment_reminder_logs_rule_id_fkey FOREIGN KEY (rule_id) REFERENCES public.payment_reminder_rules(id), CONSTRAINT payment_reminder_logs_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT payment_reminder_logs_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.payment_reminder_rules (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, name text NOT NULL, days_before_due integer DEFAULT 0, days_after_due integer DEFAULT 0, max_reminders integer DEFAULT 3, is_active boolean DEFAULT true, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), company_id uuid, CONSTRAINT payment_reminder_rules_pkey PRIMARY KEY (id), CONSTRAINT payment_reminder_rules_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT payment_reminder_rules_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.payment_report_exports (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid, portfolio_id uuid, export_scope text NOT NULL CHECK (export_scope = ANY (ARRAY['company'::text, 'portfolio'::text, 'instrument'::text, 'transaction_list'::text])), export_format text NOT NULL CHECK (export_format = ANY (ARRAY['pdf'::text, 'html'::text])), status text NOT NULL DEFAULT 'pending'::text CHECK (status = ANY (ARRAY['pending'::text, 'processing'::text, 'completed'::text, 'failed'::text])), filters jsonb NOT NULL DEFAULT '{}'::jsonb, file_url text, file_size bigint, generated_at timestamp with time zone, expires_at timestamp with time zone, error_message text, created_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT payment_report_exports_pkey PRIMARY KEY (id), CONSTRAINT payment_report_exports_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT payment_report_exports_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT payment_report_exports_portfolio_id_fkey FOREIGN KEY (portfolio_id) REFERENCES public.company_portfolios(id)); CREATE TABLE public.payment_terms (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, name text NOT NULL, days integer NOT NULL DEFAULT 30, description text, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), CONSTRAINT payment_terms_pkey PRIMARY KEY (id), CONSTRAINT payment_terms_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id));

CREATE TABLE public.payment_transaction_allocations (id uuid NOT NULL DEFAULT gen_random_uuid(), payment_transaction_id uuid NOT NULL, allocation_type text NOT NULL CHECK (allocation_type = ANY (ARRAY['invoice'::text, 'expense'::text, 'supplier_invoice'::text, 'receivable'::text, 'payable'::text, 'credit_note'::text, 'manual'::text])), target_id uuid NOT NULL, allocated_amount numeric NOT NULL CHECK (allocated_amount > 0::numeric), notes text, created_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT payment_transaction_allocations_pkey PRIMARY KEY (id), CONSTRAINT payment_transaction_allocations_payment_transaction_id_fkey FOREIGN KEY (payment_transaction_id) REFERENCES public.payment_transactions(id)); CREATE TABLE public.payment_transactions (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid NOT NULL, portfolio_id uuid, payment_instrument_id uuid NOT NULL, transaction_kind text NOT NULL CHECK (transaction_kind = ANY (ARRAY['income'::text, 'expense'::text, 'transfer_in'::text, 'transfer_out'::text, 'refund_in'::text, 'refund_out'::text, 'fee'::text, 'adjustment'::text, 'withdrawal'::text, 'deposit'::text])), flow_direction text NOT NULL CHECK (flow_direction = ANY (ARRAY['inflow'::text, 'outflow'::text])), status text NOT NULL DEFAULT 'posted'::text CHECK (status = ANY (ARRAY['draft'::text, 'pending'::text, 'posted'::text, 'reconciled'::text, 'cancelled'::text])), source_module text NOT NULL CHECK (source_module = ANY (ARRAY['payments'::text, 'expenses'::text, 'debt_payments'::text, 'bank_transactions'::text, 'manual'::text, 'supplier_invoices'::text, 'receivables'::text, 'payables'::text, 'transfers'::text])), source_table text, source_id uuid, transaction_date date NOT NULL, posting_date date, value_date date, amount numeric NOT NULL CHECK (amount > 0::numeric), currency character varying NOT NULL DEFAULT 'EUR'::character varying CHECK (currency::text ~ '^[A-Z]{3}$'::text), company_currency character varying CHECK (company_currency IS NULL OR company_currency::text ~ '^[A-Z]{3}$'::text), fx_rate numeric, amount_company_currency numeric, counterparty_name text, description text, reference text, external_reference text, category text, subcategory text, analytical_axis_id uuid, attachment_url text, notes text, is_internal_transfer boolean NOT NULL DEFAULT false, transfer_group_id uuid, matched_bank_transaction_id uuid, accounting_entry_id uuid, created_by uuid, updated_by uuid, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), deleted_at timestamp with time zone, CONSTRAINT payment_transactions_pkey PRIMARY KEY (id), CONSTRAINT payment_transactions_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT payment_transactions_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT payment_transactions_portfolio_id_fkey FOREIGN KEY (portfolio_id) REFERENCES public.company_portfolios(id), CONSTRAINT payment_transactions_payment_instrument_id_fkey FOREIGN KEY (payment_instrument_id) REFERENCES public.company_payment_instruments(id), CONSTRAINT payment_transactions_matched_bank_transaction_id_fkey FOREIGN KEY (matched_bank_transaction_id) REFERENCES public.bank_transactions(id), CONSTRAINT payment_transactions_created_by_fkey FOREIGN KEY (created_by) REFERENCES auth.users(id), CONSTRAINT payment_transactions_updated_by_fkey FOREIGN KEY (updated_by) REFERENCES auth.users(id)); CREATE TABLE public.payment_transfers (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid NOT NULL, portfolio_id uuid, from_instrument_id uuid NOT NULL, to_instrument_id uuid NOT NULL, transfer_date date NOT NULL, amount numeric NOT NULL CHECK (amount > 0::numeric), currency character varying NOT NULL DEFAULT 'EUR'::character varying CHECK (currency::text ~ '^[A-Z]{3}$'::text), fee_amount numeric NOT NULL DEFAULT 0 CHECK (fee_amount >= 0::numeric), reference text, notes text, status text NOT NULL DEFAULT 'posted'::text CHECK (status = ANY (ARRAY['draft'::text, 'pending'::text, 'posted'::text, 'cancelled'::text])), transfer_group_id uuid NOT NULL DEFAULT gen_random_uuid(),

outflow_transaction_id uuid, inflow_transaction_id uuid, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT payment_transfers_pkey PRIMARY KEY (id), CONSTRAINT payment_transfers_to_instrument_id_fkey FOREIGN KEY (to_instrument_id) REFERENCES public.company_payment_instruments(id), CONSTRAINT payment_transfers_outflow_transaction_id_fkey FOREIGN KEY (outflow_transaction_id) REFERENCES public.payment_transactions(id), CONSTRAINT payment_transfers_inflow_transaction_id_fkey FOREIGN KEY (inflow_transaction_id) REFERENCES public.payment_transactions(id), CONSTRAINT payment_transfers_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT payment_transfers_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT payment_transfers_portfolio_id_fkey FOREIGN KEY (portfolio_id) REFERENCES public.company_portfolios(id), CONSTRAINT payment_transfers_from_instrument_id_fkey FOREIGN KEY (from_instrument_id) REFERENCES public.company_payment_instruments(id)); CREATE TABLE public.payments (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, invoice_id uuid, client_id uuid, payment_date date NOT NULL DEFAULT CURRENT_DATE, amount numeric NOT NULL CHECK (amount > 0::numeric), payment_method text DEFAULT 'bank_transfer'::text CHECK (payment_method = ANY (ARRAY['cash'::text, 'bank_transfer'::text, 'check'::text, 'card'::text, 'paypal'::text, 'other'::text])), reference text, notes text, is_lump_sum boolean DEFAULT false, receipt_number text, receipt_generated_at timestamp with time zone, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), deleted_at timestamp with time zone, company_id uuid NOT NULL, payment_instrument_id uuid, payment_transaction_id uuid, CONSTRAINT payments_pkey PRIMARY KEY (id), CONSTRAINT payments_client_id_fkey FOREIGN KEY (client_id) REFERENCES public.clients(id), CONSTRAINT payments_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT payments_invoice_id_fkey FOREIGN KEY (invoice_id) REFERENCES public.invoices(id), CONSTRAINT payments_payment_instrument_id_fkey FOREIGN KEY (payment_instrument_id) REFERENCES public.company_payment_instruments(id), CONSTRAINT payments_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT fk_payments_pt_id FOREIGN KEY (payment_transaction_id) REFERENCES public.payment_transactions(id)); CREATE TABLE public.pending_subscriptions (id uuid NOT NULL DEFAULT gen_random_uuid(), stripe_customer_email text NOT NULL, stripe_customer_id text, stripe_subscription_id text NOT NULL, plan_slug text NOT NULL, plan_id uuid, credits_per_month integer NOT NULL DEFAULT 0, billing_interval text DEFAULT 'monthly'::text, current_period_end timestamp with time zone, stripe_session_id text UNIQUE, claimed_by uuid, claimed_at timestamp with time zone, created_at timestamp with time zone DEFAULT now(), CONSTRAINT pending_subscriptions_pkey PRIMARY KEY (id), CONSTRAINT pending_subscriptions_plan_id_fkey FOREIGN KEY (plan_id) REFERENCES public.subscription_plans(id), CONSTRAINT pending_subscriptions_claimed_by_fkey FOREIGN KEY (claimed_by) REFERENCES auth.users(id)); CREATE TABLE public.peppol_inbound_documents (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, scrada_document_id text NOT NULL, sender_peppol_id text, sender_name text, document_type text DEFAULT 'invoice'::text, invoice_number text, invoice_date date, total_excl_vat numeric, total_vat numeric, total_incl_vat numeric, currency text DEFAULT 'EUR'::text, ubl_xml text, pdf_url text, status text DEFAULT 'new'::text CHECK (status = ANY (ARRAY['new'::text, 'processed'::text, 'archived'::text])), metadata jsonb DEFAULT '{}'::jsonb, received_at timestamp with time zone DEFAULT now(), created_at timestamp with time zone DEFAULT now(), CONSTRAINT peppol_inbound_documents_pkey PRIMARY KEY (id),

CONSTRAINT peppol_inbound_documents_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.peppol_transmission_log (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, invoice_id uuid NOT NULL, direction text NOT NULL CHECK (direction = ANY (ARRAY['outbound'::text, 'inbound'::text])), status text NOT NULL CHECK (status = ANY (ARRAY['pending'::text, 'sent'::text, 'delivered'::text, 'accepted'::text, 'rejected'::text, 'error'::text])), ap_provider text, ap_document_id text, sender_endpoint text, receiver_endpoint text, error_message text, metadata jsonb DEFAULT '{}'::jsonb, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), company_id uuid, CONSTRAINT peppol_transmission_log_pkey PRIMARY KEY (id), CONSTRAINT peppol_transmission_log_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT peppol_transmission_log_invoice_id_fkey FOREIGN KEY (invoice_id) REFERENCES public.invoices(id), CONSTRAINT peppol_transmission_log_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.plan_entitlements (id uuid NOT NULL DEFAULT gen_random_uuid(), plan_id uuid NOT NULL, feature_key text NOT NULL, created_at timestamp with time zone DEFAULT timezone('utc'::text, now()), CONSTRAINT plan_entitlements_pkey PRIMARY KEY (id), CONSTRAINT plan_entitlements_plan_id_fkey FOREIGN KEY (plan_id) REFERENCES public.subscription_plans(id)); CREATE TABLE public.product_barcodes (id uuid NOT NULL DEFAULT gen_random_uuid(), product_id uuid, barcode text NOT NULL UNIQUE, barcode_type text NOT NULL, created_at timestamp with time zone DEFAULT now(), CONSTRAINT product_barcodes_pkey PRIMARY KEY (id), CONSTRAINT product_barcodes_product_id_fkey FOREIGN KEY (product_id) REFERENCES public.products(id)); CREATE TABLE public.product_categories (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, name text NOT NULL, description text, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), company_id uuid, CONSTRAINT product_categories_pkey PRIMARY KEY (id), CONSTRAINT product_categories_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT product_categories_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.product_stock_history (id uuid NOT NULL DEFAULT gen_random_uuid(), product_id uuid, user_product_id uuid, previous_quantity numeric DEFAULT 0, new_quantity numeric DEFAULT 0, change_quantity numeric DEFAULT 0, reason text, notes text, order_id uuid, created_by uuid, created_at timestamp with time zone DEFAULT now(), company_id uuid, CONSTRAINT product_stock_history_pkey PRIMARY KEY (id), CONSTRAINT fk_product_stock_history_product FOREIGN KEY (product_id) REFERENCES public.products(id), CONSTRAINT product_stock_history_user_product_id_fkey FOREIGN KEY (user_product_id) REFERENCES public.products(id), CONSTRAINT product_stock_history_created_by_fkey FOREIGN KEY (created_by) REFERENCES auth.users(id), CONSTRAINT fk_product_stock_history_order FOREIGN KEY (order_id) REFERENCES public.supplier_orders(id), CONSTRAINT product_stock_history_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.products (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, product_name text NOT NULL, sku text, category_id uuid, unit_price numeric DEFAULT 0, purchase_price numeric DEFAULT 0, unit text DEFAULT 'pièce'::text, stock_quantity numeric DEFAULT 0 CHECK (stock_quantity >= 0::numeric), min_stock_level numeric DEFAULT 0, description text, is_active boolean DEFAULT true, supplier_id uuid, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), company_id uuid, inventory_tracking_enabled boolean NOT NULL DEFAULT true, CONSTRAINT products_pkey

PRIMARY KEY (id), CONSTRAINT products_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT products_category_id_fkey FOREIGN KEY (category_id) REFERENCES public.product_categories(id), CONSTRAINT products_supplier_id_fkey FOREIGN KEY (supplier_id) REFERENCES public.suppliers(id), CONSTRAINT products_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.profiles (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL UNIQUE, full_name text, company_name text, role text CHECK (role IS NULL OR (role = ANY (ARRAY['admin'::text, 'user'::text, 'freelance'::text, 'accountant'::text, 'manager'::text, 'client'::text]))), avatar_url text, phone text, address text, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), city text, postal_code text, country text, currency text DEFAULT 'USD'::text, timezone text DEFAULT 'UTC'::text, signature_url text, onboarding_completed boolean DEFAULT false, onboarding_step integer DEFAULT 0, language_code text DEFAULT 'en'::text CHECK (language_code = ANY (ARRAY['en'::text, 'fr'::text, 'nl'::text])), theme_preference text DEFAULT 'dark'::text CHECK (theme_preference = ANY (ARRAY['light'::text, 'dark'::text, 'system'::text])), CONSTRAINT profiles_pkey PRIMARY KEY (id), CONSTRAINT profiles_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.project_baselines (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid NOT NULL, project_id uuid NOT NULL, version integer NOT NULL DEFAULT 1, baseline_label text NOT NULL DEFAULT 'Baseline'::text, planned_start_date date, planned_end_date date, planned_budget_hours numeric, planned_budget_amount numeric, planned_tasks_count integer DEFAULT 0, notes text, is_active boolean NOT NULL DEFAULT false, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT project_baselines_pkey PRIMARY KEY (id), CONSTRAINT fk_project_baselines_project_scope FOREIGN KEY (user_id) REFERENCES public.projects(id), CONSTRAINT fk_project_baselines_project_scope FOREIGN KEY (user_id) REFERENCES public.projects(user_id), CONSTRAINT fk_project_baselines_project_scope FOREIGN KEY (user_id) REFERENCES public.projects(company_id), CONSTRAINT fk_project_baselines_project_scope FOREIGN KEY (company_id) REFERENCES public.projects(id), CONSTRAINT fk_project_baselines_project_scope FOREIGN KEY (company_id) REFERENCES public.projects(user_id), CONSTRAINT fk_project_baselines_project_scope FOREIGN KEY (company_id) REFERENCES public.projects(company_id), CONSTRAINT fk_project_baselines_project_scope FOREIGN KEY (project_id) REFERENCES public.projects(id), CONSTRAINT fk_project_baselines_project_scope FOREIGN KEY (project_id) REFERENCES public.projects(user_id), CONSTRAINT fk_project_baselines_project_scope FOREIGN KEY (project_id) REFERENCES public.projects(company_id), CONSTRAINT project_baselines_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT project_baselines_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT project_baselines_project_id_fkey FOREIGN KEY (project_id) REFERENCES public.projects(id)); CREATE TABLE public.project_milestones (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid NOT NULL, project_id uuid NOT NULL, title text NOT NULL, description text, status text NOT NULL DEFAULT 'planned'::text CHECK (status = ANY (ARRAY['planned'::text, 'in_progress'::text, 'achieved'::text, 'overdue'::text, 'cancelled'::text])), planned_date date, actual_date date, planned_amount numeric DEFAULT 0, bonus_rule_type text NOT NULL DEFAULT 'none'::text CHECK (bonus_rule_type = ANY (ARRAY['none'::text, 'fixed'::text, 'percentage'::text, 'day'::text])), bonus_rule_value numeric NOT NULL DEFAULT 0, malus_rule_type text NOT NULL DEFAULT 'none'::text CHECK (malus_rule_type = ANY

(ARRAY['none'::text, 'fixed'::text, 'percentage'::text, 'day'::text])), malus_rule_value numeric NOT NULL DEFAULT 0, settled_amount numeric NOT NULL DEFAULT 0, settled_at timestamp with time zone, linked_invoice_id uuid, linked_payment_id uuid, notes text, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT project_milestones_pkey PRIMARY KEY (id), CONSTRAINT fk_project_milestones_project_scope FOREIGN KEY (user_id) REFERENCES public.projects(id), CONSTRAINT fk_project_milestones_project_scope FOREIGN KEY (user_id) REFERENCES public.projects(user_id), CONSTRAINT fk_project_milestones_project_scope FOREIGN KEY (user_id) REFERENCES public.projects(company_id), CONSTRAINT fk_project_milestones_project_scope FOREIGN KEY (company_id) REFERENCES public.projects(id), CONSTRAINT fk_project_milestones_project_scope FOREIGN KEY (company_id) REFERENCES public.projects(user_id), CONSTRAINT fk_project_milestones_project_scope FOREIGN KEY (company_id) REFERENCES public.projects(company_id), CONSTRAINT fk_project_milestones_project_scope FOREIGN KEY (project_id) REFERENCES public.projects(id), CONSTRAINT fk_project_milestones_project_scope FOREIGN KEY (project_id) REFERENCES public.projects(user_id), CONSTRAINT fk_project_milestones_project_scope FOREIGN KEY (project_id) REFERENCES public.projects(company_id), CONSTRAINT project_milestones_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT project_milestones_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT project_milestones_project_id_fkey FOREIGN KEY (project_id) REFERENCES public.projects(id), CONSTRAINT project_milestones_linked_invoice_id_fkey FOREIGN KEY (linked_invoice_id) REFERENCES public.invoices(id), CONSTRAINT project_milestones_linked_payment_id_fkey FOREIGN KEY (linked_payment_id) REFERENCES public.payments(id)); CREATE TABLE public.project_resource_allocations (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid NOT NULL, project_id uuid NOT NULL, resource_type text NOT NULL CHECK (resource_type = ANY (ARRAY['human'::text, 'material'::text])), team_member_id uuid, resource_name text, unit text NOT NULL DEFAULT 'hour'::text, planned_quantity numeric NOT NULL DEFAULT 0, actual_quantity numeric NOT NULL DEFAULT 0, planned_cost numeric NOT NULL DEFAULT 0, actual_cost numeric NOT NULL DEFAULT 0, start_date date, end_date date, status text NOT NULL DEFAULT 'planned'::text CHECK (status = ANY (ARRAY['planned'::text, 'active'::text, 'completed'::text, 'cancelled'::text])), notes text, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT project_resource_allocations_pkey PRIMARY KEY (id), CONSTRAINT fk_project_resource_allocations_project_scope FOREIGN KEY (user_id) REFERENCES public.projects(id), CONSTRAINT fk_project_resource_allocations_project_scope FOREIGN KEY (user_id) REFERENCES public.projects(user_id), CONSTRAINT fk_project_resource_allocations_project_scope FOREIGN KEY (user_id) REFERENCES public.projects(company_id), CONSTRAINT fk_project_resource_allocations_project_scope FOREIGN KEY (company_id) REFERENCES public.projects(id), CONSTRAINT fk_project_resource_allocations_project_scope FOREIGN KEY (company_id) REFERENCES public.projects(user_id), CONSTRAINT fk_project_resource_allocations_project_scope FOREIGN KEY (company_id) REFERENCES public.projects(company_id), CONSTRAINT fk_project_resource_allocations_project_scope FOREIGN KEY (project_id) REFERENCES public.projects(id), CONSTRAINT fk_project_resource_allocations_project_scope FOREIGN KEY (project_id) REFERENCES public.projects(user_id), CONSTRAINT fk_project_resource_allocations_project_scope FOREIGN KEY (project_id) REFERENCES public.projects(company_id), CONSTRAINT project_resource_allocations_user_id_fkey

FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT project_resource_allocations_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT project_resource_allocations_project_id_fkey FOREIGN KEY (project_id) REFERENCES public.projects(id), CONSTRAINT project_resource_allocations_team_member_id_fkey FOREIGN KEY (team_member_id) REFERENCES public.team_members(id)); CREATE TABLE public.projects (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid, client_id uuid, name text NOT NULL, description text, budget_hours integer, hourly_rate numeric, status text DEFAULT 'active'::text, created_at timestamp with time zone DEFAULT now(), start_date date, end_date date, company_id uuid NOT NULL, CONSTRAINT projects_pkey PRIMARY KEY (id), CONSTRAINT projects_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT projects_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT fk_projects_client_scope FOREIGN KEY (user_id) REFERENCES public.clients(id), CONSTRAINT fk_projects_client_scope FOREIGN KEY (user_id) REFERENCES public.clients(user_id), CONSTRAINT fk_projects_client_scope FOREIGN KEY (user_id) REFERENCES public.clients(company_id), CONSTRAINT fk_projects_client_scope FOREIGN KEY (client_id) REFERENCES public.clients(id), CONSTRAINT fk_projects_client_scope FOREIGN KEY (client_id) REFERENCES public.clients(user_id), CONSTRAINT fk_projects_client_scope FOREIGN KEY (client_id) REFERENCES public.clients(company_id), CONSTRAINT fk_projects_client_scope FOREIGN KEY (company_id) REFERENCES public.clients(id), CONSTRAINT fk_projects_client_scope FOREIGN KEY (company_id) REFERENCES public.clients(user_id), CONSTRAINT fk_projects_client_scope FOREIGN KEY (company_id) REFERENCES public.clients(company_id)); CREATE TABLE public.purchase_orders (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, client_id uuid, po_number text NOT NULL, date date DEFAULT CURRENT_DATE, due_date date, items jsonb DEFAULT '[]'::jsonb, total numeric DEFAULT 0, status text DEFAULT 'draft'::text CHECK (status = ANY (ARRAY['draft'::text, 'sent'::text, 'confirmed'::text, 'completed'::text, 'cancelled'::text])), payment_terms_id uuid, notes text, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), company_id uuid, supplier_id uuid, file_url text, file_generated_at timestamp with time zone, CONSTRAINT purchase_orders_pkey PRIMARY KEY (id), CONSTRAINT purchase_orders_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT purchase_orders_payment_terms_id_fkey FOREIGN KEY (payment_terms_id) REFERENCES public.payment_terms(id), CONSTRAINT purchase_orders_client_id_fkey FOREIGN KEY (client_id) REFERENCES public.clients(id), CONSTRAINT fk_purchase_orders_supplier FOREIGN KEY (supplier_id) REFERENCES public.suppliers(id), CONSTRAINT purchase_orders_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.push_notification_logs (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid, title text NOT NULL, body text, action_url text, sent_at timestamp with time zone DEFAULT now(), read_at timestamp with time zone, status text DEFAULT 'sent'::text, CONSTRAINT push_notification_logs_pkey PRIMARY KEY (id), CONSTRAINT push_notification_logs_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.push_subscriptions (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid, endpoint text NOT NULL, auth_key text NOT NULL, p256dh_key text NOT NULL, device_type text, is_active boolean DEFAULT true, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), CONSTRAINT push_subscriptions_pkey PRIMARY KEY (id), CONSTRAINT push_subscriptions_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.quote_signature_evidence (id uuid NOT NULL DEFAULT gen_random_uuid(), quote_id

uuid NOT NULL, user_id uuid NOT NULL, company_id uuid, provider text NOT NULL DEFAULT 'native'::text CHECK (provider = ANY (ARRAY['native'::text, 'yousign'::text, 'docusign'::text, 'other'::text])), action text NOT NULL CHECK (action = ANY (ARRAY['requested'::text, 'signed'::text, 'rejected'::text])), signer_name text, signer_email text, signature_sha256 text, signature_storage_url text, ip_address text, user_agent text, accept_language text, request_id text, proof_token text, metadata jsonb NOT NULL DEFAULT '{}'::jsonb, signed_at timestamp with time zone NOT NULL DEFAULT now(), created_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT quote_signature_evidence_pkey PRIMARY KEY (id), CONSTRAINT quote_signature_evidence_quote_id_fkey FOREIGN KEY (quote_id) REFERENCES public.quotes(id), CONSTRAINT quote_signature_evidence_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT quote_signature_evidence_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.quotes (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid, client_id uuid, quote_number text NOT NULL UNIQUE, date date, status text DEFAULT 'draft'::text CHECK (status = ANY (ARRAY['draft'::text, 'sent'::text, 'accepted'::text, 'rejected'::text, 'expired'::text, 'converted'::text])), total_ht numeric DEFAULT 0, tax_rate numeric DEFAULT 0, total_ttc numeric DEFAULT 0, created_at timestamp with time zone DEFAULT now(), signature_status text NOT NULL DEFAULT 'unsigned'::text CHECK (signature_status = ANY (ARRAY['unsigned'::text, 'pending'::text, 'signed'::text, 'rejected'::text, 'expired'::text])), signature_token text UNIQUE, signature_token_expires_at timestamp with time zone, signed_by text, signer_email text, signed_at timestamp with time zone, signature_url text, company_id uuid NOT NULL, notes text, updated_at timestamp with time zone DEFAULT now(), converted_invoice_id uuid, file_url text, file_generated_at timestamp with time zone, CONSTRAINT quotes_pkey PRIMARY KEY (id), CONSTRAINT quotes_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT fk_quotes_converted_invoice FOREIGN KEY (converted_invoice_id) REFERENCES public.invoices(id), CONSTRAINT quotes_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT fk_quotes_client_scope FOREIGN KEY (user_id) REFERENCES public.clients(id), CONSTRAINT fk_quotes_client_scope FOREIGN KEY (user_id) REFERENCES public.clients(user_id), CONSTRAINT fk_quotes_client_scope FOREIGN KEY (user_id) REFERENCES public.clients(company_id), CONSTRAINT fk_quotes_client_scope FOREIGN KEY (client_id) REFERENCES public.clients(id), CONSTRAINT fk_quotes_client_scope FOREIGN KEY (client_id) REFERENCES public.clients(user_id), CONSTRAINT fk_quotes_client_scope FOREIGN KEY (client_id) REFERENCES public.clients(company_id), CONSTRAINT fk_quotes_client_scope FOREIGN KEY (company_id) REFERENCES public.clients(id), CONSTRAINT fk_quotes_client_scope FOREIGN KEY (company_id) REFERENCES public.clients(user_id), CONSTRAINT fk_quotes_client_scope FOREIGN KEY (company_id) REFERENCES public.clients(company_id)); CREATE TABLE public.receivables (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, debtor_name text NOT NULL, debtor_phone text, debtor_email text, description text, amount numeric NOT NULL CHECK (amount > 0::numeric), amount_paid numeric NOT NULL DEFAULT 0 CHECK (amount_paid >= 0::numeric), currency text NOT NULL DEFAULT 'EUR'::text, date_lent date NOT NULL DEFAULT CURRENT_DATE, due_date date, status text NOT NULL DEFAULT 'pending'::text CHECK (status = ANY (ARRAY['pending'::text, 'partial'::text, 'paid'::text, 'overdue'::text, 'cancelled'::text])), category text DEFAULT 'personal'::text, notes text, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), company_id uuid, CONSTRAINT receivables_pkey PRIMARY KEY (id), CONSTRAINT receivables_user_id_fkey FOREIGN KEY (user_id) REFERENCES

auth.users(id), CONSTRAINT receivables_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.recurring_invoice_line_items (id uuid NOT NULL DEFAULT gen_random_uuid(), recurring_invoice_id uuid NOT NULL, description text NOT NULL, quantity numeric DEFAULT 1, unit_price numeric DEFAULT 0, total numeric DEFAULT 0, sort_order integer DEFAULT 0, created_at timestamp with time zone DEFAULT timezone('utc'::text, now()), CONSTRAINT recurring_invoice_line_items_pkey PRIMARY KEY (id), CONSTRAINT recurring_invoice_line_items_recurring_invoice_id_fkey FOREIGN KEY (recurring_invoice_id) REFERENCES public.recurring_invoices(id)); CREATE TABLE public.recurring_invoices (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid, client_id uuid, frequency text CHECK (frequency = ANY (ARRAY['weekly'::text, 'monthly'::text, 'quarterly'::text, 'yearly'::text])), next_date date, status text DEFAULT 'active'::text CHECK (status = ANY (ARRAY['active'::text, 'paused'::text, 'completed'::text, 'cancelled'::text])), created_at timestamp with time zone DEFAULT now(), title text DEFAULT 'Facture recurrente'::text, description text, currency text DEFAULT 'EUR'::text, total_ht numeric DEFAULT 0, tva_rate numeric DEFAULT 0, total_tva numeric DEFAULT 0, total_ttc numeric DEFAULT 0, interval_count integer DEFAULT 1, day_of_month integer CHECK (day_of_month IS NULL OR day_of_month >= 1 AND day_of_month <= 31), start_date date, end_date date, next_generation_date date, auto_send boolean DEFAULT false, invoices_generated integer DEFAULT 0, last_generated_at timestamp with time zone, updated_at timestamp with time zone DEFAULT timezone('utc'::text, now()), company_id uuid NOT NULL, CONSTRAINT recurring_invoices_pkey PRIMARY KEY (id), CONSTRAINT recurring_invoices_client_id_fkey FOREIGN KEY (client_id) REFERENCES public.clients(id), CONSTRAINT recurring_invoices_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT recurring_invoices_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.reference_accounting_source_categories (source_type text NOT NULL, code text NOT NULL, label text NOT NULL, sort_order integer NOT NULL DEFAULT 0, is_active boolean NOT NULL DEFAULT true, created_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT reference_accounting_source_categories_pkey PRIMARY KEY (source_type, code), CONSTRAINT reference_accounting_source_categories_source_type_fkey FOREIGN KEY (source_type) REFERENCES public.reference_accounting_source_types(code)); CREATE TABLE public.reference_accounting_source_types (code text NOT NULL, label text NOT NULL, sort_order integer NOT NULL DEFAULT 0, is_active boolean NOT NULL DEFAULT true, created_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT reference_accounting_source_types_pkey PRIMARY KEY (code)); CREATE TABLE public.reference_countries (code text NOT NULL, label text NOT NULL, sort_order integer NOT NULL DEFAULT 0, created_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT reference_countries_pkey PRIMARY KEY (code)); CREATE TABLE public.reference_currencies (code character varying NOT NULL, symbol text NOT NULL, name text NOT NULL, region text, sort_order integer NOT NULL DEFAULT 0, created_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT reference_currencies_pkey PRIMARY KEY (code)); CREATE TABLE public.reference_debt_categories (code text NOT NULL, label_key text NOT NULL, sort_order integer NOT NULL DEFAULT 0, is_active boolean NOT NULL DEFAULT true, created_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT reference_debt_categories_pkey PRIMARY KEY (code)); CREATE TABLE public.reference_debt_payment_methods (code text NOT NULL, label_key text NOT NULL, sort_order integer NOT NULL DEFAULT 0, is_active boolean NOT NULL DEFAULT true, created_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT reference_debt_payment_methods_pkey PRIMARY KEY (code)); CREATE TABLE public.reference_debt_statuses (code text NOT NULL, label_key text NOT NULL, display_color

text NOT NULL DEFAULT 'text-gray-400'::text, display_bg text NOT NULL DEFAULT 'bg-gray-500/20'::text, is_open boolean NOT NULL DEFAULT true, is_terminal boolean NOT NULL DEFAULT false, sort_order integer NOT NULL DEFAULT 0, is_active boolean NOT NULL DEFAULT true, created_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT reference_debt_statuses_pkey PRIMARY KEY (code)); CREATE TABLE public.reference_region_wacc (region text NOT NULL, risk_free_rate numeric NOT NULL, equity_premium numeric NOT NULL, beta numeric NOT NULL, wacc numeric NOT NULL, created_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT reference_region_wacc_pkey PRIMARY KEY (region)); CREATE TABLE public.reference_sector_benchmarks (id uuid NOT NULL DEFAULT gen_random_uuid(), sector text NOT NULL, metric_key text NOT NULL, low_value numeric NOT NULL, target_value numeric NOT NULL, high_value numeric NOT NULL, created_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT reference_sector_benchmarks_pkey PRIMARY KEY (id)); CREATE TABLE public.reference_sector_multiples (sector text NOT NULL, region text NOT NULL, low_value numeric NOT NULL, mid_value numeric NOT NULL, high_value numeric NOT NULL, created_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT reference_sector_multiples_pkey PRIMARY KEY (sector, region)); CREATE TABLE public.reference_tax_jurisdiction_vat_rates (id uuid NOT NULL DEFAULT gen_random_uuid(), jurisdiction_code text NOT NULL, rate numeric NOT NULL, label text NOT NULL, is_default boolean NOT NULL DEFAULT false, sort_order integer NOT NULL DEFAULT 0, created_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT reference_tax_jurisdiction_vat_rates_pkey PRIMARY KEY (id), CONSTRAINT reference_tax_jurisdiction_vat_rates_jurisdiction_code_fkey FOREIGN KEY (jurisdiction_code) REFERENCES public.reference_tax_jurisdictions(code)); CREATE TABLE public.reference_tax_jurisdictions (code text NOT NULL, name text NOT NULL, currency character varying NOT NULL, vat_label text NOT NULL, default_vat_rate numeric NOT NULL DEFAULT 0, vat_number_pattern text, vat_number_prefix text, fiscal_year_start text, fiscal_year_end text, export_formats ARRAY NOT NULL DEFAULT ARRAY[]::text[], declaration_periods ARRAY NOT NULL DEFAULT ARRAY[]::text[], invoice_requirements jsonb NOT NULL DEFAULT '{}'::jsonb, is_active boolean NOT NULL DEFAULT true, created_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT reference_tax_jurisdictions_pkey PRIMARY KEY (code), CONSTRAINT reference_tax_jurisdictions_currency_fkey FOREIGN KEY (currency) REFERENCES public.reference_currencies(code)); CREATE TABLE public.referrals (id uuid NOT NULL DEFAULT gen_random_uuid(), referrer_user_id uuid NOT NULL, referred_user_id uuid, referral_code text NOT NULL UNIQUE, status text DEFAULT 'pending'::text CHECK (status = ANY (ARRAY['pending'::text, 'completed'::text, 'expired'::text])), bonus_credited boolean DEFAULT false, created_at timestamp with time zone DEFAULT now(), completed_at timestamp with time zone, CONSTRAINT referrals_pkey PRIMARY KEY (id), CONSTRAINT referrals_referrer_user_id_fkey FOREIGN KEY (referrer_user_id) REFERENCES auth.users(id), CONSTRAINT referrals_referred_user_id_fkey FOREIGN KEY (referred_user_id) REFERENCES auth.users(id)); CREATE TABLE public.report_builder_templates (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid NOT NULL, name text NOT NULL, preset text NOT NULL DEFAULT 'custom'::text, period_start date NOT NULL, period_end date NOT NULL, sections jsonb NOT NULL DEFAULT '{}'::jsonb, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT report_builder_templates_pkey PRIMARY KEY (id), CONSTRAINT report_builder_templates_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT report_builder_templates_company_id_fkey FOREIGN KEY (company_id)

REFERENCES public.company(id)); CREATE TABLE public.report_templates (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid, name text NOT NULL, description text, template_type text, html_template text, css_styles text, is_active boolean DEFAULT true, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), CONSTRAINT report_templates_pkey PRIMARY KEY (id), CONSTRAINT report_templates_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.role_permissions (id uuid NOT NULL DEFAULT gen_random_uuid(), role text NOT NULL CHECK (role = ANY (ARRAY['admin'::text, 'manager'::text, 'accountant'::text, 'user'::text, 'freelance'::text, 'client'::text])), permission text NOT NULL, created_at timestamp with time zone DEFAULT now(), CONSTRAINT role_permissions_pkey PRIMARY KEY (id)); CREATE TABLE public.scenario_assumptions (id uuid NOT NULL DEFAULT uuid_generate_v4(), scenario_id uuid NOT NULL, name text NOT NULL, description text, category text NOT NULL, assumption_type text NOT NULL CHECK (assumption_type = ANY (ARRAY['growth_rate'::text, 'fixed_amount'::text, 'recurring'::text, 'one_time'::text, 'percentage_change'::text, 'conditional'::text])), parameters jsonb NOT NULL DEFAULT '{}'::jsonb, start_date date, end_date date, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), CONSTRAINT scenario_assumptions_pkey PRIMARY KEY (id), CONSTRAINT scenario_assumptions_scenario_id_fkey FOREIGN KEY (scenario_id) REFERENCES public.financial_scenarios(id)); CREATE TABLE public.scenario_comparisons (id uuid NOT NULL DEFAULT uuid_generate_v4(), user_id uuid NOT NULL, name text NOT NULL, scenario_ids ARRAY NOT NULL, comparison_metrics jsonb DEFAULT '{}'::jsonb, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), company_id uuid, CONSTRAINT scenario_comparisons_pkey PRIMARY KEY (id), CONSTRAINT scenario_comparisons_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT scenario_comparisons_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.scenario_results (id uuid NOT NULL DEFAULT uuid_generate_v4(), scenario_id uuid NOT NULL, calculation_date date NOT NULL, period_label text, metrics jsonb NOT NULL DEFAULT '{}'::jsonb, calculated_at timestamp with time zone DEFAULT now(), CONSTRAINT scenario_results_pkey PRIMARY KEY (id), CONSTRAINT scenario_results_scenario_id_fkey FOREIGN KEY (scenario_id) REFERENCES public.financial_scenarios(id)); CREATE TABLE public.scenario_templates (id uuid NOT NULL DEFAULT uuid_generate_v4(), name text NOT NULL, description text, category text NOT NULL, icon text, default_assumptions jsonb NOT NULL DEFAULT '[]'::jsonb, suggested_duration_months integer DEFAULT 12, is_public boolean DEFAULT true, user_id uuid, created_at timestamp with time zone DEFAULT now(), CONSTRAINT scenario_templates_pkey PRIMARY KEY (id), CONSTRAINT scenario_templates_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.sector_benchmarks (id uuid NOT NULL DEFAULT gen_random_uuid(), sector_code text NOT NULL, sector_name text NOT NULL, region text NOT NULL DEFAULT 'FR'::text, gross_margin_percent numeric NOT NULL, ebitda_margin numeric NOT NULL, net_margin numeric, bfr_typical numeric, caf_typical numeric, debt_ratio_max numeric, current_ratio_min numeric, dso_days integer, dpo_days integer, inventory_days integer, source text, year integer DEFAULT 2026, created_at timestamp with time zone DEFAULT now(), CONSTRAINT sector_benchmarks_pkey PRIMARY KEY (id)); CREATE TABLE public.service_categories (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, name text NOT NULL, description text, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), company_id uuid, CONSTRAINT service_categories_pkey PRIMARY KEY (id), CONSTRAINT service_categories_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id),

CONSTRAINT service_categories_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.services (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, service_name text NOT NULL, description text, category_id uuid, pricing_type text NOT NULL DEFAULT 'hourly'::text CHECK (pricing_type = ANY (ARRAY['hourly'::text, 'fixed'::text, 'per_unit'::text])), hourly_rate numeric DEFAULT 0, fixed_price numeric DEFAULT 0, unit_price numeric DEFAULT 0, unit text DEFAULT 'heure'::text, is_active boolean DEFAULT true, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), company_id uuid, CONSTRAINT services_pkey PRIMARY KEY (id), CONSTRAINT services_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT services_category_id_fkey FOREIGN KEY (category_id) REFERENCES public.service_categories(id), CONSTRAINT services_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.stock_alerts (id uuid NOT NULL DEFAULT gen_random_uuid(), product_id uuid, user_product_id uuid, alert_type text CHECK (alert_type = ANY (ARRAY['low_stock'::text, 'out_of_stock'::text, 'overstock'::text])), is_active boolean DEFAULT true, resolved_at timestamp with time zone, created_at timestamp with time zone DEFAULT now(), company_id uuid, CONSTRAINT stock_alerts_pkey PRIMARY KEY (id), CONSTRAINT fk_stock_alerts_product_id FOREIGN KEY (product_id) REFERENCES public.products(id), CONSTRAINT stock_alerts_user_product_id_fkey FOREIGN KEY (user_product_id) REFERENCES public.products(id), CONSTRAINT stock_alerts_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.stripe_settings (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL UNIQUE, stripe_customer_id text, stripe_enabled boolean DEFAULT false, stripe_publishable_key text, stripe_mode text DEFAULT 'test'::text CHECK (stripe_mode = ANY (ARRAY['test'::text, 'live'::text])), created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), CONSTRAINT stripe_settings_pkey PRIMARY KEY (id), CONSTRAINT stripe_settings_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.subscription_plans (id uuid NOT NULL DEFAULT gen_random_uuid(), name text NOT NULL, slug text NOT NULL UNIQUE, price_cents integer NOT NULL DEFAULT 0, currency text NOT NULL DEFAULT 'EUR'::text, credits_per_month integer NOT NULL DEFAULT 0, stripe_price_id text, features jsonb DEFAULT '[]'::jsonb, is_active boolean DEFAULT true, sort_order integer DEFAULT 0, created_at timestamp with time zone DEFAULT now(), stripe_price_id_yearly text, CONSTRAINT subscription_plans_pkey PRIMARY KEY (id)); CREATE TABLE public.subtasks (id uuid NOT NULL DEFAULT gen_random_uuid(), task_id uuid NOT NULL, title text NOT NULL, status text DEFAULT 'pending'::text CHECK (status = ANY (ARRAY['pending'::text, 'completed'::text])), created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), CONSTRAINT subtasks_pkey PRIMARY KEY (id), CONSTRAINT subtasks_task_id_fkey FOREIGN KEY (task_id) REFERENCES public.tasks(id)); CREATE TABLE public.supplier_invoice_line_items (id uuid NOT NULL DEFAULT gen_random_uuid(), invoice_id uuid NOT NULL, description text NOT NULL, quantity numeric DEFAULT 1, unit_price numeric, total numeric, sort_order integer DEFAULT 0, created_at timestamp with time zone DEFAULT now(), user_id uuid NOT NULL, vat_rate numeric, user_product_id uuid, CONSTRAINT supplier_invoice_line_items_pkey PRIMARY KEY (id), CONSTRAINT supplier_invoice_line_items_invoice_id_fkey FOREIGN KEY (invoice_id) REFERENCES public.supplier_invoices(id), CONSTRAINT supplier_invoice_line_items_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT fk_supplier_invoice_line_items_user_product FOREIGN KEY (user_product_id) REFERENCES public.products(id)); CREATE TABLE public.supplier_invoices (id uuid NOT NULL DEFAULT

gen_random_uuid(), supplier_id uuid CHECK (supplier_id IS NOT NULL), invoice_number text NOT NULL, invoice_date date, due_date date, total_amount numeric NOT NULL DEFAULT 0, vat_amount numeric DEFAULT 0, vat_rate numeric DEFAULT 21, payment_status text DEFAULT 'pending'::text CHECK (payment_status = ANY (ARRAY['pending'::text, 'paid'::text, 'overdue'::text, 'cancelled'::text])), file_url text, notes text, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), total_ht numeric, total_ttc numeric, currency text DEFAULT 'EUR'::text, supplier_name_extracted text, supplier_address_extracted text, supplier_vat_number text, payment_terms text, iban text, bic text, ai_extracted boolean DEFAULT false, ai_confidence numeric, ai_raw_response jsonb, ai_extracted_at timestamp with time zone, company_id uuid NOT NULL, approval_status text NOT NULL DEFAULT 'pending'::text CHECK (approval_status = ANY (ARRAY['pending'::text, 'approved'::text, 'rejected'::text])), approved_by uuid, approved_at timestamp with time zone, rejected_reason text, user_id uuid, status text DEFAULT 'draft'::text, supplier_order_id uuid, CONSTRAINT supplier_invoices_pkey PRIMARY KEY (id), CONSTRAINT supplier_invoices_supplier_id_fkey FOREIGN KEY (supplier_id) REFERENCES public.suppliers(id), CONSTRAINT fk_supplier_invoices_order FOREIGN KEY (supplier_order_id) REFERENCES public.supplier_orders(id), CONSTRAINT fk_supplier_invoices_user FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT supplier_invoices_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT supplier_invoices_approved_by_fkey FOREIGN KEY (approved_by) REFERENCES auth.users(id)); CREATE TABLE public.supplier_locations (id uuid NOT NULL DEFAULT gen_random_uuid(), supplier_id uuid, location_name text NOT NULL, latitude numeric, longitude numeric, address text, postal_code text, city text, country text, phone text, email text, opening_hours jsonb, is_primary boolean DEFAULT false, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), company_id uuid, CONSTRAINT supplier_locations_pkey PRIMARY KEY (id), CONSTRAINT supplier_locations_supplier_id_fkey FOREIGN KEY (supplier_id) REFERENCES public.suppliers(id), CONSTRAINT supplier_locations_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.supplier_order_items (id uuid NOT NULL DEFAULT gen_random_uuid(), order_id uuid, service_id uuid, product_id uuid, quantity numeric NOT NULL DEFAULT 1, unit_price numeric NOT NULL DEFAULT 0, total_price numeric NOT NULL DEFAULT 0, created_at timestamp with time zone DEFAULT now(), user_product_id uuid, CONSTRAINT supplier_order_items_pkey PRIMARY KEY (id), CONSTRAINT supplier_order_items_order_id_fkey FOREIGN KEY (order_id) REFERENCES public.supplier_orders(id), CONSTRAINT supplier_order_items_service_id_fkey FOREIGN KEY (service_id) REFERENCES public.supplier_services(id), CONSTRAINT supplier_order_items_product_id_fkey FOREIGN KEY (product_id) REFERENCES public.supplier_products(id), CONSTRAINT supplier_order_items_user_product_id_fkey FOREIGN KEY (user_product_id) REFERENCES public.products(id)); CREATE TABLE public.supplier_orders (id uuid NOT NULL DEFAULT gen_random_uuid(), supplier_id uuid, user_id uuid, order_number text NOT NULL, order_date date DEFAULT CURRENT_DATE, expected_delivery_date date, actual_delivery_date date, order_status text DEFAULT 'pending'::text CHECK (order_status IS NULL OR (order_status = ANY (ARRAY['draft'::text, 'pending'::text, 'confirmed'::text, 'delivered'::text, 'received'::text, 'cancelled'::text]))), total_amount numeric DEFAULT 0, notes text, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), company_id uuid NOT NULL, CONSTRAINT supplier_orders_pkey PRIMARY KEY (id), CONSTRAINT supplier_orders_supplier_id_fkey FOREIGN KEY (supplier_id) REFERENCES public.suppliers(id), CONSTRAINT supplier_orders_user_id_fkey FOREIGN KEY (user_id)

REFERENCES auth.users(id), CONSTRAINT supplier_orders_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.supplier_product_categories (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid, name text NOT NULL, description text, created_at timestamp with time zone DEFAULT now(), company_id uuid, CONSTRAINT supplier_product_categories_pkey PRIMARY KEY (id), CONSTRAINT supplier_product_categories_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT supplier_product_categories_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.supplier_products (id uuid NOT NULL DEFAULT gen_random_uuid(), supplier_id uuid, category_id uuid, product_name text NOT NULL, description text, sku text, unit_price numeric NOT NULL DEFAULT 0, unit text DEFAULT 'pcs'::text, stock_quantity integer DEFAULT 0, min_stock_level integer DEFAULT 5, reorder_quantity integer DEFAULT 10, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), company_id uuid, CONSTRAINT supplier_products_pkey PRIMARY KEY (id), CONSTRAINT supplier_products_supplier_id_fkey FOREIGN KEY (supplier_id) REFERENCES public.suppliers(id), CONSTRAINT supplier_products_category_id_fkey FOREIGN KEY (category_id) REFERENCES public.supplier_product_categories(id), CONSTRAINT supplier_products_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.supplier_reports_cache (id uuid NOT NULL DEFAULT gen_random_uuid(), supplier_id uuid, user_id uuid, report_type text NOT NULL, period text NOT NULL, start_date date, end_date date, data jsonb DEFAULT '{}'::jsonb, generated_at timestamp with time zone DEFAULT now(), expires_at timestamp with time zone, company_id uuid, CONSTRAINT supplier_reports_cache_pkey PRIMARY KEY (id), CONSTRAINT supplier_reports_cache_supplier_id_fkey FOREIGN KEY (supplier_id) REFERENCES public.suppliers(id), CONSTRAINT supplier_reports_cache_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT supplier_reports_cache_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.supplier_services (id uuid NOT NULL DEFAULT gen_random_uuid(), supplier_id uuid, service_name text NOT NULL, description text, pricing_type text CHECK (pricing_type = ANY (ARRAY['hourly'::text, 'fixed'::text])), hourly_rate numeric, fixed_price numeric, unit text, availability text, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), company_id uuid, CONSTRAINT supplier_services_pkey PRIMARY KEY (id), CONSTRAINT supplier_services_supplier_id_fkey FOREIGN KEY (supplier_id) REFERENCES public.suppliers(id), CONSTRAINT supplier_services_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.suppliers (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid, company_name text NOT NULL, contact_person text, email text, phone text, address text, postal_code text, city text, country text, website text, payment_terms text, supplier_type text CHECK (supplier_type = ANY (ARRAY['service'::text, 'product'::text, 'both'::text])), status text DEFAULT 'active'::text CHECK (status = ANY (ARRAY['active'::text, 'inactive'::text])), notes text, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), tax_id text, currency text DEFAULT 'EUR'::text, bank_name text, iban text, bic_swift text, company_id uuid NOT NULL, CONSTRAINT suppliers_pkey PRIMARY KEY (id), CONSTRAINT suppliers_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT suppliers_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id)); CREATE TABLE public.tasks (id uuid NOT NULL DEFAULT gen_random_uuid(), project_id uuid, name text NOT NULL, description text, status text DEFAULT 'pending'::text, created_at timestamp with time zone DEFAULT now(), estimated_hours numeric, service_id uuid, title text, priority text DEFAULT 'medium'::text CHECK (priority = ANY (ARRAY['low'::text, 'medium'::text,

'high'::text])), assigned_to text, due_date timestamp with time zone, started_at timestamp with time zone, completed_at timestamp with time zone, color text DEFAULT '#3b82f6'::text, invoice_id uuid, quote_id uuid, purchase_order_id uuid, updated_at timestamp with time zone NOT NULL DEFAULT now(), requires_quote boolean NOT NULL DEFAULT false, start_date date, end_date date, depends_on ARRAY DEFAULT '{}'::uuid[], assigned_member_id uuid, CONSTRAINT tasks_pkey PRIMARY KEY (id), CONSTRAINT tasks_project_id_fkey FOREIGN KEY (project_id) REFERENCES public.projects(id), CONSTRAINT tasks_invoice_id_fkey FOREIGN KEY (invoice_id) REFERENCES public.invoices(id), CONSTRAINT tasks_quote_id_fkey FOREIGN KEY (quote_id) REFERENCES public.quotes(id), CONSTRAINT tasks_purchase_order_id_fkey FOREIGN KEY (purchase_order_id) REFERENCES public.purchase_orders(id), CONSTRAINT tasks_service_id_fkey FOREIGN KEY (service_id) REFERENCES public.services(id), CONSTRAINT tasks_assigned_member_id_fkey FOREIGN KEY (assigned_member_id) REFERENCES public.team_members(id)); CREATE TABLE public.tax_brackets (id uuid NOT NULL DEFAULT gen_random_uuid(), country_code text NOT NULL, tax_type text NOT NULL DEFAULT 'corporate_income'::text, bracket_min numeric NOT NULL, bracket_max numeric, rate numeric NOT NULL, label text NOT NULL, effective_year integer NOT NULL DEFAULT 2026, source text, created_at timestamp with time zone DEFAULT now(), CONSTRAINT tax_brackets_pkey PRIMARY KEY (id)); CREATE TABLE public.tax_rate_presets (id uuid NOT NULL DEFAULT gen_random_uuid(), country_code text NOT NULL, preset_name text NOT NULL, rate numeric NOT NULL, account_code text NOT NULL, tax_type text NOT NULL DEFAULT 'VAT'::text, is_active boolean DEFAULT true, sort_order integer DEFAULT 0, created_at timestamp with time zone DEFAULT now(), CONSTRAINT tax_rate_presets_pkey PRIMARY KEY (id)); CREATE TABLE public.team_member_compensations (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, company_id uuid NOT NULL, project_id uuid NOT NULL, team_member_id uuid NOT NULL, task_id uuid, timesheet_id uuid, amount numeric NOT NULL CHECK (amount >= 0::numeric), compensation_type text NOT NULL DEFAULT 'hourly'::text CHECK (compensation_type = ANY (ARRAY['hourly'::text, 'fixed'::text, 'bonus'::text, 'malus'::text, 'adjustment'::text])), payment_status text NOT NULL DEFAULT 'planned'::text CHECK (payment_status = ANY (ARRAY['planned'::text, 'approved'::text, 'paid'::text, 'cancelled'::text])), planned_payment_date date, paid_at timestamp with time zone, payment_reference text, notes text, created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT team_member_compensations_pkey PRIMARY KEY (id), CONSTRAINT fk_team_member_compensations_project_scope FOREIGN KEY (user_id) REFERENCES public.projects(id), CONSTRAINT fk_team_member_compensations_project_scope FOREIGN KEY (user_id) REFERENCES public.projects(user_id), CONSTRAINT fk_team_member_compensations_project_scope FOREIGN KEY (user_id) REFERENCES public.projects(company_id), CONSTRAINT fk_team_member_compensations_project_scope FOREIGN KEY (company_id) REFERENCES public.projects(id), CONSTRAINT fk_team_member_compensations_project_scope FOREIGN KEY (company_id) REFERENCES public.projects(user_id), CONSTRAINT fk_team_member_compensations_project_scope FOREIGN KEY (company_id) REFERENCES public.projects(company_id), CONSTRAINT fk_team_member_compensations_project_scope FOREIGN KEY (project_id) REFERENCES public.projects(id), CONSTRAINT fk_team_member_compensations_project_scope FOREIGN KEY (project_id) REFERENCES public.projects(user_id), CONSTRAINT fk_team_member_compensations_project_scope FOREIGN KEY (project_id) REFERENCES public.projects(company_id), CONSTRAINT fk_team_member_compensations_task_project FOREIGN KEY (project_id) REFERENCES public.tasks(id), CONSTRAINT fk_team_member_compensations_task_project FOREIGN KEY (project_id) REFERENCES

public.tasks(project_id), CONSTRAINT fk_team_member_compensations_task_project FOREIGN KEY (task_id) REFERENCES public.tasks(id), CONSTRAINT fk_team_member_compensations_task_project FOREIGN KEY (task_id) REFERENCES public.tasks(project_id), CONSTRAINT fk_team_member_compensations_timesheet_scope FOREIGN KEY (user_id) REFERENCES public.timesheets(id), CONSTRAINT fk_team_member_compensations_timesheet_scope FOREIGN KEY (user_id) REFERENCES public.timesheets(user_id), CONSTRAINT fk_team_member_compensations_timesheet_scope FOREIGN KEY (user_id) REFERENCES public.timesheets(project_id), CONSTRAINT fk_team_member_compensations_timesheet_scope FOREIGN KEY (user_id) REFERENCES public.timesheets(company_id), CONSTRAINT fk_team_member_compensations_timesheet_scope FOREIGN KEY (company_id) REFERENCES public.timesheets(id), CONSTRAINT fk_team_member_compensations_timesheet_scope FOREIGN KEY (company_id) REFERENCES public.timesheets(user_id), CONSTRAINT fk_team_member_compensations_timesheet_scope FOREIGN KEY (company_id) REFERENCES public.timesheets(project_id), CONSTRAINT fk_team_member_compensations_timesheet_scope FOREIGN KEY (company_id) REFERENCES public.timesheets(company_id), CONSTRAINT fk_team_member_compensations_timesheet_scope FOREIGN KEY (project_id) REFERENCES public.timesheets(id), CONSTRAINT fk_team_member_compensations_timesheet_scope FOREIGN KEY (project_id) REFERENCES public.timesheets(user_id), CONSTRAINT fk_team_member_compensations_timesheet_scope FOREIGN KEY (project_id) REFERENCES public.timesheets(project_id), CONSTRAINT fk_team_member_compensations_timesheet_scope FOREIGN KEY (project_id) REFERENCES public.timesheets(company_id), CONSTRAINT fk_team_member_compensations_timesheet_scope FOREIGN KEY (timesheet_id) REFERENCES public.timesheets(id), CONSTRAINT fk_team_member_compensations_timesheet_scope FOREIGN KEY (timesheet_id) REFERENCES public.timesheets(user_id), CONSTRAINT fk_team_member_compensations_timesheet_scope FOREIGN KEY (timesheet_id) REFERENCES public.timesheets(project_id), CONSTRAINT fk_team_member_compensations_timesheet_scope FOREIGN KEY (timesheet_id) REFERENCES public.timesheets(company_id), CONSTRAINT fk_team_member_compensations_member_owner FOREIGN KEY (user_id) REFERENCES public.team_members(id), CONSTRAINT fk_team_member_compensations_member_owner FOREIGN KEY (user_id) REFERENCES public.team_members(user_id), CONSTRAINT fk_team_member_compensations_member_owner FOREIGN KEY (team_member_id) REFERENCES public.team_members(id), CONSTRAINT fk_team_member_compensations_member_owner FOREIGN KEY (team_member_id) REFERENCES public.team_members(user_id), CONSTRAINT team_member_compensations_timesheet_id_fkey FOREIGN KEY (timesheet_id) REFERENCES public.timesheets(id), CONSTRAINT team_member_compensations_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT team_member_compensations_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT team_member_compensations_project_id_fkey FOREIGN KEY (project_id) REFERENCES public.projects(id), CONSTRAINT team_member_compensations_team_member_id_fkey FOREIGN KEY (team_member_id) REFERENCES public.team_members(id), CONSTRAINT team_member_compensations_task_id_fkey FOREIGN KEY (task_id) REFERENCES

public.tasks(id)); CREATE TABLE public.team_members (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, name text NOT NULL, email text NOT NULL, role text DEFAULT 'viewer'::text CHECK (role = ANY (ARRAY['admin'::text, 'manager'::text, 'member'::text, 'viewer'::text])), joined_at date DEFAULT CURRENT_DATE, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), CONSTRAINT team_members_pkey PRIMARY KEY (id), CONSTRAINT team_members_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.timesheets (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid, task_id uuid, project_id uuid, client_id uuid, date date NOT NULL, start_time time without time zone, end_time time without time zone, duration_minutes integer, description text, status text DEFAULT 'draft'::text, created_at timestamp with time zone DEFAULT now(), notes text, billable boolean DEFAULT true, hourly_rate numeric DEFAULT NULL::numeric, invoice_id uuid, billed_at timestamp with time zone, service_id uuid, company_id uuid NOT NULL, executed_by_member_id uuid, CONSTRAINT timesheets_pkey PRIMARY KEY (id), CONSTRAINT timesheets_task_id_fkey FOREIGN KEY (task_id) REFERENCES public.tasks(id), CONSTRAINT timesheets_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT timesheets_invoice_id_fkey FOREIGN KEY (invoice_id) REFERENCES public.invoices(id), CONSTRAINT timesheets_service_id_fkey FOREIGN KEY (service_id) REFERENCES public.services(id), CONSTRAINT timesheets_company_id_fkey FOREIGN KEY (company_id) REFERENCES public.company(id), CONSTRAINT timesheets_executed_by_member_id_fkey FOREIGN KEY (executed_by_member_id) REFERENCES public.team_members(id), CONSTRAINT fk_timesheets_project_scope FOREIGN KEY (user_id) REFERENCES public.projects(id), CONSTRAINT fk_timesheets_project_scope FOREIGN KEY (user_id) REFERENCES public.projects(user_id), CONSTRAINT fk_timesheets_project_scope FOREIGN KEY (user_id) REFERENCES public.projects(company_id), CONSTRAINT fk_timesheets_project_scope FOREIGN KEY (project_id) REFERENCES public.projects(id), CONSTRAINT fk_timesheets_project_scope FOREIGN KEY (project_id) REFERENCES public.projects(user_id), CONSTRAINT fk_timesheets_project_scope FOREIGN KEY (project_id) REFERENCES public.projects(company_id), CONSTRAINT fk_timesheets_project_scope FOREIGN KEY (company_id) REFERENCES public.projects(id), CONSTRAINT fk_timesheets_project_scope FOREIGN KEY (company_id) REFERENCES public.projects(user_id), CONSTRAINT fk_timesheets_project_scope FOREIGN KEY (company_id) REFERENCES public.projects(company_id), CONSTRAINT fk_timesheets_client_scope FOREIGN KEY (user_id) REFERENCES public.clients(id), CONSTRAINT fk_timesheets_client_scope FOREIGN KEY (user_id) REFERENCES public.clients(user_id), CONSTRAINT fk_timesheets_client_scope FOREIGN KEY (user_id) REFERENCES public.clients(company_id), CONSTRAINT fk_timesheets_client_scope FOREIGN KEY (client_id) REFERENCES public.clients(id), CONSTRAINT fk_timesheets_client_scope FOREIGN KEY (client_id) REFERENCES public.clients(user_id), CONSTRAINT fk_timesheets_client_scope FOREIGN KEY (client_id) REFERENCES public.clients(company_id), CONSTRAINT fk_timesheets_client_scope FOREIGN KEY (company_id) REFERENCES public.clients(id), CONSTRAINT fk_timesheets_client_scope FOREIGN KEY (company_id) REFERENCES public.clients(user_id), CONSTRAINT fk_timesheets_client_scope FOREIGN KEY (company_id) REFERENCES public.clients(company_id)); CREATE TABLE public.user_accounting_settings (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL UNIQUE, country text NOT NULL DEFAULT 'BE'::text, is_initialized boolean DEFAULT false, auto_journal_enabled boolean DEFAULT true, fiscal_year_start text DEFAULT '01-01'::text, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(),

CONSTRAINT user_accounting_settings_pkey PRIMARY KEY (id), CONSTRAINT user_accounting_settings_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.user_company_preferences (user_id uuid NOT NULL, active_company_id uuid, updated_at timestamp with time zone DEFAULT now(), CONSTRAINT user_company_preferences_pkey PRIMARY KEY (user_id), CONSTRAINT user_company_preferences_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT user_company_preferences_active_company_id_fkey FOREIGN KEY (active_company_id) REFERENCES public.company(id)); CREATE TABLE public.user_credits (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL UNIQUE, free_credits integer DEFAULT 50, paid_credits integer DEFAULT 0, total_used integer DEFAULT 0, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), referral_code text UNIQUE, referred_by text, subscription_credits integer DEFAULT 0, subscription_plan_id uuid, stripe_customer_id text, stripe_subscription_id text, subscription_status text DEFAULT 'inactive'::text, current_period_end timestamp with time zone, free_credits_refreshed_at timestamp with time zone DEFAULT timezone('utc'::text, now()), CONSTRAINT user_credits_pkey PRIMARY KEY (id), CONSTRAINT user_credits_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id), CONSTRAINT user_credits_subscription_plan_id_fkey FOREIGN KEY (subscription_plan_id) REFERENCES public.subscription_plans(id)); CREATE TABLE public.user_roles (user_id uuid NOT NULL, role text NOT NULL CHECK (role = ANY (ARRAY['admin'::text, 'user'::text])), created_at timestamp with time zone NOT NULL DEFAULT now(), updated_at timestamp with time zone NOT NULL DEFAULT now(), CONSTRAINT user_roles_pkey PRIMARY KEY (user_id), CONSTRAINT user_roles_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id)); CREATE TABLE public.webhook_deliveries (id uuid NOT NULL DEFAULT gen_random_uuid(), webhook_endpoint_id uuid NOT NULL, event text NOT NULL, payload jsonb NOT NULL, status_code integer, response_body text, delivered boolean DEFAULT false, attempts integer DEFAULT 0, created_at timestamp with time zone DEFAULT now(), CONSTRAINT webhook_deliveries_pkey PRIMARY KEY (id), CONSTRAINT webhook_deliveries_webhook_endpoint_id_fkey FOREIGN KEY (webhook_endpoint_id) REFERENCES public.webhook_endpoints(id)); CREATE TABLE public.webhook_endpoints (id uuid NOT NULL DEFAULT gen_random_uuid(), user_id uuid NOT NULL, url text NOT NULL, secret text NOT NULL, events ARRAY NOT NULL, is_active boolean DEFAULT true, last_triggered_at timestamp with time zone, failure_count integer DEFAULT 0, created_at timestamp with time zone DEFAULT now(), updated_at timestamp with time zone DEFAULT now(), CONSTRAINT webhook_endpoints_pkey PRIMARY KEY (id), CONSTRAINT webhook_endpoints_user_id_fkey FOREIGN KEY (user_id) REFERENCES auth.users(id));

Prd App Gestion Rh Materielle

Product Requirements Document (PRD) – Version 2

Application SaaS de gestion des ressources humaines et matérielles orientée projets

Version: 2.0
Statut: Document de cadrage produit détaillé
Type de produit: Plateforme web SaaS modulaire
Positionnement: Gestion opérationnelle, analytique et paie préparatoire des ressources

humaines et matérielles
Cible: Entreprises multi-projets mobilisant personnel, équipements et coûts analytiques

---

1. Executive Summary

L'application a pour objectif de centraliser dans une plateforme unique la gestion des ressources humaines et matérielles intervenant sur des projets, depuis leur création jusqu'au suivi réel du temps, de l'usage, des coûts et de la préparation des éléments variables de paie.

Le produit cible les organisations qui gèrent simultanément :

- des collaborateurs internes et externes,
- des équipements ou matériels affectables,
- des projets et tâches nécessitant une planification fine,
- des obligations RH et de paie,
- des besoins de suivi de coûts, de capacité et de rentabilité.

La valeur produit repose sur 4 promesses principales :

1. **Planifier correctement** les ressources humaines et matérielles.
2. **Collecter fiablement** les temps et usages réels.
3. **Préparer sans erreur** les éléments variables de paie.
4. **Piloter avec visibilité** les coûts, charges et écarts projet.

---

2. Vision produit

Construire une plateforme SaaS unifiée qui permette à une société de services, d'ingénierie, de maintenance, de BTP ou d'exploitation multi-projets de gérer l'ensemble de ses ressources humaines et matérielles, d'en suivre l'affectation et la consommation réelle, et d'automatiser la préparation de la paie ainsi que le pilotage opérationnel et financier.

---

3. Problème à résoudre

Les entreprises gérant des ressources affectées à des projets rencontrent généralement les difficultés suivantes :

- outils dispersés entre Excel, emails, ERP, RH, paie et planification,
- absence de référentiel unique sur les collaborateurs et matériels,
- saisies manuelles redondantes,
- timesheets incomplètes ou validées tardivement,
- faible traçabilité des heures supplémentaires et absences,

- manque de visibilité sur l'utilisation du matériel,
- difficulté à calculer le coût réel d'un projet,
- erreurs ou retards dans la préparation de la paie.

Le produit doit supprimer ces ruptures de processus et offrir une chaîne continue entre **prévision**, **affectation**, **exécution**, **validation**, **consolidation** et **reporting**.

---

4. Objectifs business

4.1 Objectifs stratégiques

- Centraliser les données RH, matériel, projet et paie dans une seule plateforme.
- Réduire les erreurs opérationnelles et de paie liées aux temps et usages.
- Fiabiliser les coûts analytiques par projet.
- Améliorer le taux d'utilisation des ressources humaines et matérielles.
- Réduire le délai de clôture mensuelle.
- Renforcer la conformité et l'auditabilité des processus.

4.2 Objectifs mesurables

- 95% des timesheets soumises dans les délais.
- Réduction de 60% des erreurs de préparation de paie liées aux feuilles de temps.
- Réduction de 50% du temps de consolidation mensuelle RH/projets.
- 100% des validations critiques traçables.
- Disponibilité des tableaux de bord consolidés en J+1 après clôture.

---

5. Positionnement produit

5.1 Nature du produit

- Application web SaaS multi-tenant
- Possibilité d'hébergement dédié pour grands comptes
- Architecture modulaire et API-first
- Cœur orienté opérations + RH + contrôle de gestion + paie préparatoire

5.2 Différenciation

Le produit ne se limite ni à un outil RH ni à un outil de gestion de projet. Il relie :

- l'employé,

- le matériel,
- la mission/projet,
- le temps réel,
- le coût réel,
- la validation,
- la préparation de la paie.

---

6. Périmètre fonctionnel

6.1 In scope – MVP

- Référentiels société, entités, sites, centres de coûts
- Gestion des employés
- Gestion des contrats et paramètres RH de base
- Gestion des ressources matérielles
- Gestion des projets, tâches et activités
- Affectation des ressources humaines et matérielles
- Planning de charge et disponibilité
- Timesheets RH
- Journaux d’utilisation des matériels
- Gestion des absences et congés
- Calcul des heures normales et supplémentaires
- Préparation des éléments variables de paie
- Export vers logiciel de paie
- Reporting opérationnel et analytique standard
- Gestion des rôles, permissions, audit et notifications

6.2 Out of scope – MVP

- Calcul complet du bulletin de paie multi-législation
- Comptabilité générale complète
- ATS / recrutement avancé
- Gestion achat-stock-maintenance complexe
- Facturation client avancée

- BI avancée embarquée
- IA de prévision de charge en V1

6.3 Roadmap post-MVP

- Portail collaborateur avancé
- Mobile natif iOS/Android
- Signature électronique RH
- Facturation projet
- Budget prévisionnel et simulation de scénarios
- Maintenance préventive du matériel
- Optimisation automatique d'affectation
- Intégration ERP / comptabilité / badgeuse / SIRH approfondie

---

7. Personae

7.1 Administrateur plateforme

Objectif: Configurer la plateforme, gouverner les droits, référentiels et workflows.
Douleurs: Complexité des paramétrages, dépendance à l'IT, manque de contrôle sur la sécurité.
Besoins: Paramétrage fin, historisation, multi-entités, gestion des habilitations.

7.2 Responsable RH

Objectif: Gérer les salariés, absences, contrats et règles RH.
Douleurs: Données dispersées, manque de visibilité sur les anomalies.
Besoins: Référentiel fiable, workflow de validation, contrôle amont de la paie.

7.3 Gestionnaire paie

Objectif: Générer des éléments variables exacts et exploitables.
Douleurs: Données incomplètes, corrections de dernière minute, heures supplémentaires non tracées.
Besoins: Contrôles, journal de recalcul, exports standardisés, audit complet.

7.4 Chef de projet

Objectif: Affecter les bonnes ressources, suivre le consommé réel et les coûts.
Douleurs: Manque de visibilité sur disponibilité et charge, validation tardive des temps.
Besoins: Planning clair, alertes de surcharge, validation rapide, suivi budget vs réel.

7.5 Manager d'équipe

Objectif: Superviser capacité, présence, performance et allocation.
Douleurs: Absences découvertes tardivement, faible qualité des timesheets.
Besoins: Vue équipe, workflow simple, indicateurs d'écarts.

7.6 Collaborateur

Objectif: Déclarer simplement son temps, ses absences et consulter ses affectations.
Douleurs: Saisie longue, manque de lisibilité, règles peu claires.
Besoins: Saisie guidée, préremplissage, notifications, historique personnel.

7.7 Gestionnaire matériel / logistique

Objectif: Suivre le parc, sa disponibilité et son coût d'utilisation.
Douleurs: Affectations non tracées, matériel indisponible au mauvais moment.
Besoins: Inventaire centralisé, statut temps réel, historique, alertes maintenance.

7.8 Direction / contrôle de gestion

Objectif: Piloter charge, coûts, rentabilité et fiabilité opérationnelle.
Douleurs: Données peu consolidées, indicateurs arrivant trop tard.
Besoins: Tableaux de bord consolidés, coûts analytiques fiables, drill-down par projet.

---

8. Architecture fonctionnelle

Le produit est structuré en 9 domaines :

1. **Core Administration**
2. **Ressources humaines**
3. **Ressources matérielles**
4. **Projets & tâches**
5. **Planning & affectation**
6. **Temps & usages (timesheets)**
7. **Absences & disponibilités**
8. **Paie préparatoire**
9. **Reporting & audit**

---

9. Modules détaillés

9.1 Module Administration & Référentiels

Finalité

Fournir le socle commun de configuration métier et sécurité.

Fonctionnalités

- Gestion des sociétés / entités légales
- Gestion des sites / agences / ateliers

- Gestion des départements / équipes
- Gestion des centres de coût
- Gestion des calendriers de travail
- Gestion des jours fériés
- Paramétrage des types d'absence
- Paramétrage des types d'heures
- Paramétrage des primes / indemnités / retenues
- Gestion des rôles et permissions
- Paramétrage des workflows de validation
- Paramétrage des cycles de clôture
- Paramétrage des règles d'arrondi, seuils et plafonds

Critères d'acceptation

- Un administrateur peut créer une société et ses sites.
- Chaque paramètre peut être historisé avec date d'effet.
- Les droits peuvent être restreints par société, rôle, département et projet.

9.2 Module Ressources humaines

Finalité

Centraliser le référentiel des collaborateurs mobilisables.

Fonctionnalités

- Création et mise à jour du dossier employé
- Gestion du matricule et identifiants internes
- Gestion du poste, département, manager
- Gestion du contrat et dates d'effet
- Gestion du taux horaire / journalier / coût employeur
- Gestion du calendrier individuel
- Gestion des compétences et certifications
- Gestion des statuts (actif, suspendu, sorti, en congé, indisponible)
- Pièces jointes RH
- Historique des changements

Données clés

- Identité
- Type de contrat
- Centre de coût
- Coût interne
- Salaire de référence
- Devise
- Temps contractuel
- Compétences
- Séniorité

Critères d'acceptation

- Un RH peut créer un employé actif avec date d'entrée.
- Un salarié sorti n'apparaît plus dans les affectations futures sauf filtre dédié.
- Toute modification de contrat est historisée.

9.3 Module Ressources matérielles

Finalité

Gérer le parc matériel mobilisable sur les projets.

Fonctionnalités

- Création fiche matériel
- Classification par catégorie
- Gestion numéro de série / code inventaire
- Gestion statut disponible / affecté / maintenance / hors service
- Gestion localisation actuelle
- Gestion coût d'acquisition et coût d'usage
- Gestion capacité ou métrique métier (heures, kilomètres, cycles, jours)
- Gestion planning de maintenance
- Historique des affectations
- Historique des indisponibilités

Critères d'acceptation

- Un matériel en maintenance ne peut pas être affecté.
- Chaque matériel possède un coût d'usage par unité configurable.
- L'historique des affectations est consultable par période.

9.4 Module Projets & Tâches

Finalité

Structurer l'activité projet qui consommera les ressources.

Fonctionnalités

- Création projet
- Définition client interne/externe
- Définition responsable projet
- Définition dates prévisionnelles
- Définition budget global et budget par tâche
- Structuration en lots, phases, tâches, activités
- Statuts projet (brouillon, actif, suspendu, clos)
- Rattachement centre de coût / code analytique
- Rattachement ressources prévues

Critères d'acceptation

- Un projet peut contenir plusieurs phases et tâches.
- Chaque tâche peut recevoir des ressources humaines et matérielles.
- Les budgets prévisionnels sont comparables aux consommés réels.

9.5 Module Planning & Affectation

Finalité

Visualiser et gérer la capacité, la disponibilité et les affectations.

Fonctionnalités

- Planning par ressource, équipe, projet, matériel
- Vue jour / semaine / mois
- Affectation manuelle ou groupée
- Détection de conflits de disponibilité

- Contrôle de surcharge
- Réaffectation rapide
- Capacité théorique vs planifiée vs réelle
- Gestion des indisponibilités bloquantes

Critères d'acceptation

- Une ressource absente sur une période n'est pas affectable sans dérogation.
- Une ressource surchargée déclenche une alerte.
- Un chef de projet peut voir le taux de charge d'une équipe avant affectation.

9.6 Module Timesheets RH

Finalité

Capturer le temps réel passé par les collaborateurs.

Fonctionnalités

- Saisie journalière / hebdomadaire / mensuelle configurable
- Préremplissage depuis planning
- Saisie par projet / tâche / activité / code analytique
- Types d'heures : normales, supplémentaires, nuit, week-end, jour férié, astreinte, non productif
- Soumission pour validation
- Validation mono ou multi-niveau
- Rejet avec commentaire
- Réouverture contrôlée après validation
- Clôture de période

Données de ligne

- Date
- Ressource
- Projet
- Tâche / activité
- Nombre d'heures
- Type d'heure

- Commentaire
- Justificatif
- Statut

Critères d’acceptation

- Un collaborateur peut soumettre une feuille complète.
- Une feuille validée devient verrouillée selon les règles de clôture.
- Les heures supplémentaires sont isolées pour la paie.

9.7 Module Journal d’utilisation matériel

Finalité

Capturer l’utilisation réelle et le coût d’usage du matériel.

Fonctionnalités

- Déclaration du temps d’utilisation ou unité consommée
- Association à projet / tâche / site / opérateur
- Calcul du coût d’usage
- Validation par responsable
- Blocage si matériel indisponible
- Écart prévu vs réel
- Historique détaillé d’usage

Critères d’acceptation

- Le coût d’usage est calculé automatiquement à partir du référentiel matériel.
- Les imputations sur matériel indisponible sont refusées.
- L’utilisation est visible par projet et par matériel.

9.8 Module Absences & Congés

Finalité

Gérer les indisponibilités impactant la capacité, les temps et la paie.

Fonctionnalités

- Demandes d’absence
- Validation hiérarchique

- Typologie configurable : congé payé, RTT, maladie, sans solde, formation, mission, récupération
- Calcul des soldes
- Impact automatique sur planning
- Impact automatique sur timesheets
- Historique des validations et refus

Critères d'acceptation

- Une absence validée réduit la capacité disponible.
- Une journée entièrement absente empêche la saisie d'heures productives, sauf règles d'exception.
- Les absences non payées remontent en paie préparatoire.

9.9 Module Paie préparatoire

Finalité

Produire des éléments variables fiables à destination de la paie.

Fonctionnalités

- Consolidation des heures validées
- Calcul heures normales
- Calcul heures supplémentaires par tranche
- Calcul nuit / week-end / férié
- Primes et indemnités paramétrables
- Gestion retenues / absences non payées
- Contrôles d'anomalies avant clôture
- Génération d'état préparatoire par salarié
- Exports paie CSV/API
- Journal de recalcul et versioning des périodes

Sorties attendues

- État variables de paie par salarié
- État des anomalies
- Export paie

- Journal de clôture de paie préparatoire

Critères d’acceptation

- Seules les données validées sont prises en compte.
- Chaque recalcul laisse une trace.
- Un export peut être relancé pour une période avec numéro de version.

9.10 Module Reporting & Audit

Finalité

Donner une lecture consolidée et actionnable de l’activité.

Tableaux de bord

- Taux de saisie des timesheets
- Taux de validation dans les délais
- Charge / capacité RH
- Utilisation du matériel
- Heures supplémentaires par équipe
- Absences par type
- Coûts RH par projet
- Coûts matériels par projet
- Consommé vs budget
- Rentabilité projet
- Anomalies paie
- État de clôture mensuelle

Audit

- Historique des actions par utilisateur
- Historique des validations
- Historique des changements sur règles et données sensibles

Critères d’acceptation

- Tous les objets critiques sont auditables.
- Les rapports sont exportables en CSV, Excel et PDF.
- Les dashboards sont filtrables par société, site, période, projet et manager.

10. Workflows clés

10.1 Workflow timesheet RH

1. Préremplissage à partir du planning
2. Saisie ou correction par le collaborateur
3. Soumission
4. Validation manager ou chef de projet
5. Éventuelle validation RH/paie selon règle
6. Clôture de période
7. Intégration dans la paie préparatoire

10.2 Workflow absence

1. Demande absence
2. Validation manager
3. Contrôle RH si nécessaire
4. Mise à jour capacité/planning
5. Remontée paie si impact financier

10.3 Workflow matériel

1. Création ou import du matériel
2. Mise à disposition
3. Affectation au projet
4. Déclaration d’usage
5. Validation / contrôle
6. Analyse coûts et disponibilité

10.4 Workflow paie préparatoire

1. Collecte des données validées
2. Application des règles
3. Détection anomalies
4. Correction / réouverture éventuelle
5. Validation paie préparatoire
6. Export vers logiciel de paie

11. Règles métier prioritaires

1. Une donnée non validée ne peut pas alimenter la paie.
2. Les ressources humaines et matérielles doivent être rattachées à une entité et un statut actif pour être affectables.
3. Un matériel indisponible, en maintenance ou hors service ne peut pas être imputé.
4. Une absence validée doit impacter automatiquement la capacité et les propositions de saisie.
5. Les heures supplémentaires doivent être distinctes des heures normales.
6. Toute réouverture de période doit être tracée avec motif.
7. Les coûts RH et matériels doivent être calculables séparément par projet.
8. Les données clôturées doivent être immuables sans autorisation spéciale.
9. Les règles de majoration doivent être paramétrables par société.
10. Les exports paie doivent être versionnés par période.

12. Rôles & permissions

Rôle	Créer	Modifier	Valider	Voir reporting	Export paie	Administration
Administrateur	Oui	Oui	Oui	Oui	Oui	Oui
RH	Oui	Oui	Oui	Oui	Selon droit	Non
Paie	Non	Contrôlé	Oui	Oui	Oui	Non
Chef de projet	Affectations	Oui projet	Oui temps/projet	Oui projet	Non	Non
Manager	Non	Équipe	Oui équipe	Oui équipe	Non	Non
Collaborateur	Saisie perso	Saisie perso	Non	Vue perso	Non	Non
Gestionnaire matériel	Oui matériel	Oui matériel	Oui usage matériel	Oui matériel	Non	Non
Direction	Non	Non	Non	Oui	Non	Non

13. Exigences non fonctionnelles

13.1 Architecture

- SaaS multi-tenant
- API-first
- Services modulaires
- Journalisation centralisée
- Séparation configuration / transaction / analytique

13.2 Performance

- Temps de réponse < 3 secondes sur 95% des écrans standards
- Export standard < 60 secondes pour une période mensuelle standard
- Génération timesheets préremplies < 30 secondes pour une équipe standard

13.3 Sécurité

- Authentification forte
- SSO SAML/OIDC souhaité
- RBAC granulaire
- Chiffrement TLS en transit
- Chiffrement des données sensibles au repos
- Logs d'accès et actions critiques
- Conformité RGPD

13.4 Disponibilité

- Objectif SLA 99,5% minimum
- Sauvegardes quotidiennes
- RPO/RTO à définir selon cible client

13.5 Scalabilité

- Support initial 500 utilisateurs concurrents
- Support multi-sites, multi-sociétés, multi-pays à terme

13.6 UX

- Responsive desktop/tablette minimum
- Parcours de saisie rapide
- Notifications contextualisées
- Accessibilité standard web

14. Architecture applicative recommandée

14.1 Frontend

- Application web SPA
- Back-office administrateur
- Portail collaborateur
- Portail manager / chef de projet

14.2 Backend

- API Gateway
- Service Identity & Access
- Service RH
- Service Ressources matérielles
- Service Projet & Affectation
- Service Timesheets
- Service Absences
- Service Paie préparatoire
- Service Reporting / Export
- Service Notifications
- Service Audit

14.3 Data

- Base transactionnelle principale
- Stockage documentaire pour pièces jointes
- Base analytique / vues matérialisées pour reporting
- Bus d’événements ou file de traitement pour exports et calculs

14.4 Intégrations

- SSO / annuaire
- Paie
- ERP / comptabilité
- CSV / Excel
- Email / SMS / Teams / Slack selon cible

15. Modèle de données conceptuel

Entités principales

- Company
- LegalEntity
- Site
- Department
- CostCenter
- Employee
- EmploymentContract
- Skill
- Certification
- MaterialAsset
- AssetCategory
- AssetMaintenanceWindow
- Project
- ProjectPhase
- ProjectTask
- HumanAssignment
- AssetAssignment
- Timesheet
- TimesheetLine
- AssetUsageLog
- LeaveRequest
- LeaveBalance
- PayrollPeriod
- PayrollVariableItem
- PremiumRule
- ApprovalStep
- Notification

- AuditLog
- Attachment

Relations majeures

- Un Employee possède 0..n contrats
- Un Employee appartient à un Department et un Manager
- Un Project contient 0..n tâches
- Un ProjectTask reçoit 0..n affectations RH et matériel
- Une Timesheet appartient à un Employee et contient 1..n lignes
- Un PayrollPeriod agrège 0..n PayrollVariableItem
- Un MaterialAsset possède 0..n usages et 0..n maintenances

---

16. Spécifications API – haut niveau

16.1 Domaines API

- /auth
- /employees
- /contracts
- /assets
- /projects
- /assignments
- /timesheets
- /asset-usage
- /leave-requests
- /payroll-periods
- /payroll-variables
- /reports
- /admin
- /audit

16.2 Exemples d’opérations

- POST /employees
- GET /employees/{id}

- POST /projects/{id}/assignments
- POST /timesheets/{id}/submit
- POST /timesheets/{id}/approve
- POST /asset-usage
- POST /leave-requests
- POST /payroll-periods/{id}/calculate
- POST /payroll-periods/{id}/export

16.3 Exigences API

- Pagination
- Filtres standards
- Idempotence sur actions critiques si nécessaire
- Webhooks pour intégrations
- Journal d'événements pour exports et recalculs

---

17. Backlog MVP priorisé

Epic 1 – Administration

User stories

- En tant qu'administrateur, je veux créer les sociétés, sites et centres de coût afin de structurer l'organisation.
- En tant qu'administrateur, je veux définir les rôles et permissions afin de sécuriser l'accès.
- En tant qu'administrateur, je veux paramétrer les types d'heures et absences afin d'adapter la plateforme au métier.

Epic 2 – Référentiel RH

User stories

- En tant que RH, je veux créer une fiche employé afin de rendre le collaborateur planifiable.
- En tant que RH, je veux historiser les contrats afin de sécuriser le calcul futur de paie.
- En tant que manager, je veux voir les compétences disponibles afin d'affecter les bonnes personnes.

Epic 3 – Référentiel matériel

User stories

- En tant que gestionnaire matériel, je veux enregistrer un équipement afin de l'affecter à des projets.
- En tant que gestionnaire matériel, je veux indiquer son statut afin d'éviter une affectation impossible.

Epic 4 – Projets & affectation

User stories

- En tant que chef de projet, je veux créer un projet structuré en tâches afin de suivre correctement les consommations.
- En tant que chef de projet, je veux affecter des ressources selon disponibilité afin d'éviter les conflits.

Epic 5 – Temps RH

User stories

- En tant que collaborateur, je veux saisir mes heures par projet afin qu'elles soient prises en compte.
- En tant que manager, je veux valider les feuilles de temps afin de confirmer la réalité du travail effectué.
- En tant que paie, je veux isoler les heures supplémentaires afin de préparer la paie.

Epic 6 – Usage matériel

User stories

- En tant qu'opérationnel, je veux déclarer l'usage d'un matériel afin d'imputer son coût réel au projet.
- En tant que gestionnaire matériel, je veux voir les utilisations par période afin de contrôler le parc.

Epic 7 – Absences

User stories

- En tant que salarié, je veux demander un congé afin qu'il soit pris en compte dans le planning.
- En tant que manager, je veux approuver ou refuser une absence afin de conserver la capacité nécessaire sur mes projets.

Epic 8 – Paie préparatoire

User stories

- En tant que gestionnaire paie, je veux consolider les temps validés afin de générer les variables de paie.
- En tant que gestionnaire paie, je veux détecter les anomalies avant export afin d'éviter les corrections après paie.

Epic 9 – Reporting

User stories

- En tant que direction, je veux visualiser les coûts RH et matériels par projet afin de piloter la rentabilité.
- En tant que manager, je veux suivre le taux de complétion des timesheets afin de sécuriser la clôture.

---

18. Priorisation MoSCoW

Fonctionnalité	Priorité
Référentiels société / RH / matériel	Must
Gestion projets et tâches	Must
Affectation RH et matériel	Must
Timesheets RH	Must
Journal usage matériel	Must
Absences et congés	Must
Validation workflows	Must
Préparation paie et export	Must
Reporting standard	Must
Mobile natif	Should
Signature électronique	Could
IA d'optimisation	Could
Calcul complet bulletin de paie	Won't (MVP)

---

19. KPIs produit

Adoption

- Taux d'utilisateurs actifs mensuels
- Taux de complétion des timesheets
- Taux de soumission dans les délais

Efficacité opérationnelle

- Temps moyen de validation des feuilles
- Nombre moyen de relances par période
- Délai de clôture mensuelle

Qualité

- Taux d'anomalies avant export paie
- Taux de rejet des feuilles
- Nombre de corrections post-clôture

Pilotage économique

- Taux d'occupation RH
- Taux d'utilisation matériel
- Écart budget vs réalisé
- Coût moyen par projet et par ressource

---

20. Hypothèses, dépendances et risques

Hypothèses

- Les règles RH internes sont documentées.
- Les catégories de matériel et unités de consommation sont identifiables.
- Les sociétés disposent d'un outil de paie cible pour import si la paie complète n'est pas internalisée.

Dépendances

- Référentiel salariés existant
- Référentiel matériel existant
- Arbitrages métier sur heures supplémentaires, primes et absences
- Format d'intégration avec paie/ERP

Risques majeurs

- Variabilité des règles de paie
- Données historiques peu fiables
- Complexité multi-pays
- Changements fréquents des règles métier
- Résistance à la discipline de saisie

Stratégies de mitigation

- Moteur de règles paramétrable
- Période pilote sur un périmètre réduit
- Imports contrôlés et nettoyage de données
- UX centrée sur rapidité de saisie
- Tableaux d'anomalies avant clôture

---

21. Plan de livraison recommandé

Phase 0 – Discovery & cadrage

- Ateliers métier RH, paie, projets, matériel
- Cartographie des règles métier
- Spécification des intégrations
- Cadrage des rôles et workflows

Phase 1 – MVP Core

- Référentiels
- RH de base
- Matériel de base
- Projets et affectations
- Timesheets RH
- Absences

Phase 2 – MVP Extended

- Journal usage matériel
- Consolidation paie préparatoire
- Reporting standard
- Audit et exports

Phase 3 – Industrialisation

- API avancées
- SSO
- reporting renforcé
- performance / sécurité / observabilité

Phase 4 – Scale

- Mobile
- multi-pays
- simulation budgétaire
- automatisations avancées

---

22. Critères de succès du MVP

Le MVP sera considéré réussi si :

- les collaborateurs peuvent saisir leurs temps sans recours à Excel,
- les managers valident les temps et absences dans l'outil,
- les matériels sont affectables et leur usage traçable,
- les éléments variables de paie sont générés à partir des données validées,
- les coûts RH et matériels sont visibles par projet,
- la clôture mensuelle est plus rapide et plus fiable qu'avant déploiement.

---

23. Recommandations produit et techniques

Recommandations produit

- Séparer clairement les notions de **prévu**, **saisi**, **validé**, **payé**.
- Concevoir une expérience de saisie très rapide, car c'est le point d'adoption principal.
- Mettre la qualité des règles et référentiels avant la sophistication des dashboards.
- Prévoir dès le départ le multi-entités et les dates d'effet.

Recommandations techniques

- Architecture orientée domaines métiers.
- Moteur de règles paramétrable pour heures, majorations, primes, retenues.
- Versioning des périodes de paie préparatoire.
- Event log pour audit et recalculs.
- API-first pour préparer les intégrations futures.

---

24. Open Questions

1. L'application doit-elle rester en **paie préparatoire** ou intégrer à terme le **calcul complet du bulletin** ?
2. Le périmètre doit-il couvrir les **prestataires externes** comme les salariés internes ?
3. La V1 doit-elle être **multi-sociétés** ou également **multi-pays** ?
4. Les matériels doivent-ils être suivis à l'heure, au jour, au cycle, au kilomètre ou selon plusieurs unités ?
5. La validation des timesheets doit-elle être **manager**, **chef de projet**, ou **double validation** ?
6. Les coûts affichés doivent-ils être **internes confidentiels** ou visibles aux chefs de projet ?
7. Quel logiciel de paie ou ERP sera la cible d'intégration ?

---

25. Annexes

25.1 Écrans principaux à concevoir

- Dashboard direction
- Dashboard manager
- Dashboard paie
- Liste employés
- Fiche employé
- Liste matériels
- Fiche matériel
- Liste projets
- Fiche projet avec budget et consommation
- Planning charge/capacité
- Écran timesheet collaborateur
- Écran validation manager
- Écran absences
- Écran calcul paie préparatoire
- Écran anomalies de paie
- Rapports analytiques

25.2 User stories critiques à développer en premier

- Créer un employé planifiable
- Créer un matériel affectable
- Créer un projet et des tâches
- Affecter ressource humaine et matérielle
- Saisir une timesheet RH
- Valider une timesheet RH
- Déclarer une absence
- Déclarer un usage matériel
- Calculer une période de paie préparatoire
- Exporter les variables de paie

25.3 Proposition de stack technique indicative

- Frontend: React / Next.js
- Backend: Node.js / NestJS ou Java / Spring Boot
- Base de données: PostgreSQL
- Cache: Redis
- File stockage: S3 compatible
- Auth: Keycloak / Auth0 / Azure AD
- BI/reporting: vues SQL + moteur de reporting intégré
- Messaging: RabbitMQ / Kafka selon volume

---

26. Backlog MVP détaillé – format Agile

26.1 Structure du backlog

Le backlog MVP est organisé selon la hiérarchie suivante :

- **Epic**
- **Feature**
- **User Story**
- **Critères d'acceptation**
- **Priorité**
- **Dépendances**

26.2 Epic A – Administration & Sécurité

Feature A1 – Gestion des organisations

User Story A1.1
En tant qu'administrateur, je veux créer une société, ses entités, sites et départements afin de structurer les données du système.

Critères d'acceptation

- Création d'une société avec code, nom, devise et fuseau horaire.
- Ajout d'un ou plusieurs sites rattachés.
- Ajout de départements et centres de coût.
- Possibilité d'activer/désactiver un élément sans suppression physique.

Priorité: Must
Dépendances: aucune

Feature A2 – Gestion des rôles et permissions

User Story A2.1
En tant qu'administrateur, je veux attribuer des rôles aux utilisateurs afin de contrôler l'accès aux fonctionnalités.

Critères d'acceptation

- Attribution d'un ou plusieurs rôles à un utilisateur.
- Restriction par société, site, département ou projet.
- Un utilisateur non autorisé ne peut pas accéder à un écran ou une action interdite.
- Toute modification de droits est auditée.

Priorité: Must
Dépendances: A1

Feature A3 – Paramétrage métier

User Story A3.1
En tant qu'administrateur, je veux paramétrer types d'heures, types d'absences et règles de validation afin d'adapter la plateforme à l'entreprise.

Critères d'acceptation

- Création de types d'heures avec code et libellé.
- Création de types d'absence avec impact paie oui/non.
- Paramétrage d'un workflow de validation à un ou deux niveaux.
- Historisation des paramètres avec date d'effet.

Priorité: Must
Dépendances: A1

26.3 Epic B – Référentiel RH

Feature B1 – Création d'un employé

User Story B1.1
En tant que RH, je veux créer la fiche d'un employé afin qu'il puisse être affecté sur des projets et saisir ses temps.

Critères d'acceptation

- Création avec identité, matricule, poste, manager, département, site.
- Affectation d'un calendrier de travail.
- Statut initial "actif" ou "brouillon".
- Contrôle d'unicité du matricule.

Priorité: Must
Dépendances: A1, A2

Feature B2 – Gestion des contrats

User Story B2.1
En tant que RH, je veux rattacher un contrat à un employé afin de stocker ses données de référence RH et coût.

Critères d'acceptation

- Contrat avec date de début et de fin optionnelle.
- Stockage type de contrat, temps contractuel, salaire de référence, taux horaire, centre de coût.
- Historique des avenants.
- Un seul contrat principal actif sur une même période sauf règle spécifique.

Priorité: Must
Dépendances: B1

Feature B3 – Compétences et disponibilité

User Story B3.1
En tant que manager, je veux consulter les compétences et la disponibilité d'un employé afin de l'affecter correctement.

Critères d'acceptation

- Ajout de compétences et niveaux.
- Ajout de certifications avec date d'expiration.

- Visualisation du taux de charge.
- Signalement des périodes d'indisponibilité.

Priorité: Should
Dépendances: B1, D1

26.4 Epic C – Référentiel Matériel

Feature C1 – Création d'une ressource matérielle

User Story C1.1
En tant que gestionnaire matériel, je veux enregistrer un équipement afin de le rendre affectable à un projet.

Critères d'acceptation

- Création avec code inventaire, catégorie, désignation, localisation, statut.
- Ajout du coût d'usage par unité.
- Pièces jointes possibles.
- Historique des statuts.

Priorité: Must
Dépendances: A1

Feature C2 – Indisponibilité et maintenance

User Story C2.1
En tant que gestionnaire matériel, je veux déclarer une maintenance afin d'empêcher les affectations durant cette période.

Critères d'acceptation

- Création d'une fenêtre de maintenance avec date début/fin.
- Le matériel passe au statut indisponible ou maintenance.
- Aucune affectation ou imputation n'est possible sur la période bloquée.
- Historique consultable.

Priorité: Must
Dépendances: C1

26.5 Epic D – Projets & Affectation

Feature D1 – Création projet

User Story D1.1
En tant que chef de projet, je veux créer un projet structuré afin de suivre les consommations humaines et matérielles.

Critères d'acceptation

- Création projet avec code, nom, dates prévisionnelles, responsable, centre de coût, budget.
- Ajout de phases, tâches et activités.
- Définition du statut du projet.
- Possibilité de clôturer un projet.

Priorité: Must
Dépendances: A1

Feature D2 – Affectation RH

User Story D2.1
En tant que chef de projet, je veux affecter un employé à une tâche afin de planifier sa charge.

Critères d'acceptation

- Affectation par période.
- Contrôle de disponibilité et surcharge.
- Affichage de la charge théorique avant validation.
- Possibilité de modifier ou retirer une affectation.

Priorité: Must
Dépendances: B1, D1

Feature D3 – Affectation matériel

User Story D3.1
En tant que chef de projet, je veux affecter un matériel à une tâche afin de réserver sa disponibilité.

Critères d'acceptation

- Affectation par période et projet.
- Refus si statut matériel incompatible.
- Historique des affectations.
- Vue consolidée par projet.

Priorité: Must
Dépendances: C1, C2, D1

Feature D4 – Vue planning

User Story D4.1
En tant que manager, je veux voir un planning charge/capacité afin d'anticiper les conflits d'allocation.

Critères d'acceptation

- Vue par jour, semaine, mois.
- Filtres par équipe, projet, site.
- Mise en évidence des surcharges.
- Vue séparée RH et matériel.

Priorité: Must
Dépendances: D2, D3

26.6 Epic E – Timesheets RH

Feature E1 – Saisie des temps

User Story E1.1
En tant que collaborateur, je veux saisir mes heures par projet et tâche afin que mon activité soit enregistrée.

Critères d'acceptation

- Saisie par jour ou semaine selon paramétrage.
- Sélection projet/tâche autorisés uniquement.
- Total journalier contrôlé selon règles.
- Sauvegarde brouillon.

Priorité: Must
Dépendances: B1, D1, D2

Feature E2 – Préremplissage

User Story E2.1
En tant que collaborateur, je veux que ma feuille de temps soit préremplie à partir de mes affectations afin de gagner du temps.

Critères d'acceptation

- Les affectations de la période apparaissent automatiquement.
- L'utilisateur peut ajuster les heures.
- Les absences validées sont visibles dans la période.
- Les lignes non autorisées ne sont pas proposées.

Priorité: Should
Dépendances: D2, G1

Feature E3 – Soumission et validation

User Story E3.1
En tant que collaborateur, je veux soumettre ma feuille afin qu'elle soit approuvée.

Critères d'acceptation

- Passage du statut brouillon à soumis.
- Vérification des champs obligatoires.
- Notification au valideur.
- Horodatage et trace de soumission.

Priorité: Must
Dépendances: E1, A3

User Story E3.2
En tant que manager ou chef de projet, je veux valider ou rejeter une feuille afin de contrôler les temps déclarés.

Critères d'acceptation

- Vue des feuilles en attente.
- Validation/rejet avec commentaire obligatoire en cas de rejet.
- Historique des actions.
- Verrouillage après validation selon règles de clôture.

Priorité: Must
Dépendances: E3.1

Feature E4 – Heures spéciales

User Story E4.1
En tant que gestionnaire paie, je veux distinguer les heures supplémentaires, nuit, week-end et férié afin de préparer la paie.

Critères d'acceptation

- Les types d'heures sont sélectionnables selon paramétrage.
- Les heures supplémentaires sont identifiées séparément.
- Les règles d'éligibilité sont appliquées.
- Les exports de paie récupèrent cette distinction.

Priorité: Must
Dépendances: A3, E1

26.7 Epic F – Usage Matériel

Feature F1 – Déclaration d’utilisation

User Story F1.1
En tant qu’opérationnel, je veux déclarer l’utilisation d’un matériel sur un projet afin que son coût soit imputé correctement.

Critères d’acceptation

- Saisie date, matériel, projet, tâche, unité consommée.
- Refus si le matériel est indisponible.
- Calcul automatique du coût d’usage.
- Sauvegarde brouillon ou soumission directe selon rôle.

Priorité: Must
Dépendances: C1, D1, D3

Feature F2 – Validation usage matériel

User Story F2.1
En tant que gestionnaire matériel ou chef de projet, je veux valider les usages déclarés afin de fiabiliser les coûts projet.

Critères d’acceptation

- Liste des usages en attente.
- Validation/rejet avec commentaire.
- Historique des modifications et validations.
- Les usages validés alimentent les coûts analytiques.

Priorité: Must
Dépendances: F1.1

26.8 Epic G – Absences & Congés

Feature G1 – Demande d’absence

User Story G1.1
En tant que collaborateur, je veux demander une absence afin qu’elle soit soumise à validation et prise en compte dans le planning.

Critères d’acceptation

- Création demande avec type, dates, commentaire et justificatif optionnel.
- Contrôle de cohérence des dates.

- Vérification du solde si applicable.
- Statut brouillon ou soumis.

Priorité: Must
Dépendances: B1, A3

Feature G2 – Validation absence

User Story G2.1
En tant que manager, je veux valider ou refuser une absence afin de maîtriser la disponibilité de l'équipe.

Critères d'acceptation

- Visualisation de l'impact sur le planning.
- Validation ou refus avec motif.
- Notification au salarié.
- Mise à jour automatique de la capacité disponible.

Priorité: Must
Dépendances: G1.1

Feature G3 – Impact sur temps et paie

User Story G3.1
En tant que gestionnaire paie, je veux que les absences validées impactent correctement la période de paie afin d'éviter les retraitements manuels.

Critères d'acceptation

- Une absence payée ou non payée est distinguée.
- Les jours absents ajustent la saisie possible.
- Les absences impactant la paie remontent dans le calcul préparatoire.
- Les anomalies de recouvrement temps/absence sont signalées.

Priorité: Must
Dépendances: G2, E1, H1

26.9 Epic H – Paie préparatoire

Feature H1 – Calcul de période

User Story H1.1
En tant que gestionnaire paie, je veux calculer une période de paie préparatoire afin de consolider les variables issues des temps et absences validés.

Critères d'acceptation

- Sélection d’une période ouverte.
- Collecte uniquement des données validées.
- Production des variables par salarié.
- Génération d’un journal de calcul.

Priorité: Must
Dépendances: E3.2, E4.1, G3.1

Feature H2 – Contrôle des anomalies

User Story H2.1
En tant que gestionnaire paie, je veux voir les anomalies avant export afin de corriger les données à la source.

Critères d’acceptation

- Affichage des salariés sans timesheet, des incohérences absence/temps, et des dépassements.
- Possibilité de filtrer par type d’anomalie.
- L’anomalie est liée à la donnée source.
- Export des anomalies.

Priorité: Must
Dépendances: H1

Feature H3 – Export paie

User Story H3.1
En tant que gestionnaire paie, je veux exporter les variables de paie afin de les intégrer dans mon logiciel de paie.

Critères d’acceptation

- Export CSV ou format paramétrable.
- Versioning de l’export.
- Horodatage et identité de l’utilisateur exporteur.
- Blocage ou avertissement si anomalies critiques ouvertes.

Priorité: Must
Dépendances: H1, H2

26.10 Epic I – Reporting & Pilotage

Feature I1 – Dashboard opérationnel

User Story I1.1
En tant que manager, je veux voir le taux de saisie et de validation des timesheets afin de sécuriser la clôture.

Critères d'acceptation

- Vue par équipe et période.
- Nombre de feuilles brouillon, soumises, validées, rejetées.
- Filtre par site et manager.
- Export des résultats.

Priorité: Must
Dépendances: E3

Feature I2 – Dashboard coûts projet

User Story I2.1
En tant que direction ou chef de projet, je veux voir les coûts RH et matériels par projet afin d'évaluer les écarts par rapport au budget.

Critères d'acceptation

- Affichage du coût RH cumulé.
- Affichage du coût matériel cumulé.
- Comparaison budget vs réel.
- Filtre par période, projet, société.

Priorité: Must
Dépendances: F2, H1

Feature I3 – Audit

User Story I3.1
En tant qu'administrateur ou auditeur, je veux consulter l'historique des changements afin de garantir la traçabilité du système.

Critères d'acceptation

- Historique des créations, validations, rejets, exports.
- Filtre par utilisateur, type d'objet et période.
- Consultation sécurisée.
- Export audit.

Priorité: Must
Dépendances: A2

26.11 Priorisation Sprint par Sprint

Sprint 1

- A1 Gestion des organisations
- A2 Gestion des rôles
- B1 Création employé
- C1 Création matériel
- D1 Création projet

Sprint 2

- B2 Contrats
- D2 Affectation RH
- D3 Affectation matériel
- D4 Vue planning

Sprint 3

- E1 Saisie temps
- E3 Soumission/validation temps
- G1 Demande absence
- G2 Validation absence

Sprint 4

- E4 Heures spéciales
- F1 Déclaration usage matériel
- F2 Validation usage matériel
- G3 Impact absence sur temps/paie

Sprint 5

- H1 Calcul période paie
- H2 Contrôle anomalies
- H3 Export paie
- I1 Dashboard opérationnel

Sprint 6

- I2 Dashboard coûts projet
- I3 Audit

- E2 Préremplissage
- B3 Compétences et disponibilité

26.12 Définition of Done (DoD)

Une user story est considérée terminée si :

- les critères d'acceptation sont validés,
- les tests fonctionnels passent,
- les règles de sécurité et permissions sont appliquées,
- l'audit est disponible pour les actions critiques,
- la documentation produit ou technique minimale est mise à jour,
- la feature est démontrable dans un environnement de recette.

26.13 Définition of Ready (DoR)

Une user story est prête à entrer en sprint si :

- la valeur métier est claire,
- les dépendances sont identifiées,
- les critères d'acceptation sont définis,
- les maquettes ou règles nécessaires existent,
- le périmètre est assez petit pour être livré dans le sprint.

---

27. Spécifications fonctionnelles détaillées – MVP

27.1 Objectif de cette section

Cette section traduit le PRD et le backlog MVP en spécifications fonctionnelles exploitables par les équipes produit, UX, technique et QA. Elle décrit, pour chaque module prioritaire :

- les écrans,
- les champs,
- les règles de gestion,
- les statuts,
- les validations,
- les actions utilisateur,

- les notifications,
- les cas limites.

27.2 Module Administration & Référentiels

27.2.1 Écran – Liste des organisations

Objectif: permettre à l'administrateur de visualiser et gérer les sociétés, sites, départements et centres de coût.

Colonnes

- Code
- Nom
- Type (société, site, département, centre de coût)
- Entité parente
- Statut
- Date de création
- Dernière modification

Actions

- Créer
- Modifier
- Désactiver
- Réactiver
- Rechercher
- Filtrer

Filtres

- Type
- Société
- Statut
- Date de création

Règles de gestion

- Un centre de coût doit être rattaché à une société.
- Un site ne peut pas être supprimé s'il est référencé par un employé actif.

- La désactivation est préférée à la suppression.

27.2.2 Écran – Formulaire organisation

Champs obligatoires

- Type d’organisation
- Code
- Nom
- Société de rattachement si applicable
- Devise pour une société
- Fuseau horaire pour une société
- Statut

Validations

- Le code doit être unique dans le périmètre défini.
- Le nom ne peut pas être vide.
- Une devise est obligatoire pour une société.

27.2.3 Écran – Gestion des rôles et permissions

Objectif: attribuer des habilitations par rôle et périmètre.

Champs

- Utilisateur
- Rôle
- Société
- Site
- Département
- Périmètre projet optionnel
- Date d’effet
- Date de fin optionnelle

Règles de gestion

- Un utilisateur peut cumuler plusieurs rôles.
- En cas de conflit, le refus prime sur l’autorisation implicite.
- Toute modification de droit doit être journalisée.

27.2.4 Écran – Paramétrage métier

Sous-référentiels à gérer

- Types d’heures
- Types d’absence
- Types de primes
- Types de retenues
- Workflows de validation
- Seuils de surcharge
- Règles d’arrondi

Champs standards

- Code
- Libellé
- Description
- Société de rattachement
- Actif oui/non
- Date d’effet
- Date de fin

Règles de gestion

- Aucun paramètre utilisé dans une période clôturée ne peut être supprimé.
- Toute modification doit créer une nouvelle version si la date d’effet change.

27.3 Module Ressources humaines

27.3.1 Écran – Liste des employés

Colonnes

- Matricule
- Nom complet
- Poste
- Département
- Manager
- Site
- Type de contrat

- Statut
- Date d’entrée
- Taux d’occupation actuel

Actions

- Créer une fiche
- Modifier
- Voir détail
- Archiver / sortie
- Importer
- Exporter

Filtres

- Société
- Site
- Département
- Manager
- Statut
- Type de contrat

27.3.2 Écran – Fiche employé

Sections

1. Informations générales
2. Organisation
3. Contrat actuel
4. Coûts et paramètres RH
5. Compétences et certifications
6. Disponibilités / indisponibilités
7. Documents
8. Historique

Champs obligatoires – Informations générales

- Matricule
- Nom

- Prénom
- Email professionnel ou identifiant
- Société
- Site
- Département
- Manager principal
- Date d’entrée
- Statut

Champs contrat de base

- Type de contrat
- Temps contractuel
- Date de début
- Date de fin optionnelle
- Taux horaire ou coût journalier
- Centre de coût
- Calendrier de travail

Règles de gestion

- Le matricule est unique par société.
- Un employé “sorti” ne peut plus être affecté après sa date de sortie.
- Un employé sans calendrier ne peut pas être planifié.
- Un employé doit avoir un manager ou un validateur alternatif.

Statuts possibles

- Brouillon
- Actif
- Suspendu
- En congé longue durée
- Sorti

27.3.3 Écran – Contrats et avenants

Colonnes

- Employé

- Type de contrat
- Date début
- Date fin
- Salaire de référence
- Taux horaire
- Temps contractuel
- Statut

Actions

- Ajouter contrat
- Ajouter avenant
- Clôturer
- Consulter historique

Règles de gestion

- Deux contrats principaux actifs sur la même période sont interdits sauf paramétrage spécifique.
- Les avenants reprennent le contrat source et changent seulement les champs modifiés.

27.3.4 Écran – Compétences

Champs

- Compétence
- Niveau
- Date d’obtention
- Date d’expiration optionnelle
- Commentaire

Règles

- Une certification expirée doit être visible comme non valide.
- Les compétences doivent être filtrables à l’affectation.

27.4 Module Ressources matérielles

27.4.1 Écran – Liste du parc matériel

Colonnes

- Code inventaire
- Désignation
- Catégorie
- Site / localisation
- Statut
- Coût d’usage
- Dernière affectation
- Prochaine maintenance

Actions

- Créer matériel
- Modifier
- Déclarer maintenance
- Changer statut
- Voir historique

Filtres

- Catégorie
- Statut
- Site
- Disponibilité

27.4.2 Écran – Fiche matériel

Sections

1. Informations générales
2. Statut et disponibilité
3. Coûts
4. Capacités et unités
5. Maintenance
6. Historique des affectations
7. Documents

Champs obligatoires

- Code inventaire

- Désignation
- Catégorie
- Société
- Site ou localisation initiale
- Statut
- Unité de consommation principale
- Coût d’usage

Unités possibles

- Heure
- Jour
- Kilomètre
- Cycle
- Quantité

Règles de gestion

- Un matériel en maintenance est non affectable.
- Un matériel hors service est non imputable.
- Un matériel peut avoir plusieurs unités secondaires mais une unité principale de calcul.

27.4.3 Écran – Maintenance / indisponibilité

Champs

- Matériel
- Type d’indisponibilité
- Date/heure début
- Date/heure fin
- Motif
- Commentaire
- Pièce jointe optionnelle

Validations

- La date de fin doit être postérieure à la date de début.
- Les périodes d’indisponibilité ne doivent pas se chevaucher pour un même matériel sans règle spéciale.

27.5 Module Projets & Tâches

27.5.1 Écran – Liste des projets

Colonnes

- Code projet
- Nom
- Client
- Responsable
- Date début
- Date fin
- Budget
- Consommé RH
- Consommé matériel
- Statut

Actions

- Créer
- Modifier
- Clôturer
- Consulter budget/consommation
- Voir affectations

Filtres

- Société
- Responsable projet
- Statut
- Client
- Période

27.5.2 Écran – Fiche projet

Sections

1. Informations générales
2. Budget

3. Phases et tâches
4. Affectations RH
5. Affectations matériel
6. Consommations réelles
7. Risques / alertes
8. Historique

Champs obligatoires

- Code projet
- Nom
- Société
- Responsable projet
- Date début prévisionnelle
- Centre de coût
- Statut

Champs optionnels

- Client
- Date fin prévisionnelle
- Budget RH
- Budget matériel
- Budget global
- Description

Règles de gestion

- Un projet clos n’accepte plus d’affectation ni de timesheet sur des dates post-clôture.
- Une tâche doit appartenir à un projet.
- Les tâches peuvent être imputables ou non imputables.

27.5.3 Écran – Gestion des tâches

Champs

- Code tâche
- Libellé
- Phase

- Responsable
- Date début
- Date fin
- Budget heures
- Budget coût
- Statut
- Imputable oui/non

Règles

- Une tâche non imputable ne doit pas être proposée dans la saisie des temps.
- Les budgets tâche doivent pouvoir être agrégés au niveau projet.

27.6 Module Planning & Affectation

27.6.1 Écran – Planning global

Vues disponibles

- Par personne
- Par équipe
- Par matériel
- Par projet
- Par semaine / mois

Éléments affichés

- Capacité théorique
- Capacité disponible
- Charge planifiée
- Absences
- Maintenances
- Surcharges

Actions

- Créer affectation
- Modifier affectation
- Déplacer affectation

- Supprimer affectation
- Voir conflits

27.6.2 Écran – Affectation RH

Champs obligatoires

- Projet
- Tâche
- Employé
- Date début
- Date fin
- Charge planifiée (heures ou pourcentage)
- Type d’affectation

Champs optionnels

- Commentaire
- Priorité
- Remplaçant

Règles de gestion

- L’employé doit être actif sur la période.
- L’employé doit disposer d’une capacité disponible.
- Une alerte apparaît si la charge dépasse le seuil paramétré.
- Une affectation sur absence validée est bloquée ou signalée selon paramétrage.

27.6.3 Écran – Affectation matériel

Champs obligatoires

- Projet
- Tâche
- Matériel
- Date début
- Date fin
- Quantité ou intensité planifiée si applicable

Règles de gestion

- Le matériel doit être disponible sur la période.

- La période ne doit pas chevaucher une maintenance bloquante.
- Les conflits d’affectation sont signalés.

27.7 Module Timesheets RH

27.7.1 Écran – Saisie timesheet collaborateur

Modes

- Vue hebdomadaire
- Vue mensuelle
- Saisie manuelle
- Préremplissage depuis planning

Colonnes de saisie

- Date
- Projet
- Tâche
- Type d’heure
- Nombre d’heures
- Commentaire
- Statut de ligne

Actions

- Ajouter ligne
- Supprimer ligne
- Copier la veille
- Préremplir
- Enregistrer brouillon
- Soumettre

Règles de gestion

- Seules les tâches imputables et autorisées sont sélectionnables.
- Le total journalier ne peut pas dépasser le plafond paramétré sauf dérogation.
- Les dates en dehors de la période ouverte sont verrouillées.
- Les jours d’absence validée limitent ou empêchent la saisie selon règle.

Validations

- Projet obligatoire
- Tâche obligatoire si le projet impose une granularité tâche
- Type d’heure obligatoire
- Nombre d’heures > 0
- Commentaire obligatoire pour certaines lignes exceptionnelles

27.7.2 Écran – Liste des timesheets

Colonnes

- Employé
- Période
- Statut
- Total heures
- Dernière soumission
- Validateur
- Dernière action

Statuts possibles

- Brouillon
- Soumis
- En validation N1
- En validation N2
- Validé
- Rejeté
- Clôturé
- Réouvert

Règles de transition

- Brouillon → Soumis
- Soumis → Validé ou Rejeté
- Validé → Clôturé lors de la clôture de période
- Clôturé → Réouvert uniquement avec droit spécifique

27.7.3 Écran – Validation manager

Informations affichées

- Détail de la feuille
- Écarts par rapport au planning
- Absences de la période
- Totaux par type d'heure
- Historique des actions

Actions

- Valider
- Rejeter
- Demander modification
- Commenter

Règles de gestion

- Un rejet nécessite un commentaire.
- Le validateur ne peut valider que les ressources de son périmètre.
- Une validation tardive après la date limite peut être signalée.

27.7.4 Règles spécifiques heures supplémentaires

- Les heures au-delà du temps contractuel ou du seuil journalier/hebdomadaire peuvent être automatiquement proposées comme heures supplémentaires.
- Un workflow d'approbation spécifique peut être imposé.
- Une heure supplémentaire refusée ne doit pas alimenter la paie majorée.

27.8 Module Usage matériel

27.8.1 Écran – Saisie usage matériel

Champs obligatoires

- Date
- Matériel
- Projet
- Tâche
- Unité consommée
- Quantité

Champs optionnels

- Opérateur
- Commentaire
- Justificatif

Calculs automatiques

- Coût unitaire récupéré depuis la fiche matériel
- Coût total = quantité x coût unitaire

Règles

- Le matériel doit être actif et disponible.
- Le projet doit être imputable sur la période.
- Les quantités négatives sont interdites.

27.8.2 Écran – Validation usage matériel

Colonnes

- Date
- Matériel
- Projet
- Quantité
- Coût
- Déclarant
- Statut

Actions

- Valider
- Rejeter
- Corriger si droit spécifique

Statuts

- Brouillon
- Soumis
- Validé
- Rejeté
- Clôturé

27.9 Module Absences & Congés

27.9.1 Écran – Demande d’absence

Champs obligatoires

- Employé
- Type d’absence
- Date début
- Date fin
- Demi-journée oui/non si applicable
- Commentaire

Champs conditionnels

- Justificatif pour maladie ou absence réglementée
- Motif obligatoire pour congé sans solde

Règles de gestion

- La période doit être cohérente.
- Les soldes sont contrôlés pour les types à quota.
- Le chevauchement avec une absence déjà validée est interdit.

Statuts

- Brouillon
- Soumis
- Validé
- Refusé
- Annulé

27.9.2 Écran – Validation absence

Affichages utiles

- Période demandée
- Solde avant/après
- Impact sur planning d’équipe
- Affectations projet concernées

Actions

- Valider
- Refuser
- Demander modification

Règles

- Une absence validée impacte immédiatement la capacité.
- Les affectations existantes restent visibles mais signalées.

27.9.3 Écran – Solde de congés

Colonnes

- Type d’absence
- Solde acquis
- Solde consommé
- Solde restant
- Période de référence

27.10 Module Paie préparatoire

27.10.1 Écran – Liste des périodes de paie

Colonnes

- Période
- Société
- Statut
- Nombre de salariés concernés
- Date de calcul
- Date de clôture
- Dernière version export

Statuts

- Ouverte
- En calcul
- Calculée
- En contrôle
- Validée

- Exportée
- Clôturée
- Réouverte

27.10.2 Écran – Calcul d’une période

Paramètres

- Société
- Date début
- Date fin
- Inclure uniquement validé oui/non
- Lancer recalcul complet ou incrémental

Résultats affichés

- Nombre de salariés calculés
- Nombre d’anomalies
- Totaux heures normales
- Totaux heures supplémentaires
- Totaux absences payées/non payées
- Nombre de primes/indemnités générées

Règles de gestion

- Seules les données validées à date sont intégrées.
- Le recalcul incrémental ne doit reprendre que les données modifiées depuis le dernier calcul.
- Chaque exécution crée un journal de calcul.

27.10.3 Écran – Contrôle des variables de paie

Colonnes

- Employé
- Heures normales
- Heures supplémentaires
- Nuit / week-end / férié
- Absences payées
- Absences non payées

- Primes
- Retenues
- Statut anomalie

Filtres

- Employé
- Département
- Type d’anomalie
- Statut de traitement

Types d’anomalies

- Timesheet manquante
- Temps/absence incohérent
- Dépassement plafond
- Taux manquant
- Contrat manquant
- Validation manquante

27.10.4 Écran – Export paie

Paramètres

- Société
- Période
- Format d’export
- Version
- Inclure uniquement salariés sans anomalie critique

Sorties

- Fichier exportable
- Journal export
- Historique versions

Règles

- Un export génère un identifiant unique.
- Un export ne supprime jamais l’historique précédent.
- Les anomalies critiques peuvent bloquer l’export selon paramétrage.

27.11 Module Reporting & Pilotage

27.11.1 Dashboard manager

Widgets

- Taux de soumission des timesheets
- Taux de validation
- Ressources absentes cette semaine
- Ressources en surcharge
- Projets en dépassement d’heures

Filtres

- Période
- Société
- Site
- Équipe
- Projet

27.11.2 Dashboard direction / contrôle de gestion

Widgets

- Coût RH par projet
- Coût matériel par projet
- Budget vs réel
- Taux d’occupation RH
- Taux d’utilisation matériel
- Anomalies de clôture

27.11.3 Audit

Éléments auditables

- Création/modification employé
- Modification contrat
- Affectation RH/matériel
- Soumission/validation/rejet timesheet
- Demande/validation absence

- Calcul période paie
- Export paie
- Changement de droits

Colonnes

- Date/heure
- Utilisateur
- Action
- Objet
- Identifiant objet
- Ancienne valeur
- Nouvelle valeur
- Source

27.12 Notifications

Notifications minimum du MVP

- Rappel de saisie timesheet avant échéance
- Confirmation de soumission timesheet
- Validation ou rejet timesheet
- Soumission demande d’absence
- Validation ou refus d’absence
- Alerte surcharge ressource
- Alerte matériel indisponible
- Fin de calcul paie préparatoire
- Export paie réalisé

Canaux

- In-app
- Email

Règles

- Les notifications doivent être contextualisées.
- L’utilisateur doit pouvoir accéder directement à l’objet concerné.

27.13 Règles transverses de validation

Données

- Les champs obligatoires dépendent du paramétrage société.
- Toute donnée référentielle critique est versionnée ou historisée.

Dates

- Les dates doivent être cohérentes avec la période active.
- Les actions sur période clôturée sont restreintes.

Sécurité

- Toute action est filtrée par rôle et périmètre.
- Les exports sensibles sont réservés aux profils autorisés.

Traçabilité

- Toute validation, réouverture, rejet ou export doit être tracé.

27.14 Matrice des statuts principaux

Objet	Statuts
Employé	Brouillon, Actif, Suspendu, En congé longue durée, Sorti
Matériel	Disponible, Affecté, Maintenance, Indisponible, Hors service
Projet	Brouillon, Actif, Suspendu, Clos
Timesheet	Brouillon, Soumis, En validation N1, En validation N2, Validé, Rejeté, Clôturé, Réouvert
Usage matériel	Brouillon, Soumis, Validé, Rejeté, Clôturé
Absence	Brouillon, Soumis, Validé, Refusé, Annulé
Période paie	Ouverte, En calcul, Calculée, En contrôle, Validée, Exportée, Clôturée, Réouverte

---

27.15 Cas limites à couvrir en recette

RH

- Employé sans manager

- Contrat manquant
- Deux contrats qui se chevauchent
- Taux horaire manquant

Temps

- Timesheet soumise avec dépassement journalier
- Saisie sur période clôturée
- Saisie sur journée d'absence validée
- Réouverture après validation

Matériel

- Affectation sur matériel en maintenance
- Usage saisi sur matériel hors service
- Conflit de réservation sur la même période

Paie

- Calcul sans timesheet validée
- Employé avec absence non payée et heures déclarées le même jour
- Export avec anomalies critiques
- Recalcul après correction d'une donnée source

27.16 Exigences QA / Recette

Jeux de tests minimaux

- Cycle complet employé → affectation → timesheet → validation → paie
- Cycle complet matériel → affectation → usage → validation → coût projet
- Cycle absence → validation → impact capacité → impact paie
- Cycle recalcul paie après correction
- Contrôle permissions selon rôle

Critères de recette transverses

- Aucun utilisateur ne voit de données hors périmètre autorisé.
- Les totaux calculés sont cohérents entre écrans détail et synthèse.
- Les exports correspondent aux données validées affichées.
- Toute action critique laisse une trace d'audit.

27.17 Recommandation de livrables suivants

Après cette spécification fonctionnelle détaillée, les livrables les plus utiles sont :

1. **Maquettes UX / wireframes écran par écran**
2. **Spécifications techniques détaillées**
3. **Schéma de base de données relationnel**
4. **Contrats API détaillés**
5. **Plan de tests fonctionnels**

---

28. Intégration dans la base de données CashPilot

28.1 Objectif

La base actuelle de CashPilot contient déjà des briques utiles pour supporter une partie du périmètre RH, projet, analytique et facturation :

- company
- clients
- projects
- tasks
- timesheets
- team_members
- project_resource_allocations
- team_member_compensations
- cost_centers
- accounting_entries
- analytical_objects
- analytical_allocations
- accounting_fixed_assets
- accounting_depreciation_schedule
- notifications
- audit_log

Cependant, le modèle actuel n'est pas suffisant pour une gestion RH et matérielle complète, pour trois raisons principales :

1. il mélange la notion d'utilisateur applicatif, de membre projet et de collaborateur RH ;
2. il ne couvre pas correctement les contrats, calendriers, absences, cycles de paie et variables de paie ;
3. il ne modélise pas le parc matériel opérationnel, ses disponibilités, ses affectations détaillées et ses usages réels.

La stratégie recommandée est donc :

- **conserver** les tables projet/comptabilité existantes,
- **étendre** le modèle avec un domaine RH et un domaine matériel,
- **réutiliser** les
  tables projects, tasks, timesheets, cost_centers, accounting_entries, analytical_objects,
- **éviter** de surcharger team_members pour des besoins RH complets.

28.2 Analyse de l'existant réutilisable

Tables réutilisables telles quelles

- company : société légale / entité de rattachement
- clients : client de projet
- projects : projet maître
- tasks : tâches ou activités projet
- cost_centers : centre de coût analytique
- accounting_entries : écritures comptables
- analytical_objects : objet analytique projet / service / custom
- analytical_allocations : ventilation analytique
- notifications : notifications in-app
- audit_log : journal transverse minimal

Tables à faire évoluer

- timesheets : trop orientée saisie simple, insuffisante pour timesheets RH multi-statut et préparation paie
- team_members : utile pour l'affectation projet légère, mais insuffisante pour dossier salarié, contrat, coût, planning, absences
- project_resource_allocations : utile comme base de planification, mais doit être normalisée pour les ressources humaines et matérielles

- team_member_compensations : utile comme point de départ, mais insuffisante pour un cycle de paie préparatoire complet
- accounting_fixed_assets : utile pour immobilisations comptables, mais pas pour la gestion opérationnelle de matériel affectable

Problèmes techniques visibles dans le schéma existant

Certaines tables présentent des contraintes dupliquées ou incohérentes, notamment autour de projects, clients, timesheets, team_member_compensations, project_milestones, project_baselines. Pour l'intégration RH/matériel, il est recommandé de :

- ne pas reproduire ce pattern,
- simplifier les clés étrangères,
- utiliser des FK explicites vers les identifiants métiers corrects,
- éviter les pseudo-contrôles de scope par multiplications de FK.

28.3 Stratégie de modélisation recommandée

Principe 1 – Séparer identité applicative, collaborateur RH et membre projet

- auth.users = identité technique de connexion
- profiles = profil utilisateur applicatif
- employees = fiche RH complète
- team_members = vue projet simplifiée ou couche de compatibilité

Principe 2 – Conserver projects **/** tasks **comme référentiel projet principal**

Les nouveaux modules RH et matériels doivent pointer vers :

- projects.id
- tasks.id
- company.id
- cost_centers.id

Principe 3 – Distinguer matériel opérationnel et immobilisation comptable

- material_assets = parc opérationnel affectable
- accounting_fixed_assets = immobilisations et amortissements comptables
- un lien facultatif peut exister entre les deux

Principe 4 – Séparer temps opérationnel, temps validé, temps payé

- timesheets peut être conservée pour compatibilité ou facturation simple

- les besoins RH doivent être portés par un nouveau modèle :
    - hr_timesheet_periods
    - hr_timesheet_lines
    - hr_timesheet_approvals
    - payroll_variable_items

28.4 Nouveau domaine RH proposé

28.4.1 Table hr_departments

But: gérer les départements RH indépendamment des équipes projet.

28.4.2 Table hr_work_calendars

But: gérer les calendriers de travail, heures théoriques, jours travaillés.

28.4.3 Table hr_employees

But: fiche salarié complète.

28.4.4 Table hr_employee_contracts

But: contrats, avenants, coûts, dates d'effet.

28.4.5 Table hr_employee_skills

But: compétences et certifications.

28.4.6 Table hr_leave_types

But: typologie des absences.

28.4.7 Table hr_leave_requests

But: demandes d'absence et validation.

28.4.8 Table hr_timesheet_periods

But: feuille de temps RH agrégée par période et salarié.

28.4.9 Table hr_timesheet_lines

But: lignes de temps détaillées par date / projet / tâche / type d'heure.

28.4.10 Table hr_timesheet_approvals

But: workflow de validation.

28.4.11 Table hr_payroll_periods

But: cycle de paie préparatoire.

28.4.12 Table hr_payroll_variable_items

But: variables de paie générées depuis temps/absences/règles.

28.4.13 Table hr_payroll_exports

But: historiser les exports de paie.

28.5 Nouveau domaine matériel proposé

28.5.1 Table material_categories

But: catégorisation du parc matériel.

28.5.2 Table material_assets

But: référentiel matériel opérationnel.

28.5.3 Table material_maintenance_windows

But: fenêtres de maintenance / indisponibilité.

28.5.4 Table material_assignments

But: affectation planifiée du matériel à un projet / tâche.

28.5.5 Table material_usage_logs

But: usage réel déclaré par période / projet / tâche.

28.5.6 Table material_usage_approvals

But: validation des usages matériels.

28.6 Relations avec l'existant

RH vers existant

- hr_employees.company_id → company.id
- hr_employees.user_id → auth.users.id (optionnel)
- hr_employees.manager_employee_id → hr_employees.id
- hr_employees.department_id → hr_departments.id
- hr_employees.cost_center_id → cost_centers.id
- hr_employee_contracts.employee_id → hr_employees.id
- hr_timesheet_lines.project_id → projects.id
- hr_timesheet_lines.task_id → tasks.id
- hr_timesheet_lines.cost_center_id → cost_centers.id

Matériel vers existant

- material_assets.company_id → company.id
- material_assets.cost_center_id → cost_centers.id
- material_assets.linked_fixed_asset_id → accounting_fixed_assets.id (optionnel)
- material_assignments.project_id → projects.id
- material_assignments.task_id → tasks.id
- material_usage_logs.project_id → projects.id
- material_usage_logs.task_id → tasks.id

Paie / analytique / comptabilité

- hr_payroll_variable_items.employee_id → hr_employees.id
- hr_payroll_variable_items.payroll_period_id → hr_payroll_periods.id
- hr_payroll_variable_items.source_timesheet_line_id → hr_timesheet_lines.id
- hr_payroll_variable_items.source_leave_request_id → hr_leave_requests.id
- journalisation comptable potentielle dans accounting_entries
- ventilation analytique potentielle dans analytical_allocations

---

28.7 Décision recommandée concernant team_members

Option recommandée

Conserver team_members comme entité légère de collaboration projet, mais introduire hr_employees comme vérité métier RH.

Mapping recommandé

- team_members.user_id = utilisateur propriétaire du tenant/app
- ajouter team_members.employee_id nullable → hr_employees.id
- ajouter team_members.company_id → company.id
- conserver assigned_member_id dans tasks pour compatibilité projet
- à terme, privilégier les affectations via hr_employees

Pourquoi

Le modèle team_members actuel ne couvre pas :

- contrat,
- calendrier,
- statut RH,

- coût employeur,
- absences,
- paie,
- validation hiérarchique.

28.8 Décision recommandée concernant timesheets

Constats

La table timesheets actuelle est utile pour :

- facturation,
- preuve simple d'activité,
- lien vers invoice_items.

Mais elle est insuffisante pour :

- workflow de validation multi-niveaux,
- distinction heures normales/supplémentaires/nuit,
- clôture mensuelle,
- corrections contrôlées,
- préparation de paie.

Recommandation

Conserver timesheets pour la compatibilité de facturation, mais créer le modèle RH suivant :

- hr_timesheet_periods
- hr_timesheet_lines
- hr_timesheet_approvals

Synchronisation possible

Une vue ou un job peut alimenter timesheets depuis hr_timesheet_lines pour les heures facturables uniquement.

28.9 Décision recommandée concernant project_resource_allocations

Constats

La table actuelle mélange :

- humain / matériel,

- planifié / réel,
- coût / quantité,
- et référence team_members seulement côté humain.

Recommandation

Deux options possibles :

Option A – Évolution contrôlée

Conserver project_resource_allocations comme table d'agrégat / reporting planning et créer :

- hr_project_assignments
- material_assignments

Option B – Refonte

Transformer project_resource_allocations en supertype plus propre avec :

- resource_type
- employee_id
- material_asset_id
- task_id
- planned_quantity
- planned_unit
- planned_cost_rate

Recommandation finale

Prendre **Option A**. Elle est plus sûre, plus lisible et mieux adaptée à la séparation RH / matériel.

28.10 Extensions de schéma recommandées (DDL cible)

```
-- ========================
-- RH - Temps
-- ========================


create table public.hr_timesheet_periods (
  id uuid primary key default gen_random_uuid(),
  company_id uuid not null references public.company(id),
```

```
  employee_id uuid not null references public.hr_employees(id),

  period_start date not null,

  period_end date not null,

  period_type text not null check (period_type in ('weekly','monthly')),

  status text not null default 'draft' check (status in
('draft','submitted','approved_l1','approved_l2','validated','rejected','closed','reopened')),

  submitted_at timestamptz,

  validated_at timestamptz,

  created_at timestamptz not null default now(),

  updated_at timestamptz not null default now(),

  unique (company_id, employee_id, period_start, period_end)

);


create table public.hr_timesheet_lines (

  id uuid primary key default gen_random_uuid(),

  company_id uuid not null references public.company(id),

  timesheet_period_id uuid not null references public.hr_timesheet_periods(id) on delete
cascade,

  employee_id uuid not null references public.hr_employees(id),

  work_date date not null,

  project_id uuid references public.projects(id),

  task_id uuid references public.tasks(id),

  cost_center_id uuid references public.cost_centers(id),

  service_id uuid references public.services(id),

  line_type text not null check (line_type in
('work','overtime','night','weekend','holiday','absence','on_call','travel','non_productive')),

  duration_minutes integer not null check (duration_minutes >= 0),

  billable boolean not null default false,

  billable_rate numeric,

  notes text,

  source_leave_request_id uuid references public.hr_leave_requests(id),
```

```
  legacy_timesheet_id uuid references public.timesheets(id),

  status text not null default 'draft' check (status in
('draft','submitted','approved','rejected','closed')),

  created_at timestamptz not null default now(),

  updated_at timestamptz not null default now()

);


create table public.hr_timesheet_approvals (

  id uuid primary key default gen_random_uuid(),

  company_id uuid not null references public.company(id),

  timesheet_period_id uuid not null references public.hr_timesheet_periods(id) on delete
cascade,

  approval_level integer not null,

  approver_employee_id uuid references public.hr_employees(id),

  decision text not null check (decision in ('submitted','approved','rejected','reopened')),

  comment text,

  decided_at timestamptz not null default now()

);


-- ========================

-- RH - Paie préparatoire

-- ========================


create table public.hr_payroll_periods (

  id uuid primary key default gen_random_uuid(),

  company_id uuid not null references public.company(id),

  period_start date not null,

  period_end date not null,

  status text not null default 'open' check (status in
('open','calculating','calculated','under_review','validated','exported','closed','reopened')),

  calculation_version integer not null default 1,
```

```
  calculated_at timestamptz,
  validated_at timestamptz,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  unique (company_id, period_start, period_end, calculation_version)
);

create table public.hr_payroll_variable_items (
  id uuid primary key default gen_random_uuid(),
  company_id uuid not null references public.company(id),
  payroll_period_id uuid not null references public.hr_payroll_periods(id) on delete cascade,
  employee_id uuid not null references public.hr_employees(id),
  item_code text not null,
  item_label text not null,
  item_category text not null check (item_category in
('normal_hours','overtime','night','weekend','holiday','bonus','allowance','deduction','unpaid_leav
e','other')),
  quantity numeric,
  rate numeric,
  amount numeric not null default 0,
  currency varchar(3) not null default 'EUR',
  source_timesheet_line_id uuid references public.hr_timesheet_lines(id),
  source_leave_request_id uuid references public.hr_leave_requests(id),
  metadata jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table public.hr_payroll_exports (
  id uuid primary key default gen_random_uuid(),
```

```
  company_id uuid not null references public.company(id),

  payroll_period_id uuid not null references public.hr_payroll_periods(id),

  export_format text not null,

  export_status text not null default 'generated' check (export_status in
('generated','downloaded','transmitted','cancelled')),

  version integer not null default 1,

  file_url text,

  generated_by uuid references auth.users(id),
```

28.11 Évolutions ciblées sur tables existantes

team_members

```
alter table public.team_members

  add column if not exists company_id uuid references public.company(id),

  add column if not exists employee_id uuid references public.hr_employees(id);
```

projects

```
alter table public.projects

  add column if not exists project_code text,

  add column if not exists cost_center_id uuid references public.cost_centers(id),

  add column if not exists project_manager_employee_id uuid references
public.hr_employees(id),

  add column if not exists budget_amount numeric,

  add column if not exists currency varchar(3) default 'EUR';
```

tasks

```
alter table public.tasks

  add column if not exists task_code text,

  add column if not exists billable boolean not null default true,

  add column if not exists imputable boolean not null default true,

  add column if not exists cost_center_id uuid references public.cost_centers(id);
```

timesheets

```
alter table public.timesheets
```

```
add column if not exists executed_by_employee_id uuid references public.hr_employees(id),
add column if not exists timesheet_period_id uuid references public.hr_timesheet_periods(id),
add column if not exists line_type text,
add column if not exists approval_status text;
```

28.12 Flux de données recommandés

Flux 1 – Saisie temps RH

1. L'employé est créé dans hr_employees.
2. Sa période est créée dans hr_timesheet_periods.
3. Les lignes sont enregistrées dans hr_timesheet_lines.
4. Les validations passent par hr_timesheet_approvals.
5. Les lignes facturables peuvent alimenter timesheets ou invoice_items.
6. Les lignes validées alimentent hr_payroll_variable_items.

Flux 2 – Absence

1. L'absence est saisie dans hr_leave_requests.
2. Après validation, elle impacte le planning.
3. Si elle est non payée ou spécifique, elle génère des items de paie.
4. Des lignes hr_timesheet_lines de type absence peuvent être générées automatiquement.

Flux 3 – Matériel

1. Le matériel est référencé dans material_assets.
2. Son planning de maintenance est stocké dans material_maintenance_windows.
3. Ses réservations projet sont gérées dans material_assignments.
4. Son usage réel est saisi dans material_usage_logs.
5. Les coûts peuvent alimenter project_resource_allocations, analytical_allocations ou accounting_entries.

Flux 4 – Paie préparatoire

1. La période est ouverte dans hr_payroll_periods.
2. Les temps validés et absences approuvées sont consolidés.
3. Les variables sont générées dans hr_payroll_variable_items.

4. L’export est historisé dans hr_payroll_exports.

28.13 Impacts sur la facturation

La base actuelle permet déjà la facturation à partir
de timesheets via invoice_items.timesheet_id.

Recommandation

- conserver le lien existant pour la facturation,
- n’utiliser hr_timesheet_lines comme source métier,
- synchroniser vers timesheets uniquement les lignes :
    - facturables,
    - validées,
    - rattachées à un projet/client/tâche.

Bénéfice

Cela évite de casser la logique existante de facturation tout en ajoutant une vraie couche RH.

28.14 Impacts sur la comptabilité et l’analytique

Coûts RH

Les variables de paie ou coûts RH consolidés peuvent être ventilés :

- dans accounting_entries pour journal de paie,
- dans analytical_allocations pour coût projet,
- dans cost_centers pour ventilation organisationnelle.

Coûts matériels

Les coûts calculés dans material_usage_logs peuvent alimenter :

- analytical_allocations,
- accounting_entries,
- ou un agrégat de performance projet.

Recommandation analytique

Créer un analytical_objects de type project pour chaque projet s’il n’existe pas déjà, afin de rattacher proprement les coûts RH et matériels aux mêmes objets analytiques.

28.15 Plan de migration recommandé

Phase 1 – Stabilisation du schéma existant

- corriger ou isoler les FK incohérentes dans les tables projet/timesheet existantes,
- documenter les usages réels
  de team_members, timesheets, project_resource_allocations,
- identifier les données legacy à conserver.

Phase 2 – Ajout du domaine RH

- créer tables hr_*,
- migrer progressivement les membres projet vers hr_employees lorsque pertinent,
- ajouter les mappings vers team_members.

Phase 3 – Ajout du domaine matériel

- créer tables material_*,
- relier certains matériels à accounting_fixed_assets si nécessaire,
- implémenter affectation et usage réel.

Phase 4 – Paie préparatoire

- implémenter calcul des variables,
- mettre en place exports,
- ajouter journalisation et contrôles d'anomalies.

Phase 5 – Synchronisation facturation / analytique

- alimentation contrôlée de timesheets,
- alimentation analytique et comptable.

28.16 Décisions d'architecture à valider

1. hr_employees remplace-t-elle à terme team_members pour l'affectation projet ?
2. timesheets reste-t-elle une table de facturation uniquement ?
3. Souhaitez-vous gérer uniquement la **paie préparatoire** ou aussi les bulletins ?
4. Les matériels suivis doivent-ils inclure seulement les équipements affectables ou aussi les consommables/stocks ?
5. Les coûts RH doivent-ils provenir du contrat, d'un barème, ou d'un coût standard analytique ?

---

29. Conclusion

L'intégration de la gestion RH et matérielle dans CashPilot doit se faire par **extension contrôlée du schéma**, et non par surcharge des tables existantes.

La meilleure approche est de :

- **conserver** le noyau projet/facturation/comptabilité actuel,
- **ajouter** un domaine RH structuré,
- **ajouter** un domaine matériel opérationnel,
- **connecter** ces domaines
  à projects, tasks, cost_centers, timesheets, accounting_entries et analytical_allocations.

Cette approche limite les risques de régression, facilite la migration et prépare une trajectoire propre vers :

- planning,
- timesheets RH avancées,
- paie préparatoire,
- suivi matériel,
- coût complet projet.

---

30. Recommandation immédiate

La prochaine étape la plus utile est de produire, à partir de cette cible :

1. un **schéma relationnel final normalisé**,
2. un **script SQL de migration par étapes**,
3. un **mapping legacy → nouveau modèle**,
4. puis les **APIs backend** correspondantes.

Merci — avec ce schéma, on peut déjà définir une **stratégie d'intégration propre**.

Ce que j'ai fait

J'ai ajouté au document une section complète sur :

- l'**analyse du schéma CashPilot existant**
- les tables **réutilisables**
- les tables à **faire évoluer**
- la **stratégie d'intégration RH et matérielle**
- le **modèle cible recommandé**

- les **relations avec l'existant**
- un **DDL SQL proposé** pour les nouvelles tables
- les **altérations ciblées** sur projects, tasks, timesheets, team_members
- un **plan de migration par phases**

Conclusion principale

La bonne approche n'est pas de forcer la RH complète dans les tables actuelles.

Il faut :

Conserver

- company
- projects
- tasks
- clients
- cost_centers
- timesheets pour la compatibilité facturation
- accounting_entries
- analytical_allocations
- accounting_fixed_assets

Ajouter

un vrai domaine métier dédié :

RH

- hr_employees
- hr_employee_contracts
- hr_work_calendars
- hr_leave_requests
- hr_timesheet_periods
- hr_timesheet_lines
- hr_timesheet_approvals
- hr_payroll_periods
- hr_payroll_variable_items
- hr_payroll_exports

Matériel

- material_categories
- material_assets
- material_maintenance_windows
- material_assignments
- material_usage_logs
- material_usage_approvals

Pourquoi c'est la meilleure option

Parce que votre schéma actuel couvre déjà :

- le projet,
- la facturation,
- l'analytique,
- la comptabilité,
- les immobilisations,

mais **pas correctement** :

- le dossier salarié,
- les contrats,
- les absences,
- les workflows de validation RH,
- les cycles de paie,
- le parc matériel opérationnel,
- les usages matériels réels.

Point important sur votre schéma actuel

J'ai aussi identifié un point d'attention important :
plusieurs tables existantes contiennent des **FK redondantes ou incohérentes**, surtout autour de :

- projects
- timesheets
- team_member_compensations
- project_milestones

- project_baselines

Pour la suite, il faudra éviter de reproduire ce pattern.

Recommandation immédiate

La suite la plus utile maintenant est de produire l'un de ces livrables opérationnels :

Option 1 — Schéma SQL final normalisé

Un modèle propre, prêt à être implémenté dans PostgreSQL.

Option 2 — Script de migration SQL

Avec :

- création des nouvelles tables,
- ALTER TABLE sur l'existant,
- stratégie de compatibilité.

Option 3 — Mapping legacy → nouveau modèle

Pour savoir exactement :

- quoi garder,
- quoi déprécier,
- quoi synchroniser.

Je recommande de commencer par le **script SQL de migration par étapes**, car il vous donnera un plan d'implémentation concret.